

程序的转换及机器级表示

计算机的指令有微指令、机器指令和伪（宏）指令之分。微指令是微程序级命令，属于硬件范畴；伪指令是由若干机器指令组成的指令序列，属于软件范畴；机器指令介于二者之间，处于硬件和软件的交界面。本章中提及的指令指机器指令，机器指令的汇编语言表示形式为汇编指令。机器指令和汇编指令一一对应，它们都与具体机器结构有关，都属于机器级指令。

本章主要介绍 C 语言程序与 IA-32 机器级指令之间的对应关系。主要内容包括：程序转换概述、IA-32 指令系统、C 语言中的过程调用和各类控制语句的机器级实现、复杂数据类型（数组、结构等）的机器级实现等。本章所用的机器级表示主要以汇编语言形式表示为主。

本章中多处需要对指令功能进行描述，为简化对指令功能的说明，将采用寄存器传送语言（Register Transfer Language, RTL）来说明。

本书 RTL 规定： $R[r]$ 表示寄存器 r 中内容， $M[addr]$ 表示存储单元 $addr$ 中内容，寄存器 r 采用不带 % 的形式表示。若要表示 PC 所指存储单元的内容，则用 $M[PC]$ 表示。若要表示寄存器间接寻址操作（即读取寄存器 r 的内容所指的存储单元的内容），则用 $M[R[r]]$ 表示。传送方向用 $\leftarrow$ 表示，即传送源在右，传送目的在左。例如，对于指令 “movw 4(%ebp), %ax”，其功能为 $R[ax] \leftarrow M[R[ebp] + 4]$ ，含义是：将寄存器 EBP 的内容和 4 相加得到的地址对应的两个连续存储单元中的内容送到寄存器 AX 中。

本书中寄存器名称的书写约定如下：寄存器的名称若出现在单独一行的汇编指令或寄存器传送语言中则用小写表示，若出现在正文段落或其他部分则用大写表示。

本书对汇编指令或汇编指令名称的书写约定如下：具体一条汇编指令或指令名称用小写表示，但在泛指某一类指令的指令类别名称时用大写表示。

3.1 程序转换概述

计算机硬件只能识别和理解机器语言程序，用各种汇编语言或高级语言编写的源程序都要翻译（汇编、解释或编译）成以机器指令形式表示的机器语言才能在计算机上执行。通常对于编译执行的程序来说，都是先将高级语言源程序通过编译器转换为汇编语言目标程序，然后将汇编语言源程序通过汇编程序转换为机器语言目标程序。

3.1.1 机器指令及汇编指令

机器语言程序是一个由若干条机器指令组成的序列。每条机器指令由若干字段组成，例如，操作码字段用来指出指令的操作性质，立即数字段用来指出操作数或偏移量，寄存器编号字段给出操作数或操作数地址所在的寄存器编号等。每个字段都是一串由 0 和 1 组成的二进制数字序列，例如，在 MIPS 指令中，操作码字段为 100011 时表示“load word”指令，操作码字段为 000010 时表示“jump”指令。因此，机器指令实际上就是一个 0/1 序列，即位串。因为人类很难记住这些位串的含义，所以机器指令的可读性很差。

为了能直观地表示机器语言程序，引入了一种与机器语言一一对应的符号化表示语言，称为汇编语言。在汇编语言中，通常用容易记忆的英文单词或缩写来表示指令操作码的含义，用标号、变量名称、寄存器名称、常数等表示操作数或地址码。这些英文单词或其缩写、标号、变量名称等都被称为汇编助记符。用若干个助记符表示的与机器指令一一对应的指令称为汇编指令，用汇编语言编写的程序称为汇编语言程序，因此，汇编语言程序主要是由汇编指令构成的。

100010 DW	mod	reg	r/m	disp8
100010 0 0	01	001	001	11111010

图 3.1 机器指令举例

对于如图 3.1 所示的 8086/8088 中的机器指令“88 49 FAH”，其指令格式包含有若干字段，每个字段对应

不同的含义。其中，开始的 6 位“100010”表示是 mov 指令；位 D 表示 reg 字段给出的是否为目的操作数，D=1 说明 reg 字段给出的是目的操作数，否则是源操作数；位 W 表示操作数的宽度，W=0 时为 8 位，W=1 时为 16 位；mod 字段表示寻址方式；reg 字段是源或目的操作数所在的寄存器编号；r/m 字段给出源或目的操作数所在寄存器编号或有效地址计算方式；disp8 给出在有效地址计算中用到的 8 位位移量。

根据指令字段的划分可知，在图 3.1 中，D=0，W=0，mod=01，reg=001，r/m=001，disp8=11111010。说明该指令的操作数为 8 位，reg 指出的是源操作数。目的操作数的有效地址由 mod 和 r/m 两个字段组合确定。根据寄存器编号查表可知，001 是寄存器 CL 的编号，根据 mod=01 且 r/m=001 的情况查表可知，目的操作数的有效地址为 $R[bx] + R[di] + disp8$ 。根据 disp8 字段为 1111 1010 可知，位移量 disp8 的值为 $-110B = -6$ 。因此，对应的汇编指令表示为“mov [bx + di - 6], cl”，其功能为 $M[R[bx] + R[di] - 6] \leftarrow R[cl]$ ，也即，将 CL 寄存器的内容传送到一个存储单元中，该存储单元的偏移地址计算方法为 BX 和 DI 两个寄存器的内容相加再减 6。这里，汇编指令中的 mov、bx、di、cl 等都是助记符。

显然，对于人类来说，明白汇编指令的含义比弄懂机器指令中的一串二进制数字要容易得多。但是，对于计算机硬件来说，情况却相反，计算机硬件不能直接执行汇编指令而只能执行机器指令。用来将汇编语言源程序中的汇编指令翻译成机器指令的程序称为汇编程序。而将机器指令反过来翻译成汇编指令的程序称为反汇编程序。

机器语言和汇编语言统称为机器级语言。用机器指令表示的机器语言程序和用汇编指令表示的汇编语言程序统称为机器级程序，它是对应高级语言程序的机器级表示。任何一个高级语言程序一定存在一个与之对应的机器级程序，而且是不唯一的。因此，如何将高级语言程序生成对应的机器级程序并在时间和空间上达到最优，是编译优化要解决的问题。

3.1.2 指令集体系结构

第1章详细介绍了计算机系统的层次结构，说明了计算机系统是由多个不同的抽象层构成的，每个抽象层的引入，都是为了对它的上层屏蔽或隐藏其下层的实现细节，从而为其上层提供简单的使用接口。在计算机系统的抽象层中，最重要的抽象层就是指令集体系结构 (Instruction Set Architecture, ISA)，它作为计算机硬件之上的抽象层，对使用硬件的软件屏蔽了底层硬件的实现细节，将物理上的计算机硬件抽象成一个逻辑上的虚拟计算机，称为机器语言级虚拟机。

ISA 定义了机器语言级虚拟机的属性和功能特性，主要包括如下信息。

- 可执行的指令的集合，包括指令格式、操作种类以及每种操作对应的操作数的相应规定；
- 指令可以接受的操作数的类型；
- 操作数所能存放的寄存器组的结构，包括每个寄存器的名称、编号、长度和用途；
- 操作数所能存放的存储空间的大小和编址方式；
- 操作数在存储空间存放时按照大端方式还是小端方式存放；
- 指令获取操作数的方式，即寻址方式；
- 指令执行过程的控制方式，包括程序计数器、条件码定义等。

ISA 规定了机器级程序的格式和行为，也就是说，ISA 属于软件看得见（即能感觉到）的特性。用机器指令或汇编指令编写机器级程序的程序员必须对运行该程序的机器的 ISA 非常熟悉。不过，在工作中大多数程序员不用汇编指令编写程序，更不会用机器指令编写程序。大多数情况下，程序员用抽象层更高的高级语言（如 C/C++、Java）编写程序，这样程序开发效率会更高，也更容易出错。高级语言程序在机器硬件上执行之前，由编译器将其转换为机器级程序，并在转换过程中进行语法检查、数据类型检查等工作，因而能帮助程序员发现许多错误。程序员现在大多用高级语言编写程序而不再直接编写机器级程序，似乎程序员不需要了解 ISA 和底层硬件的执行机理。但是，由于高级语言抽象层太高，隐藏了许多机器级程序的行为细节，使得高级语言程序员不能很好地利用与机器结构相关的一些优化方法来提升程序的性能，也不能很好地预见和防止潜在的安全漏洞或发现他人程序中的安全漏洞。如果程序员对 ISA 和底层硬件实现细节有充分的了解，则可以更好地编制高性能程序并避免程序的安全漏洞。有关这方面的情况，第2章内容中已经有过一些论述，在后续章节中将提供更多的例子来说明了解机器级编程的重要性。

3.1.3 生成机器代码的过程

1.3.1 节中曾描述了使用 GCC 工具将一个 C 语言程序转换为可执行目标代码的过程，图 1.7 给出了一个示例。通常，这个转换过程分为以下 4 个步骤。① 预处理。例如，在 C 语言源程序中有一些以 # 开头的语句，可以在预处理阶段对这些语句进行处理，在源程序中插入所有用 #include 命令指定的文件和用 #define 声明指定的宏。② 编译。将预处理后的源程序文件编译生成相应的汇编语言程序。③ 汇编。由汇编程序将汇编语言源程序文件转换为可重定位的机器语言目标代码文件。④ 链接。由链接器将多个可重定位的机器语言目标文件以及库例

程（如 `printf()` 库函数）链接起来，生成最终的可执行文件。

小贴士

GNU 是“GNU's Not Unix”的递归缩写。GNU 计划是由 Richard Stallman 在 1983 年 9 月 27 日公开发起的。它的目标是创建一套完全自由的类 UNIX 操作系统，其源代码可以被自由地“使用、复制、修改和发布”。GNU 包含 3 个协议条款，如 GNU 通用公共许可证（GNU General Public License, GPL）和 GNU 较宽松公共许可证（GNU Lesser General Public License, LGPL）。

1985 年 Richard Stallman 又创立了自由软件基金会（Free Software Foundation）来为 GNU 计划提供技术、法律以及财政支持。当 GNU 计划开始逐渐获得成功时，一些商业公司开始介入开发和技术支持。当中最著名的就是之后被 Red Hat 兼并的 Cygnus Solutions。到了 1990 年，GNU 计划已经开发出的软件包括了一个功能强大的文字编辑器 Emacs。1991 年 Linus Torvalds 编写出了与 UNIX 兼容的 Linux 操作系统内核并在 GPL 条款下发布。Linux 之后在网上广泛流传，许多程序员参与了开发与修改。1992 年 Linux 与其他 GNU 软件结合，完全自由的操作系统正式诞生。该操作系统往往被称为“GNU/Linux”或简称 Linux。

GCC（GNU Compiler Collection, GNU 编译器套件）是一套由 GNU 项目开发的编程语言编译器。它是一套以 GPL 及 LGPL 许可证所发行的自由软件，也是 GNU 计划的关键部分，是自由的类 UNIX 及苹果电脑 Mac OS X 操作系统的标准编译器。GCC 原名为 GNU C 语言编译器，因为它原本只能处理 C 语言。后来 GCC 扩展很快，可处理 C++、Fortran、Pascal、Objective-C、Java，以及 Ada 与其他语言。GCC 通常是跨平台软件的编译器首选。有别于一般局限于特定系统与执行环境的编译器，GCC 在所有平台上都使用同一个前端处理程序。

`gcc` 是 GCC 套件中的编译驱动程序名。C 语言编译器所遵循的部分约定规则为：源程序文件后缀名为 `.c`；源程序所包含的头文件后缀名为 `.h`；预处理过的源代码文件后缀名为 `.i`；汇编语言源程序文件后缀名为 `.s`；编译后的可重定位目标文件后缀名为 `.o`；最终生成的可执行目标文件可以没有后缀。

使用 `gcc` 编译器时，必须给出一系列必要的编译选项和文件名称，其编译选项有 100 多个，但是多数根本用不到。最基本的用法是：`gcc [-options] [filenames]`，其中 `[-options]` 指定编译选项，`filenames` 给出相关文件名。

`gcc` 可以基于不同的编译选项选择按照不同的 C 语言版本进行编译。因为 ANSI C 和 ISO C90 两个 C 语言版本一样，所以，编译选项 `-ansi` 和 `-std = C89` 的效果相同，目前是默认选项。C90 有时也称为 C89，因为 C90 的标准化工作是从 1989 年开始的。若指定编译选项 `-std = C99`，则会使 `gcc` 按照 ISO C99 的 C 语言版本进行编译。

下面以 C 编译器 `gcc` 为例来说明一个 C 语言程序被转换为可执行代码的过程。假定一个 C 程序包含两个源程序文件 `prog1.c` 和 `prog2.c`，最终生成的可执行文件为 `prog`，则可用以下命令一步到位生成最终的可执行文件。

```
gcc -O1 prog1.c prog2.c -o prog
```


该命令中的选项-o 指出输出文件名。编译选项-O1 表示采用最基本的第一级优化。通常，提高优化级别会得到更好的性能，但会使编译时间增长，而且使目标代码与源程序对应关系变得更复杂，从程序执行的性能来说，通常认为对应选项-O2 的第二级优化是最好的选择。本章的目的是建立高级语言源程序与机器级程序之间的对应关系，而没有优化过的机器级程序与源程序的对应关系最准确，所以，后面的例子都采用默认的优化选项-O0，即无任何编译优化。

也可以将上述完整的预处理、汇编、编译和链接过程，通过以下多个不同的编译选项命令分步骤进行。① 使用命令“gcc -E prog1.c -o prog1.i”，对 prog1.c 进行预处理，生成预处理结果文件 prog1.i；② 使用命令“gcc -S prog1.i -o prog1.s”或“gcc -S prog1.c -o prog1.s”，对 prog1.i 或 prog1.c 进行编译，生成汇编代码文件 prog1.s；③ 使用命令“gcc -c prog1.s -o prog1.o”，对 prog1.s 进行汇编，生成可重定位目标文件 prog1.o；④ 使用命令“gcc prog1.o prog2.o -o prog”，将两个可重定位目标文件 prog1.o 和 prog2.o 链接起来，生成可执行文件 prog。

gcc 编译选项具体的含义可使用命令 man gcc 进行查看。附录 B 给出了常用选项说明。

例 3.1 在 IA-32 + Linux 平台上，对下列源程序 test.c 使用 GCC 命令进行相应的处理，以分别得到预处理后的文件 test.i、汇编代码文件 test.s 和可重定位目标文件 test.o。这些输出文件中，哪些是可显示的文本文件？哪些是不能显示的二进制文件？请给出所有可显示文本文件的输出结果。

```
1 // test.c
2
3 int add(int i, int j)
4 {
5     int x = i + j;
6     return x;
7 }
```

解 使用命令“gcc -E test.c -o test.i”可生成 test.i；使用命令“gcc -S test.i -o test.s”可生成 test.s；使用命令“gcc -c test.s -o test.o”可生成 test.o。其中，可显示的文本文件有 test.i 和 test.s，而 test.o 是不可显示的二进制文件。

对于预处理后的文件 test.i，不同版本的 gcc 输出结果可能不同，gcc 4.4.7 版本输出的结果有 800 多行。因篇幅有限，在此省略其内容。

汇编代码文件 test.s 是可显示文本文件，其输出的部分结果如下：

```
.....
add:
    pushl   %ebp
    movl    %esp, %ebp
    subl    $16, %esp
    movl    12(%ebp), %eax
    movl    8(%ebp), %edx
    leal    (%edx, %eax), %eax
    movl    %eax, -4(%ebp)
    movl    -4(%ebp), %eax
    leave
    ret
```

对于不可显示的可重定位目标文件，如何查看其内容呢？从 3.1.1 节知道，反汇编程序能够将机器指令反过来翻译成汇编指令，因此，我们需要用反汇编工具来查看目标文件中的内容。在 Linux 中可以用带 -d 选项的 objdump 命令来对目标代码进行反汇编。如果需要进一步对机器级程序进行分析的话，可以用 GNU 调试工具 gdb 来跟踪和调试。

对于上述例 3.1 中的 test.o 程序，使用命令 “objdump -d test.o” 可以得到以下显示结果：

```
00000000 <add>:
0: 55          push    %ebp
1: 89 e5       mov     %esp, %ebp
3: 83 ec 10    sub     $0x10, %esp
6: 8b 45 0c    mov     0xc(%ebp), %eax
9: 8b 55 08    mov     0x8(%ebp), %edx
c: 8d 04 02    lea     (%edx,%eax,1), %eax
f: 89 45 fc    mov     %eax, -0x4(%ebp)
12: 8b 45 fc    mov     -0x4(%ebp), %eax
15: c9         leave  %eax
16: c3         ret
```

test.o 是可重定位目标文件，因而目标代码从相对地址 0 开始（如第 1 行所示），冒号前面的值表示每条指令相对于起始地址 0 的偏移量，冒号后面紧接着的是用十六进制表示的机器指令，右边是对应的汇编指令。从这个例子可以看出，每条机器指令的长度可能不同，例如，第一条指令只有一个字节，第二条指令是两个字节。说明 IA-32 的指令系统采用的是变长指令字结构，有关 IA-32 指令系统的细节将在 3.2 节中介绍。

将上述用 objdump 反汇编出来的汇编代码与直接由 gcc 汇编得到的汇编代码（test.s 输出结果）进行比较后可以发现，它们几乎完全相同，只是在数值形式和指令助记符的后缀等方面稍有不同。gcc 生成的汇编指令中用十进制形式表示数值，而 objdump 反汇编出来的汇编指令中则用十六进制形式表示数值。两者都以 \$ 开头表示一个立即数。gcc 生成的很多汇编指令助记符结尾中带有 ‘l’ 或 ‘w’ 等长度后缀，它是操作数长度指示符，这里 ‘l’ 表示指令中处理的操作数为双字，即 32 位，‘w’ 表示指令中处理的操作数为单字，即 16 位。对于 IA-32 来说，因为大多数情况下操作数都是 32 位，所以大多数情况下可以像 objdump 工具那样省略后缀 ‘l’。上述这种汇编格式称为 AT&T 格式，它是 objdump 和 gcc 使用的默认格式。

细心的读者可能发现，在 3.1.1 节介绍的一个关于 Intel 8086/8088 机器指令和汇编指令的例子中，汇编指令为 “mov [bx + di - 6], cl”，它与例 3.1 中给出的 AT&T 格式有较大的不同，常出现在许多介绍 Intel 汇编语言程序设计的书中，这些书主要采用微软的宏汇编程序 MASM 作为编程工具。MASM 采用的是 Intel 格式，它是大小写不敏感的，也即，“mov [bx + di - 6], cl” 也可以写成 “MOV [BX + DI - 6], CL”。Intel 格式与 AT&T 格式最大的不同是，Intel 格式中的目的操作数在左而源操作数在右，AT&T 格式则相反，如果要相互转换的话，就比较麻烦。此外 Intel 格式还有几点不同，如不带长度后缀，不在寄存器前加 %，偏移量写在括号中等。本教材主要使用 AT&T 格式。

小贴士

AT&T 格式：

长度后缀 b 表示指令中处理的操作数长度为字节，即 8 位；w 表示字，即 16 位；l 表示双字，即 32 位；q 表示四字，即 64 位。

寄存器操作数形式为“% + 寄存器名”，例如，“%eax”表示操作数为寄存器 EAX 中的内容，即 $R[eax]$ 。

存储器操作数形式为“偏移量（基址寄存器，变址寄存器，比例因子）”，例如，“100(%ebx,%esi,4)”表示存储单元的地址为 EBX 的内容加 ESI 的内容乘以 4 再加 100，即操作数为 $M[R[ebx] + 4 * R[esi] + 100]$ ，偏移量、基址寄存器、变址寄存器和比例因子都可以省略。

汇编指令形式为“op src,dst”，含义为“ $dst \leftarrow dst \text{ op } src$ ”。例如，“addl(,ebx,2),%eax”的含义为“ $R[eax] \leftarrow R[eax] + M[2 * R[ebx]]$ ”。

可重定位目标文件 test.o 并不能被直接执行，需要转换为可执行文件才能执行。若要生成可执行文件，可将其包含在一个 main 函数中并进行链接。

假设 main 函数所在的源程序文件 main.c 的内容如下：

```
1 // main.c
2 int main()
3 {
4     return add(20, 13);
5 }
```

可以用命令“gcc -o test main.c test.o”来生成可执行文件 test。若用命令“objdump -d test”来反汇编 test 文件，则得到与 add 函数对应的一段输出结果如下：

```
080483d4 <add>:
080483d4: 55                push    %ebp
080483d5: 89 e5             mov     %esp, %ebp
080483d7: 83 ec 10          sub     $0x10, %esp
080483da: 8b 45 0c           mov     0xc(%ebp), %eax
080483dd: 8b 55 08           mov     0x8(%ebp), %edx
080483e0: 8d 04 02           lea     (%edx,%eax,1), %eax
080483e3: 89 45 fc           mov     %eax, -0x4(%ebp)
080483e6: 8b 45 fc           mov     -0x4(%ebp), %eax
080483e9: c9                leave   %eax
080483ea: c3                ret
```

上述输出结果与 test.o 的输出差不多，只是左边的地址不再是从 0 开始，链接器将代码定位在一个特定的存储区域，其中 add 函数对应的指令序列从存储单元 080483d4H 开始存放。上述源程序中没有用到库函数调用，因而链接时无需考虑与静态库或动态库的链接。有关链接的具体内容详见第 4 章。

上述例子中包含的各指令的含义在 3.2 节中给出。

3.2 IA-32 指令系统概述

ISA 规定了机器级程序的格式和行为，因而先介绍相应的 Intel 指令集体系结构。x86 是 Intel 开发的一种处理器体系结构的泛称。该系列中较早期的处理器名称以数字来表示，并以

“86”结尾,包括 Intel 8086、80286、i386 和 i486 等,因此其架构被称为“x86”。由于数字并不能作为注册商标,因此 Intel 及其竞争者均对新一代处理器使用了可注册的名称,如 Pentium、PentiumPro、Core 2、Core i7 等。现在 Intel 把 32 位 x86 架构的名称 x86-32 改称为 IA-32,全名为“Intel Architecture, 32-bit”。后来,由 AMD 首先提出了一个 Intel 指令集的 64 位版本,命名为“x86-64”。它在 IA-32 的基础上对寄存器的宽度和个数、浮点运算指令等进行了扩展,并加入了一些新的特性,指令能够直接处理长度为 64 位的数据。后来 AMD 将其更名为 AMD64,而 Intel 称其为 Intel64。有关 Intel 架构处理器的历史沿革可参看第 1 章的表 1.1。

3.2.1 数据类型及其格式

在 IA-32 中,大部分指令并不需要区分其操作数是什么类型,只把操作数当成一个 0/1 序列来处理,因而,机器指令中只要再有相应的字段能够区分操作数的长度即可,例如,在图 3.1 中的位 W 可指出操作数是 8 位还是 16 位。对于 8086/8088 来说,因为只有 8 位和 16 位两种长度,因此用一位就行,但是,发展到 IA-32,已经有 8 位(字节)、16 位(字)、32 位(双字)等不同长度,因而用来表示操作数长度的字段 W 至少要有 2 位。在对应的汇编指令中,通过在指令助记符后面加一个长度后缀,或通过专门的数据长度指示符来指出操作数长度。IA-32 由 16 位架构发展而来,因此,Intel 最初规定一个字为 16 位,因而 32 位为双字。

IA-32 中有些指令需要区分操作数类型,通常由指令操作码来区分指令操作数是带符号整数、无符号整数还是浮点数。例如,浮点运算指令处理的都是浮点数,乘法指令 `imul` 的操作数是带符号整数,乘法指令 `mul` 的操作数是无符号整数。

高级语言中的表达式最终是通过指令指定的运算来实现的,表达式中出现的变量或常数就是指令中指定的操作数,因而高级语言所支持的数据类型与指令中指定的操作数类型之间有密切的关系。下面以 C 语言和 IA-32 指令系统为例,说明高级语言与指令系统之间在数据类型方面是如何对应的。

C 语言程序中的基本数据类型主要有以下几类。

① 指针或地址:用来表示字符串或其他数据区域的指针或存储地址,可声明为 `char *` 等类型,其宽度为 32 位,对应 IA-32 中的双字。

② 序数、位串等:用来表示序号、元素个数、元素总长度、位串等的无符号数,可声明为 `unsigned char`、`unsigned short[int]`、`unsigned[int]`、`unsigned long[int]` (括号中的 `int` 可省略) 类型,分别对应 IA-32 中的字节、字、双字和双字。因为 IA-32 是 32 位架构,所以,编译器把 `long` 型数据定义为 32 位。ISO C99 规定 `long long` 型数据至少是 64 位,而 IA-32 中没有能处理 64 位数据的指令,因而编译器大多将 `unsigned long long` 型数据运算转换为多条 32 位运算指令来实现。

③ 带符号整数:它是 C 语言中运用最广泛的基本数据类型,可声明为 `char`、`short[int]`、`int`、`long[int]` 类型,分别对应 IA-32 中的字节、字、双字和双字,用补码表示。与对待 `unsigned long long` 数据一样,编译器将 `long long` 型数据运算转换为多条 32 位运算指令来实现。

④ 浮点数:用来表示实数,可声明为 `float`、`double` 和 `long double` 类型,分别采用 IEEE 754 的单精度、双精度和扩展精度标准表示。`long double` 类型是 ISO C99 中新引入的,对于许多处理器和编译器来说,它等价于 `double` 类型,但是由于与 x86 处理器配合的协处理器 x87 中使用

了深度为 8 的 80 位的浮点寄存器栈，对于 Intel 兼容机来说，GCC 采用了 80 位的“扩展精度”格式表示。x87 中定义的 80 位扩展浮点格式包含 4 个字段：1 位符号位 s 、15 位阶码 e （偏置常数为 16 383）、1 位显式首位有效位（explicit leading significant bit） j 和 63 位尾数 f 。Intel 采用的这种扩展浮点数格式与 IEEE 754 规定的单精度和双精度浮点数格式的一个重要的区别是，它没有隐藏位，有效位数共 64 位。GCC 为了提高 long double 浮点数的访存性能，将其存储为 12 个字节（即 96 位，数据访问分 32 位和 64 位两次读写），其中前两个字节不用。

表 3.1 给出了 C 语言基本数据类型和 IA-32 操作数长度之间的对应关系。

表 3.1 C 语言基本数据类型和 IA-32 操作数类型的对应关系

C 语言声明	Intel 操作数类型	汇编指令长度后缀	存储长度（位）
(unsigned) char	整数/字节	b	8
(unsigned) short	整数/字	w	16
(unsigned) int	整数/双字	l	32
(unsigned) long int	整数/双字	l	32
(unsigned) long long int	—	—	2 × 32
char *	整数/双字	l	32
float	单精度浮点数	s	32
double	双精度浮点数	l	64
long double	扩展精度浮点数	t	80/96

GCC 生成的汇编代码中的指令助记符大部分都有长度后缀，例如，传送指令可以有 movb（字节传送）、movw（字传送）、movl（双字传送）等，这里，指令助记符最后的‘b’、‘w’和‘l’是长度后缀。从表 3.1 中可看出，双字整数和双精度浮点数的长度后缀都一样。因为已经通过指令操作码区分了是浮点数还是整数，所以长度后缀相同不会产生歧义。在微软 MASM 工具生成的 Intel 汇编格式中，并不用长度后缀来表示操作数长度，而是直接通过寄存器的名称和长度指示符 PTR 等来区分操作数长度，有关信息可以查看微软和 Intel 的相关资料。

3.2.2 寄存器组织和寻址方式

不考虑 I/O 指令，IA-32 指令的操作数有三类：立即数、寄存器操作数和存储器操作数。立即数就在指令中，无需指定其存放位置。寄存器操作数需要指定操作数所在寄存器的编号，例如，图 3.1 中的指令指定了源操作数寄存器的编号为 001。当操作数为存储单元内容时，需要指定操作数所在存储单元的地址，例如，图 3.1 中的指令指定了目的操作数的存储单元地址为 BX 和 DI 两个寄存器的内容相加再减 6，得到的是一个 16 位偏移地址，它和相应的段寄存器内容进行特定的运算就可以得到操作数所在的存储单元的地址。当然，图 3.1 给出的是早期 8086 实地址模式下的指令，因而存储地址计算方式比较简单。现在，IA-32 引入了保护模式，采用的是段页式存储管理方式，因而存储地址计算变得比较复杂。

IA-32 指令中用到的寄存器主要分为定点寄存器组、浮点寄存器栈和多媒体扩展寄存器组。下面分别介绍 IA-32 的定点寄存器组、浮点寄存器栈和多媒体扩展寄存器组。

1. 定点寄存器组

因为 IA-32 由最初的 8086/8088 向后兼容扩展而来，因此，寄存器的结构也体现了逐步扩

展的特点。图 3.2 给出了定点（整数）寄存器组的结构。

	31	16	15	8	7	0	
EAX							AH (AX) AL 累加器
EBX							BH (BX) BL 基址寄存器
ECX							CH (CX) CL 计数寄存器
EDX							DH (DX) DL 数据寄存器
ESP							SP 栈指针寄存器
EBP							BP 基址指针寄存器
ESI							SI 源变址寄存器
EDI							DI 目标变址寄存器
EIP							IP 指令指针寄存器
EFLAGS							FLAGS 标志寄存器
							CS 代码段
							SS 堆栈段
							DS 数据段
							ES 附加段
							FS 附加段
							GS 附加段

图 3.2 IA-32 的定点寄存器组

从图 3.2 可以看出，IA-32 中的定点寄存器中共有 8 个通用寄存器（General-Purpose Register, GPR）、两个专用寄存器和 6 个段寄存器。定点通用寄存器是指没有专门用途的可以存放各类定点操作数的寄存器。

8 个通用寄存器的长度为 32 位，其中 EAX、EBX、ECX 和 EDX 主要用来存放操作数，可根据操作数长度是字节、字还是双字来确定存取寄存器的最低 8 位、最低 16 位还是全部 32 位。ESP、EBP、ESI 和 EDI 主要用来存放变址值或指针，可以作为 16 位或 32 位寄存器使用，其中，ESP 是栈指针寄存器，EBP 是基址指针寄存器。

两个专用寄存器分别是指令指针寄存器 EIP 和标志寄存器 EFLAGS。EIP 从 16 位的 IP 扩展而来，指令指针寄存器 IP（Instruction Pointer）与程序计数器 PC 是功能完全一样的寄存器，名称不同而已，在本教材中两者通用，它们都是用来存放即将执行指令的地址的寄存器。EFLAGS 从 16 位的 FLAGS 扩展而来。实地址模式时，使用 16 位的 IP 和 FLAGS 寄存器，保护模式时，使用 32 位的 EIP 和 EFLAGS 寄存器。

EFLAGS 寄存器主要用于记录机器的状态和控制信息，如图 3.3 所示。

31 ~ 22	21	20	19	18	17	16	15	14	13 12	11	10	9	8	7	6	5	4	3	2	1	0
保留	ID	VIP	VIF	AC	VM	RF	0	NT	IOPL	O	D	I	T	S	Z	0	A	0	P	1	C

图 3.3 状态标志寄存器 EFLAGS

EFLAGS 寄存器的第 0 ~ 11 位中的 9 个标志位是从最早的 8086 微处理器延续下来的，它们按功能可以分为 6 个条件标志和 3 个控制标志。其中，条件标志用来存放运行的状态信息，由硬件自动设定，条件标志有时也称为条件码；控制标志由软件设定，用于中断响应、串操作和单步执行等控制。

常用条件标志的含义说明如下。

① OF(Overflow Flag): 溢出标志。反映带符号数的运算结果是否超过相应数值范围。例如, 字节运算结果超出 $-128 \sim +127$ 或字运算结果超出 $-32\,768 \sim +32\,767$ 时, 称为“溢出”, 此时 $OF=1$; 否则 $OF=0$ 。

② SF(Sign Flag): 符号标志。反映带符号数运算结果的符号。负数时, $SF=1$; 否则 $SF=0$ 。

③ ZF(Zero Flag): 零标志。反映运算结果是否为 0。若结果为 0, $ZF=1$; 否则 $ZF=0$ 。

④ CF(Carry Flag): 进位标志。反映无符号整数加(减)运算后的进(借)位情况。有进(借)位则 $CF=1$; 否则 $CF=0$ 。

综上所述, OF 和 SF 对于无符号数运算来说没有意义, 而 CF 对于带符号整数运算来说没有意义。

控制标志的含义说明如下。

① DF(Direction Flag): 方向标志。用来确定串操作指令执行时变址寄存器 SI(ESI) 和 DI(EDI) 中的内容是自动递增还是递减。若 $DF=1$, 则为递减; 否则为递增。可用 std 指令和 cld 指令分别将 DF 置 1 和清 0。

② IF(Interrupt Flag): 中断允许标志。若 $IF=1$, 表示允许响应中断; 否则禁止响应中断。IF 对非屏蔽中断和内部异常不起作用, 仅对外部可屏蔽中断起作用。可用 sti 指令和 cli 指令分别将 IF 置 1 和清 0。

③ TF(Trap Flag): 陷阱标志。用来控制单步执行操作。 $TF=1$ 时, CPU 按单步方式执行指令, 此时, 可以控制在每执行完一条指令后, 就把该指令执行得到的机器状态(包括各寄存器和内存单元的值等)显示出来。没有专门的指令用于该标志的修改, 但可用栈操作指令(如 pushf/pushfd 和 popf/popfd)来改变其值。

EFLAGS 寄存器的第 12~31 位中的其他状态或控制信息是从 80286 以后逐步添加的。包括用于表示当前程序的 I/O 特权级(IOPL)、当前任务是否是嵌套任务(NT)、当前处理器是否处于虚拟 8086 方式(VM)等一些状态或控制信息。

6 个段寄存器都是 16 位, CPU 根据段寄存器的内容, 与寻址方式确定的有效地址一起, 并结合其他用户不可见的内部寄存器, 生成操作数所在的存储地址。

2. 寻址方式

根据指令给定信息得到操作数或操作数地址的方式称为寻址方式。图 3.4 给出了 IA-32 中的各种寻址方式。其中, 立即寻址指指令中直接给出操作数; 寄存器寻址指指令中给出操作数所在的寄存器的编号。除了立即寻址和寄存器寻址外, 其他寻址方式下的操作数都在存储单元中, 称为存储器操作数。

存储器操作数的寻址方式与微处理器的工作模式有关。IA-32 处理器主要有两种工作模式, 即实地址模式和保护模式。

实地址模式是为与 8086/8088 兼容而设置的, 在加电或复位时处于这一模式。此模式下的存储管理、中断控制以及应用程序运行环境等都与 8086/8088 相同。其最大寻址空间为 1MB, 32 条地址线中的 $A_{31} \sim A_{20}$ 不起作用, 存储管理采用分段方式, 每段的最大地址空间为 64KB, 物理地址由段地址乘以 16 加上偏移地址构成, 其中段地址位于段寄存器中, 偏移地址用来指定段内一个存储单元。例如, 当前指令地址为 $(CS) \ll 4 + (IP)$, 其中 CS(Code Segment) 为

代码段寄存器，用于存放当前代码段地址，IP 寄存器中存放的是当前指令在代码段内的偏移地址，这里，(CS) 和 (IP) 分别表示寄存器 CS 和 IP 中的内容。内存区 00000H ~ 003FFH 存放中断向量表，共存放 256 个中断向量，采用 8086/8088 的中断类型和中断处理方式。有关中断、中断向量和中断向量表等概念，详见第 7 章和第 8 章相关内容。

保护模式的引入是为了实现在多任务方式下对不同任务使用的虚拟存储空间进行完全的隔离，以保证不同任务之间不会相互破坏各自的代码和数据。保护模式是 80286 以上高档微处理器最常用的工作模式。系统启动后总是先进入实地址模式，对系统进行初始化，然后转入保护模式进行操作。在保护模式下，处理器采用虚拟存储器管理方式。

IA-32 采用段页式虚拟存储管理方式，CPU 首先通过分段方式得到线性地址 LA，再通过分页方式实现从线性地址到物理地址的转换。有关虚拟存储器管理、段页式、分段、分页、线性地址、物理地址等概念及其实现原理将在第 6 章的 6.5 节和 6.6 节详细介绍。

图 3.4 给出了 IA-32 在保护模式下的各种寻址方式，其中，存储器操作数的访问过程需要计算线性地址 LA，图中除了最后一行（相对寻址）计算的是转移目标指令的线性地址以外，其他的都是指操作数的线性地址。相对寻址的线性地址与 PC（即 EIP 或 IP）有关，而操作数的线性地址与 PC 无关，它取决于某个段寄存器的内容和有效地址。根据段寄存器的内容能够确定操作数所在的段在某个存储空间的起始地址，而**有效地址**则给出了操作数所在段的偏移地址。

寻址方式	说明
立即寻址	指令直接给出操作数
寄存器寻址	指定的寄存器 R 的内容为操作数
位移	$LA = (SR) + A$
基址寻址	$LA = (SR) + (B)$
基址加位移	$LA = (SR) + (B) + A$
比例变址加位移	$LA = (SR) + (I) \times S + A$
基址加变址加位移	$LA = (SR) + (B) + (I) + A$
基址加比例变址加位移	$LA = (SR) + (B) + (I) \times S + A$
相对寻址	$LA = (PC) + A$

注：LA：线性地址 (X)：X 的内容 SR：段寄存器 PC：程序计数器 R：寄存器
A：指令中给定地址段的位移量 B：基址寄存器 I：变址寄存器 S：比例系数

图 3.4 IA-32 的寻址方式

从图 3.4 中可以看出，在存储器操作数的情况下，指令必须显式或隐式地给出以下信息。

- ① 段寄存器 SR（可用段前缀显式给出，也可缺省使用默认段寄存器）。
- ② 8/16/32 位位移量 A（由位移量字段显式给出，例如，图 3.1 中的字段 disp8）。
- ③ 基址寄存器 B（由相应字段显式给出，可指定为任一通用寄存器）。
- ④ 变址寄存器 I（由相应字段显式给出，可指定除 ESP 外的任一通用寄存器）。

有比例变址和非比例变址两种变址方式。**比例变址**时，变址值等于变址寄存器内容乘以**比例系数 S**（也称为**比例因子**），S 的含义为操作数的字节个数，在 IA-32 中，S 的取值可以是 1、2、4 或 8。例如，对数组元素访问时，若数组元素的类型为 short，则比例系数就是 2；若数组元素类型为 float，则比例系数就是 4。IA-32 提供的“基址加位移”、“基址加变址加位移”等

这些复杂的存储器操作数寻址方式，主要是为了指令能够方便地访问到数组和结构等复合数据结构内的元素。非比例变址相当于比例系数为 1 的比例变址情况，也即，变址值就是变址寄存器的内容，无需乘以比例系数。

有效地址由指令中给出的寻址方式来确定如何计算。例如，对于 Intel 格式的汇编指令“mov eax, [ebp + esi * 4 + 1000H]”（AT&T 格式为“movw 4096(%ebp,%esi,4),%eax”），其源操作数的寻址方式为“基址加比例变址加位移”，因此，有效地址通过将基址寄存器 EBP 的内容、变址值（变址寄存器 ESI 的内容乘以比例系数 4）和位移量 1000H 三者相加得到。

3. 浮点寄存器栈和多媒体扩展寄存器组

IA-32 的浮点处理架构有两种。一种是与 x86 配套的浮点协处理器 x87 架构，它是一种栈结构 FPU，x87 中进行运算的浮点数来源于浮点寄存器栈的栈顶；另一种是由 MMX 发展而来的 SSE 架构，采用单指令多数据（Single Instruction Multi Data, SIMD）技术，SIMD 技术可实现单条指令同时并行处理多个数据元素的功能，其操作数来源于专门新增的 8 个 128 位寄存器 XMM0 ~ XMM7。

小贴士

FPU（Float Point Unit，浮点运算器）是专用于浮点运算的处理器，以前的 FPU 是单独的芯片，在 80486 之后，Intel 把 FPU 集成在 CPU 之内。

MMX 是 MultiMedia eXtensions（多媒体扩展）的缩写。MMX 指令于 1997 年首次运用于 P54C Pentium 处理器，称之为多能奔腾。MMX 技术主要是指在 CPU 中加入了特地为视频信号（video signal）、音频信号（audio signal）以及图像处理（graphical manipulation）而设计的 57 条指令，因此，MMX CPU 极大地提高了多媒体（如立体声、视频、三维动画等）处理能力。

x87 FPU 中有 8 个数据寄存器，每个 80 位。此外，还有控制寄存器、状态寄存器和标记寄存器各一个，它们的长度都是 16 位。数据寄存器被组织成一个浮点寄存器栈，栈顶记为 ST(0)，下一个元素是 ST(1)，再下一个是 ST(2)，以此类推。栈的大小是 8，当栈被装满时，可访问的元素为 ST(0) ~ ST(7)。控制寄存器主要用于指定浮点处理单元的舍入方式及最大有效数据位数（即精度），Intel 浮点处理器的默认精度是 64 位，即指 80 位扩展精度浮点数；状态寄存器用来记录比较结果，并标记运算是否溢出、是否产生错误等，此外还记录了数据寄存器栈的栈顶位置；标记寄存器指出了 8 个数据寄存器各自的状态，比如是否为空、是否可用、是否为零、是否是特殊值（如 NaN、 $+\infty$ 、 $-\infty$ ）等。

SSE 指令集由 MMX 指令集发展而来。MMX 指令使用的 8 个 64 位寄存器 MM0 ~ MM7 借用了 x87 FPU 中 8 个 80 位浮点数据寄存器 ST(0) ~ ST(7)，每个 MMX 寄存器实际上是对应 80 位浮点数据寄存器中 64 位尾数所占的位，因此，每条 MMX 指令可以同时处理 8 个字节，或 4 个字，或 2 个双字，或一个 64 位的数据。由于 MMX 指令并没有带来 3D 游戏性能的显著提升，1999 年 Intel 公司在 Pentium III CPU 产品中首推 SSE 指令集，后来又陆续推出了 SSE2、SSE3、SSSE3 和 SSE4 等采用 SIMD 技术的指令集，这些统称为SSE 指令集。SSE 指令集兼容 MMX 指

令, 并通过 SIMD 技术在单个时钟周期内并行处理多个浮点数来有效提高浮点运算速度。因为在 MMX 技术中借用了 x87 FPU 的 8 个浮点寄存器, 导致了 x87 浮点运算速度的降低, 因而 SSE 指令集增加了 8 个 128 位的 SSE 指令专用的多媒体扩展通用寄存器 XMM0 ~ XMM7。这样, SSE 指令的寄存器位数是 MMX 指令寄存器位数的两倍, 因而一条 SSE 指令可以同时并行处理 16 个字节, 或 8 个字, 或 4 个双字 (32 位整数或单精度浮点数), 或两个四字的数据, 而且从 SSE2 开始, 还支持 128 位整数运算或同时并行处理两个 64 位双精度浮点数。

综上所述, IA-32 中的通用寄存器共有三类: 8 个 8/16/32 位定点通用寄存器、8 个 MMX 指令/x87 FPU 使用的 64 位/80 位寄存器 MM0/ST(0) ~ MM7/ST(7)、8 个 SSE 指令使用的 128 位寄存器 XMM0 ~ XMM7。这些寄存器编号如表 3.2 所示。

表 3.2 IA-32 中通用寄存器的编号

编号	8 位寄存器	16 位寄存器	32 位寄存器	64 位寄存器	128 位寄存器
000	AL	AX	EAX	MM0/ST(0)	XMM0
001	CL	CX	ECX	MM1/ST(1)	XMM1
010	DL	DX	EDX	MM2/ST(2)	XMM2
011	BL	BX	EBX	MM3/ST(3)	XMM3
100	AH	SP	ESP	MM4/ST(4)	XMM4
101	CH	BP	EBP	MM5/ST(5)	XMM5
110	DH	SI	ESI	MM6/ST(6)	XMM6
111	BH	DI	EDI	MM7/ST(7)	XMM7

3.3 IA-32 常用指令类型及其操作

与大多数 ISA 一样, IA-32 提供了数据传送、算术和逻辑运算、程序流程控制等常用指令类型。下面分别介绍这几类常用指令类型。

3.3.1 传送指令

传送指令用于寄存器、存储单元或 I/O 端口之间传送信息, 分为通用数据传送、地址传送、标志传送和 I/O 信息传送等几类, 除了部分标志传送指令外, 其他指令均不影响标志位的状态。

1. 通用数据传送指令

通用数据传送指令传送的是寄存器或存储器中的数据, 主要有以下几种。

MOV: 一般的传送指令, 包括 movb、movw 和 movl 等。

MOVS: 符号扩展传送指令, 将短的源数据高位符号扩展后传送到目的地址, 如 movsbw 表示把一个字节进行符号扩展后送到一个字地址中。

MOVZ: 零扩展传送指令, 将短的源数据高位零扩展后传送到目的地址, 如 movzwl 表示把一个字的高位进行零扩展后送到一个双字地址中。

XCHG: 数据交换指令, 将两个寄存器内容互换。例如, xchgb 表示字节交换。

PUSH: 先执行 $R[sp] \leftarrow R[sp] - 2$ 或 $R[esp] \leftarrow R[esp] - 4$, 然后将一个字或双字从指定寄存器送到 SP 或 ESP 指示的栈单元中。如 `pushl` 表示双字压栈, `pushw` 表示字压栈。

POP: 先将一个字或双字从 SP 或 ESP 指示的栈单元送到指定寄存器中, 再执行 $R[sp] \leftarrow R[sp] + 2$ 或 $R[esp] \leftarrow R[esp] + 4$ 。如 `popl` 表示双字出栈, `popw` 表示字出栈。

栈 (stack) 是一种采用“先进后出”方式进行访问的一块存储区, 在处理过程调用时非常有用。大多数情况下, 栈是从高地址向低地址增长的, 在 IA-32 中, 用 ESP 寄存器指向当前栈顶, 而栈底通常在一个固定的高地址上。图 3.5 给出了在 16 位架构下的 `pushw` 和 `popw` 指令执行结果示意图。图中显示, 在执行 `pushw` 指令之后, SP 指向存放有 AX 内容的单元, 也即新栈顶指向了当前刚入栈的数据。若随后再执行 `popw` 指令, 则原先在栈顶的两个字节退出栈, 栈顶向高地址移动两个单元, 又回到 `pushw` 指令执行前的位置。这里请注意 AH 和 AL 的存放位置, 因为 Intel 架构采用的是小端方式, 所以应该是 AL 在低地址上, AH 在高地址上。

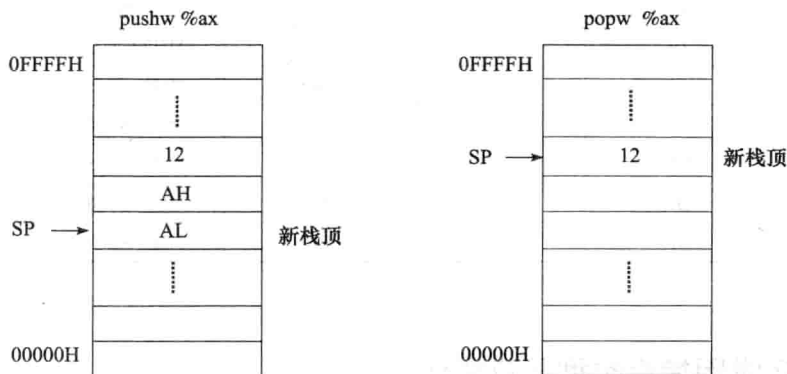


图 3.5 `pushw` 和 `popw` 指令的执行

2. 地址传送指令

地址传送指令传送的是操作数的存储地址, 指定的目的寄存器不能是段寄存器, 且源操作数必须是存储器寻址方式。注意, 这些指令均不影响标志位。主要是 **加载有效地址** (Load Effect Address, LEA) 指令, 用来将源操作数的存储地址送到目的寄存器中。如 `leal` 指令把一个 32 位的地址传送到一个 32 位的寄存器中。通常利用该指令执行一些简单操作, 例如, 对于例 3.1 中的运算 $i + j$, 编译器使用了指令 “`leal (%edx, %eax), %eax`”, 以实现 $R[edx] \leftarrow R[edx] + R[edx]$ 的功能, 该指令执行前, $R[edx] = i$, $R[edx] = j$, 指令执行后 $R[edx] = i + j$ 。

3. 输入输出指令

输入输出指令专门用于在累加器和 I/O 端口之间进行数据传送。例如, `in` 指令用于将 I/O 端口内容送累加器, `out` 指令将累加器内容送 I/O 端口。

4. 标志传送指令

标志传送指令专门用于对标志寄存器进行操作。如 `pushf` 指令用于将标志寄存器的内容压栈, `popf` 指令将栈顶内容送标志寄存器。

例 3.2 将以下 Intel 格式汇编指令转换为 GCC 默认的 AT&T 格式汇编指令。说明每条指令的含义。

```

1  push    ebp
2  mov     ebp, esp
3  mov     edx, DWORD PTR [ebp + 8]
4  mov     bl, 255
5  mov     ax, WORD PTR [ebp + edx*4 + 8]
6  mov     WORD PTR [ebp + 20], dx
7  lea     eax, [ecx + edx*4 + 8]

```

解 上述 Intel 格式汇编指令转换为 AT&T 格式汇编指令及其指令的含义说明如下（右边的符号//后描述的是相应指令的含义）。

```

1  pushl    %ebp           // R[esp] ← R[esp] - 4, M[R[esp]] ← R[ebp], 双字
2  movl     %esp, %ebp     // R[ebp] ← R[esp], 双字
3  movl     8(%ebp), %edx   // R[edx] ← M[R[ebp] + 8], 双字
4  movb     $255, %bl      // R[bl] ← 255, 字节
5  movw     8(%ebp, %edx, 4), %ax // R[ax] ← M[R[ebp] + R[edx] × 4 + 8], 字
6  movw     %dx, 20(%ebp)   // M[R[ebp] + 20] ← R[dx], 字
7  leal     8(%ecx, %edx, 4), %eax // R[eax] ← R[ecx] + R[edx] × 4 + 8, 双字

```

从第 7 条指令的功能可看出，LEA 指令是 MOV 指令的一个变形，相当于实现了 C 语言中的地址操作符 & 的功能。同时，LEA 指令可实现一些简单操作，例如，假定第 7 条指令中寄存器 ECX 和 EDX 内分别存放的是变量 x 和 y 的值，即 $R[ecx] = x$ ， $R[edx] = y$ ，则通过该指令可以计算 $x + 4y + 8$ 的值，并将其存入寄存器 EAX 中。

例 3.3 假设变量 val 和 ptr 的类型声明如下：

```

val_type val;
contofptr_type *ptr;

```

已知上述类型 val_type 和 $contofptr_type$ 是用 typedef 声明的数据类型，且 val 存储在累加器 AL/AX/EAX 中， ptr 存储在 EDX 中。现有以下两条 C 语言语句：

```

1  val = (val_type) *ptr;
2  *ptr = (contofptr_type) val;

```

当 val_type 和 $contofptr_type$ 是表 3.3 中给出的组合类型时，应分别使用什么样的 MOV 指令来实现这两条 C 语句？要求用 GCC 默认的 AT&T 形式写出。

表 3.3 例 3.3 中 val_type 和 $contofptr_type$ 的类型

val_type	$contofptr_type$	val_type	$contofptr_type$
char	char	int	unsigned char
int	char	unsigned	unsigned char
unsigned	int	unsigned short	int

解 C 操作符 * 可看成取值操作。语句 1 的含义是将 ptr 所指的存储单元中的内容送到 val 变量所在处，也即，将地址为 $R[edx]$ 的存储单元内容送到累加器 AL/AX/EAX 中；语句 2 的含义是将 val 变量的值送到 ptr 所指的存储单元中，也即，将累加器 AL/AX/EAX 中的内容送到地址为 $R[edx]$ 的存储单元中。其对应的 MOV 指令见表 3.4。

表 3.4 例 3.3 的答案

序号	val_type	contofptr_type	语句 1 对应的指令及操作	语句 2 对应的指令及操作
1	char	char	movb (%edx), %al //传送	movb %al, (%edx) //传送
2	int	char	movsbl (%edx), %eax //符号扩展, 传送	movb %al, (%edx) //截断, 传送
3	unsigned	int	movl (%edx), %eax //传送	movl %eax, (%edx) //传送
4	int	unsigned char	movzbl (%edx), %eax //零扩展, 传送	movb %al, (%edx) //截断, 传送
5	unsigned	unsigned char	movzbl (%edx), %eax //零扩展, 传送	movb %al, (%edx) //截断, 传送
6	unsigned short	int	movw (%edx), %ax //截断, 传送	movzwl %ax, (%edx) //零扩展, 传送

表 3.4 中给出的 6 种情况中, 序号为 1 和 3 的两种情况比较简单, 赋值语句两边的操作数长度一样, 即使一个是带符号整数类型, 另一个是无符号整数类型, 传送前、后的位串也不会改变 (软件通过对相同位串的不同解释来反映不同的值), 因此, 用直接传送的指令即可。对于序号为 2 的情况, 语句 1 要求将存储单元中的一个 char 型数据送到一个存放 int 型数据的 32 位寄存器中, 因此需要符号扩展; 而对于语句 2, 则是把一个 32 位寄存器中的数据截断为 8 位数据送到存储单元中, 因此直接丢弃寄存器中高 24 位, 仅将最低 8 位 (即 R[al]) 送到存储单元。对于序号为 4 和 5 的情况, 语句 1 将存储单元中的 8 位无符号整数进行零扩展为 32 位后送到寄存器; 而语句 2 是将 32 位寄存器中的内容截断为 8 位数据送到存储单元中。对于序号为 6 的情况, 语句 1 要求把存储单元中一个 int 型数据截断为一个 16 位数据送到寄存器中, 那么截断操作时, 该留下的应该是 4 个字节地址中哪两个地址的内容呢? IA-32 中数据在存储单元中按小端方式存放, 因而留下的应该是小地址中的内容, 即地址 R[edx] 和 R[edx] + 1 中的内容。例如, 假定这个将被截断的 int 型数据是 12345678H, 如图 3.6 所示, 4 个字节 12H、34H、56H 和 78H 的地址分别是 R[edx] + 3、R[edx] + 2、R[edx] + 1、R[edx], 截断操作后应该留下 5678H, 即存放在存储单元 R[edx] 开始的两个字节。若是大端方式, 则语句 1 和语句 2 各自对应的指令应该是什么呢? 答案留给读者自己思考。

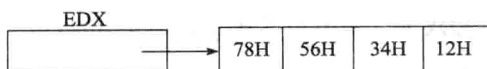


图 3.6 小端方式下 int 型数据的存放位置

3.3.2 定点算术运算指令

定点算术运算指令用于二进制数和无符号十进制数的各种算术运算。IA-32 中的二进制定点数可以是 8 位、16 位或 32 位数; 无符号十进制数 (BCD 码) 主要是采用 8421 码表示的数。高级语言中的算术运算都被转换为二进制数运算指令实现, 因此, 本书所讲的运算指令都是指二进制数运算指令。

1. 加/减运算指令

加/减类指令 (ADD/SUB) 用于对给定长度的两个位串进行相加或相减, 两个操作数中最多只能有一个是存储器操作数, 不区分是无符号数还是带符号整数, 产生的和/差送到目的地, 生成的标志信息送标志寄存器 FLAGS/EFLAGS。

2. 增/减运算指令

增/减类 (INC/DEC) 指令对给定长度的一个位串加 1 或减 1, 给定操作数既是源操作数

也是目的操作数，不区分是无符号数还是带符号整数，生成的标志信息送标志寄存器 FLAGS/EFLAGS，注意不生成 CF 标志。

3. 取负指令

取负类指令 NEG 用于求操作数的负数，也即，将给定长度的一个位串“各位取反、末尾加 1”，也称之为取补指令。给定操作数既是源操作数也是目的操作数，生成的标志信息送标志寄存器 FLAGS/EFLAGS。若字节操作数的值为 -128，或字操作数的值为 -32768，或双字操作数的值为 -2147483648，则其操作数无变化，但 OF = 1。若操作数的值为 0，则取补结果仍为 0 且 CF 置 0，否则总是使 CF 置 1。

4. 比较指令

比较类指令 CMP 用于两个寄存器操作数的比较，用目的操作数减去源操作数，结果不送回目的操作数，即两个操作数保持原值不变，只是标志位作相应改变，因而功能类似 SUB 指令。通常，该指令后面跟条件转移指令或条件设置指令。

5. 乘/除运算指令

乘法指令分成 MUL(无符号数乘)和 IMUL(带符号整数乘)两类，指令中可以明显地给出一个操作数、两个操作数或三个操作数。

若指令中只给出一个操作数 SRC，则另一个源操作数隐含在累加器 AL/AX/EAX 中，将 SRC 和累加器内容相乘，结果存放在 AX(16 位时)或 DX-AX(32 位时)或 EDX-EAX(64 位时)中。DX-AX 表示 32 位乘积的高、低 16 位分别在 DX 和 AX 中。

若指令中给出两个操作数 DST 和 SRC，则将 DST 和 SRC 相乘，结果存放在 DST 中。

若指令中给出三个操作数 REG、SRC 和 IMM，则将 SRC 和立即数 IMM 相乘，结果存放在寄存器 REG 中。

除法指令分成 DIV(无符号数除)和 IDIV(带符号整数除)两类，指令中只明显指出除数，用累加器 AL/AX/EAX 中的内容除以指令中指定的除数。若源操作数为 8 位，则 16 位的被除数隐含在 AX 寄存器中，商送回 AL，余数在 AH 中；若源操作数为 16 位，则 32 位的被除数隐含在 DX-AX 寄存器中，商送回 AX，余数在 DX 中；若源操作数是 32 位，则 64 位的被除数隐含在 EDX-EAX 寄存器中，商送回 EAX，余数在 EDX 中。需要说明的是，如果商超过目的寄存器能存放的最大值，系统产生类型为 0 的中断，并且商和余数均不确定。

以上所有定点算术运算指令汇总在表 3.5 中。

表 3.5 定点算术运算指令汇总

指令	显式操作数	影响的常用标志	操作数类型	AT&T 指令助记符	对应 C 运算符
ADD	2 个	OF、ZF、SF、CF	无/带符号整数	addb、addw、addl	+
SUB	2 个	OF、ZF、SF、CF	无/带符号整数	subb、subw、subl	-
INC	1 个	OF、ZF、SF	无/带符号整数	incb、incw、incl	++
DEC	1 个	OF、ZF、SF	无/带符号整数	decb、decw、decl	--
NEG	1 个	OF、ZF、SF、CF	无/带符号整数	negb、negw、negl	-
CMP	2 个	OF、ZF、SF、CF	无/带符号整数	cmpb、cmpw、cmpl	<, <=, >, >=
MUL	1 个	无	无符号整数	mulb、mulw、mull	*

(续)

指令	显式操作数	影响的常用标志	操作数类型	AT&T 指令助记符	对应 C 运算符
MUL	2 个	无	无符号整数	mulb、mulw、mull	*
MUL	3 个	无	无符号整数	mulb、mulw、mull	*
IMUL	1 个	无	带符号整数	imulb、imulw、imull	*
IMUL	2 个	无	带符号整数	imulb、imulw、imull	*
IMUL	3 个	无	带符号整数	imulb、imulw、imull	*
DIV	1 个	无	无符号整数	divb、divw、divl	/, %
IDIV	1 个	无	带符号整数	idivb、idivw、idivl	/, %

例 3.4 假设 $R[ax] = \text{FFFAH}$, $R[bx] = \text{FFF0H}$, 则执行 Intel 格式指令 “add ax, bx” 后, AX、BX 中的内容各是什么? 标志 CF、OF、ZF、SF 各是什么? 要求分别将操作数作为无符号数和带符号整数来解释并验证指令执行结果。

解 根据 Intel 指令格式规定可知, 指令 “add ax, bx” 的功能是 $R[ax] \leftarrow R[ax] + R[bx]$, 指令的执行在图 2.6 所示的整数加减运算器中进行, 执行后其结果在 AX 中, 即 $R[ax] = \text{FFFAH} + \text{FFF0H} = \text{FFEAH}$, 而 BX 的内容不变, 即 $R[bx] = \text{FFF0H}$, 标志 $\text{CF} = 1$, $\text{SF} = 1$, $\text{OF} = 0$, $\text{ZF} = 0$ 。若作为无符号数来解释, 则根据 $\text{CF} = 1$ 可判断其结果溢出; 若作为带符号整数来解释, 则根据 $\text{OF} = 0$ 可判断其结果不溢出且和为 -22 。无符号数加法运算结果验证如下: $R[ax] = \text{FFFAH}$, 即值为 $65\,530$, $R[bx] = \text{FFF0H}$, 即值为 $65\,520$, 所以结果为 $65\,530 + 65\,520 = 131\,050$, 显然大于 16 位最大可表示的无符号数 $65\,535$, 即结果溢出, 验证正确。带符号整数加法运算结果验证如下: $R[ax] = \text{FFFAH}$, 即值为 $-110\text{B} = -6$, $R[bx] = \text{FFF0H}$, 即值为 $-1000\text{B} = -16$, 所以结果为 $-6 + (-16) = -22$, 验证正确。■

例 3.5 假设 $R[eax] = 000000\text{B4H}$, $R[ebx] = 00000011\text{H}$, $M[000000\text{F8H}] = 000000\text{A0H}$, 请问:

① 执行指令 “mulb %bl” 后, 哪些寄存器的内容会发生变化? 是否与执行 “imulb %bl” 指令所发生的变化一样? 为什么? 请用该例给出的数据验证你的结论。

② 执行指令 “imull \$-16, (%eax, %ebx, 4), %eax” 后, 哪些寄存器和存储单元发生了变化? 乘积的机器数和真值各是多少?

解 因为 $R[eax] = 000000\text{B4H}$, $R[ebx] = 00000011\text{H}$, 所以, $R[al] = \text{B4H}$, $R[bl] = 11\text{H}$ 。

① 指令 “mulb %bl” 中指出的操作数为 8 位, 故指令的功能为 “ $R[ax] \leftarrow R[al] \times R[bl]$ ”, 因此, 改变内容的寄存器是 AX, 指令执行后 $R[ax] = 0\text{BF4H}$, 即十进制数 3060 。执行指令 “imulb %bl” 后, $R[ax] = \text{FAF4H}$, 即十进制数 -1292 。由此可见, 两条指令执行后发生变化的寄存器都是 AX, 但是存入 AX 的内容不一样。mulb 指令执行的是无符号数乘法, 而 imulb 执行的是带符号整数乘法, 根据 2.7.5 节中给出的无符号数和带符号整数的乘法运算规则可知, 若乘积只取低 8 位, 则两者的机器数一样, 在此例中两者都是 F4H ; 若乘积取 16 位, 则高 8 位可能不同, 此例中一个是 0BH , 一个是 FAH 。验证: 此例中 mulb 指令执行的运算是 $180 \times 17 = 3060$, 而 imulb 指令执行的运算是 $-76 \times 17 = -1292$ 。

② 指令 “imull \$-16, (%eax, %ebx, 4), %eax” 的功能是 “ $R[eax] \leftarrow (-16) \times M[R[eax] +$

$4 \times R[ebx]]$ ”，其中，第二个乘数所在的存储单元地址为 $R[ecx] + 4 \times R[ebx] = 0xB4 + (0x11 \ll 2) = 0xF8 = 000000F8H$ ，因为 $M[000000F8H] = 000000A0H$ ，与 -16 相乘后得到一个负的乘积，因此乘积的符号为负，乘积数值部分绝对值的机器数为 $000000A0H \ll 4 = 00000A00H$ ，对其各位取反末位加 1，得到乘积的机器数为 $FFFFFF600H$ ，即指令执行后 EAX 中存放的内容为 $FFFFFF600H$ ，其真值为 -2560 。 ■

3.3.3 按位运算指令

按位运算指令用来对不同长度的操作数进行按位操作，立即数只能作为源操作数，不能作为目的操作数，并且最多只能有一个为存储器操作数。按位运算指令主要分为逻辑运算指令和移位指令。

1. 逻辑运算指令

以下 5 类逻辑运算指令中，仅 NOT 指令不影响条件标志位，其他指令执行后，OF = CF = 0，而 ZF 和 SF 则根据运算结果来设置：若结果为全 0，则 ZF = 1；若最高位为 1，则 SF = 1。

NOT：单操作数的取反指令，它将操作数每一位取反，然后把结果送回对应位。

AND：对双操作数按位逻辑“与”，主要用来实现“掩码”操作。例如，执行指令“andb \$0xf, %al”后，AL 的高 4 位被屏蔽而变成 0，低 4 位被析取出来。

OR：对双操作数按位逻辑“或”，常用于使目的操作数的特定位置 1。例如，执行指令“orw \$0x3, %bx”后，BX 寄存器的最后两位被置 1。

XOR：对双操作数按位进行逻辑“异或”，常用于判断两个操作数中哪些位不同或用于改变指定位的值。例如，执行指令“xorw \$0x1, %bx”后，BX 寄存器最低位被取反。

TEST：根据两个操作数相“与”的结果来设置条件标志，常用于需检测某种条件但不能改变原操作数的场合。例如，可通过执行“testb \$0x1, %al”指令判断 AL 最后一位是否为 1。判断规则为：若 ZF = 0，则说明 AL 最后一位为 1；否则为 0。也可通过执行“testb %al, %al”指令来判断 AL 是否为 0、正数或负数。判断规则为：若 ZF = 1，则说明 AL 为 0；若 SF = 0 且 ZF = 0，则说明 AL 为正数；若 SF = 1，则说明 AL 为负数。

2. 移位指令

移位指令将寄存器或存储单元中的 8、16 或 32 位二进制数进行算术移位、逻辑移位或循环移位。在移位过程中，把 CF 看作扩展位，用它接受从操作数最左或最右移出的一个二进制位。只能移动 1~31 位，所移位数可以是立即数或存放在 CL 寄存器中的一个数值。

SHL：逻辑左移，每左移一次，最高位送入 CF，并在低位补 0。

SHR：逻辑右移，每右移一次，最低位送入 CF，并在高位补 0。

SAL：算术左移，操作与 SHL 指令类似，每次移位，最高位送入 CF，并在低位补 0。执行 SAL 指令时，如果移位前后符号位发生变化，则 OF = 1，表示左移后结果溢出。这是 SAL 与 SHL 的不同之处。

SAR：算术右移，每右移一次，操作数的最低位送入 CF，并在高位补符号。

ROL：循环左移，每左移一次，最高位移到最低位，并送入 CF。

ROR：循环右移，每右移一次，最低位移到最高位，并送入 CF。

RCL: 带循环左移, 将 CF 作为操作数的一部分循环左移。

RCR: 带循环右移, 将 CF 作为操作数的一部分循环右移。

例 3.6 假设 short 型变量 x 被编译器分配在寄存器 AX 中, $R[ax] = FF80H$, 则以下汇编代码段执行后变量 x 的机器数和真值分别是多少?

```
movw    %ax, %dx
salw    $2, %ax
addl    %dx, %ax
sarw    %1, %ax
```

解 显然这里的汇编指令是 GCC 默认的 AT&T 格式, \$2 和 \$1 分别表示立即数 2 和 1。假设上述代码段执行前 $R[ax] = x$, 则执行 $((x << 2) + x) >> 1$ 后, $R[ax] = 5x/2$ 。算术左移时, AX 中的内容在移位前、后符号未发生变化, 故 OF=0, 没有溢出。最终 AX 的内容为 FEC0H, 解释为 short 型整数时, 其值为 -320。验证: $x = -128$, $5x/2 = -320$ 。经验证, 结果正确。 ■

3.3.4 控制转移指令

IA-32 中指令执行的顺序由 CS 和 EIP 确定。正常情况下, 指令按照它们在存储器中的存放顺序一条一条地按序执行, 但是, 在有些情况下, 程序需要转移到另一段代码去执行, 可以采用改变 CS 和 EIP, 或者仅改变 EIP 的方法来实现转移。第一种称为段间转移, 也叫远转移, 转移目标的属性为 FAR; 第二种称为段内转移, 分近转移和短转移, 转移目标的属性分别为 NEAR 和 SHORT。

段内转移和段间转移都有直接转移和间接转移之分。直接转移是指转移的目标地址作为立即数直接出现在指令的机器码中; 间接转移则是指转移的目标地址间接存储在某一寄存器或存储单元中。

目标转移地址的计算方法有两种。一种是通过将当前 EIP 的值增加或减少某一个值, 也就是以当前指令为中心往前或往后转移, 称为相对转移; 另一种是以新的值代替当前 EIP 的值, 称为绝对转移。在 IA-32 指令系统中, 所有段内直接转移都是相对转移, 所有段内间接转移和段间转移都是绝对转移。

IA-32 提供了多种控制转移指令, 有无条件转移指令、条件转移指令、条件设置指令、调用/返回指令和中断指令等。这些指令中, 除中断指令外, 其他指令都不影响状态标志位, 但有些指令的执行受状态标志的影响。与条件转移指令和条件设置指令相关的还有条件传送指令。

1. 无条件转移指令

无条件转移到转移目标地址处执行。按不同的寻址方法可分为以下 6 种指令形式。

JMP SHORT DST: 段内直接短转移, 在 $-128B \sim +127B$ 范围相对转移, DST 为标号。

JMP NEAR PTR DST: 段内直接近转移, 在 $\pm 32KB$ 范围内相对转移, DST 为标号。

JMP DST: 段内间接转移, 在 64KB 范围内绝对转移, DST 为寄存器。

JMP WORD PTR DST: 段内间接转移, 在 64KB 范围内绝对转移, DST 为存储单元。

JMP FAR PTR DST: 段间直接转移, 段外绝对转移, DST 为标号。

JMP DWORD PTR DST: 段间间接转移, 段外绝对转移, DST 为存储单元。

2. 条件转移指令

条件转移指令以条件标志或者条件标志位的逻辑运算结果作为转移依据。如果满足转移条件, 则程序转移到由标号 label 确定的目标地址处执行; 否则继续执行下一条指令。这类指令都采用相对转移方式在段内直接转移。表 3.6 列出了常用条件转移指令的转移条件。

表 3.6 条件转移指令

序号	指令	转移条件	说明
1	jc label	CF = 1	有进位/借位
2	jnc label	CF = 0	无进位/借位
3	je/jz label	ZF = 1	相等/等于零
4	jne/jnb label	ZF = 0	不相等/不等于零
5	js label	SF = 1	是负数
6	jns label	SF = 0	是非负数
7	jo label	OF = 1	有溢出
8	jno label	OF = 0	无溢出
9	ja/jnbe label	CF = 0 AND ZF = 0	无符号整数 $A > B$
10	jae/jnb label	CF = 0 OR ZF = 1	无符号整数 $A \geq B$
11	jbe/jnae label	CF = 1 AND ZF = 0	无符号整数 $A < B$
12	jbe/jna label	CF = 1 OR ZF = 1	无符号整数 $A \leq B$
13	jg/jnle label	SF = OF AND ZF = 0	带符号整数 $A > B$
14	jge/jnl label	SF = OF OR ZF = 1	带符号整数 $A \geq B$
15	jl/jnge label	SF $\neq$ OF AND ZF = 0	带符号整数 $A < B$
16	jle/jng label	SF $\neq$ OF OR ZF = 1	带符号整数 $A \leq B$

在 2.7.4 节中提到, IA-32 中不管高级语言程序中定义的变量是带符号整数还是无符号整数, 对应的加(减)法指令都是一样的, 都是在如图 2.6 所示的电路中执行。每条加(减)法指令执行以后, 会根据运算结果产生相应的进/借位标志 CF、符号标志 SF、溢出标志 OF 和零标志 ZF 等, 并保存到标志寄存器(FLAGS/EFLAGS)中。

对于比较大小后进行分支转移的情况, 通常在条件转移指令前面的是比较指令或减法指令, 因此, 大多是通过减法来获得标志信息, 然后再根据标志信息来判定两个数的大小, 从而决定该转移到何处执行指令。对于无符号整数的情况, 判断大小时使用的是 CF 和 ZF 标志。ZF = 1 说明两数相等, CF = 1 说明有借位, 是“小于”的关系, 通过对 ZF 和 CF 的组合, 得到表 3.6 中序号 9、10、11 和 12 这四条指令中的结论。对于带符号整数的情况, 判断大小时使用 SF、OF 和 ZF 标志。ZF = 1 说明两数相等, SF = OF 时说明结果是以下两种情况之一: ① 两数之差为正数(SF = 0)且结果未溢出(OF = 0); ② 两数之差为负数(SF = 1)且结果溢出(OF = 1)。这两种情况显然反映的是“大于”关系。同样, 若 SF $\neq$ OF, 则反映“小于”关系。带符号整数比较时对应表 3.6 中序号 13、14、15 和 16 这四条指令。

假设被减数的机器数为 X , 减数的机器数为 Y , 则在如图 2.6 所示的整数加减运算器中计算两数的差时, 计算公式为: $X - Y = X + (-Y)_{补}$ 。现举两个例子来说明上述无符号整数和带

符号整数的大小判断规则。

假定 $X = 1001$, $Y = 1100$, 则 $\text{Sub} = 1$, $Y' = 0011$, 加法器中的运算为 $1001 - 1100 = 1001 + 0011 + 1 = (0)1101$, 因此 $\text{ZF} = 0$, $\text{C} = 0$ 。若是无符号整数比较, 则是 9 和 12 相比, 是“小于”的关系, 此时 $\text{CF} = \text{C} \oplus \text{Sub} = 1$, 满足表 3.6 中序号 11 对应指令中的条件; 若是带符号整数比较, 则是 -7 和 -4 比较, 显然也是“小于”关系, 此时符号位为 1, 即 $\text{SF} = 1$, 而根据两个加数符号相异一定不会溢出的原则, 得到 $\text{OF} = 0$, 因而 $\text{SF} \neq \text{OF}$, 满足表 3.6 中序号 15 对应指令中的条件。

假定 $X = 1100$, $Y = 1001$, 则 $\text{Sub} = 1$, $Y' = 0110$, 加法器中的运算为 $1100 - 1001 = 1100 + 0110 + 1 = (1)0011$, 因此 $\text{ZF} = 0$, $\text{C} = 1$ 。若是无符号数比较, 则是 12 和 9 相比, 是“大于”的关系, 显然此时 $\text{CF} = \text{C} \oplus \text{Sub} = 0$, 满足表 3.6 中序号 9 对应指令中的条件; 若是带符号整数比较, 则是 -4 和 -7 比较, 也是“大于”的关系, 显然此时 $\text{SF} = 0$ 且 $\text{OF} = 0$, 即 $\text{SF} = \text{OF}$, 满足表 3.6 中序号 13 对应指令中的条件。

3. 条件设置指令

条件设置指令用来将条件标志组合得到的条件值设置到一个 8 位通用寄存器中, 其设置的条件值与表 3.6 中条件转移指令的转移条件值完全一样, 指令助记符也类似, 只要将 J 换成 SET 即可。其格式为:

SETcc DST

DST 通常是一个 8 位寄存器。例如, 假定将组合条件值存放在 DL 寄存器中, 则对应表 3.6 中序号 1 的指令为 “setc %dl”, 其含义为: 若 $\text{CF} = 1$, 则 $\text{R}[\text{dl}] = 1$; 否则 $\text{R}[\text{dl}] = 0$ 。对应表 3.6 中序号 14 的指令为 “setge %dl”, 其含义为: 若 $\text{SF} = \text{OF}$ 或 $\text{ZF} = 1$, 则 $\text{R}[\text{dl}] = 1$; 否则 $\text{R}[\text{dl}] = 0$ 。每个条件转移指令都有对应的条件设置指令。

例 3.7 以下各组指令序列用于将 x 和 y 的某种比较结果记录到 CL 寄存器。根据以下各组指令序列, 分别判断数据 x 和 y 在 C 语言程序中的数据类型, 并说明指令序列的功能。

第一组: `cmpb %eax, %edx // R[edx] = x, R[edx] = y`

`setb %cl`

第二组: `cmpb %eax, %edx // R[edx] = x, R[edx] = y`

`setne %cl`

第三组: `cmpw %ax, %dx // R[ax] = x, R[dx] = y`

`setl %cl`

第四组: `cmpb %al, %dl // R[al] = x, R[dl] = y`

`setae %cl`

解 CMP 指令通过执行减法来设置条件标志位, 每组中第二条 SETcc 指令中使用的条件标志都是由 x 和 y 相减后设置的。

第一组 x 和 y 都是 32 位数据, 指令 `setb` 对应表 3.6 中序号为 11 的指令, 设置条件为 $\text{CF} = 1$ 且 $\text{ZF} = 0$, 说明是无符号整数小于比较, 因此, x 和 y 可能是 unsigned、unsigned long 或指针型数据。

第二组 x 和 y 都是 32 位数据, 指令 `setne` 对应表 3.6 中序号为 4 的指令, 设置条件为 $\text{ZF} = 0$,

说明是两个位串的不相等比较, 因此, x 和 y 可能是 unsigned、int、unsigned long、long 或指针型数据。

第三组 x 和 y 都是 16 位数据, 指令 `setl` 对应表 3.6 中序号为 15 的指令, 设置条件为 $SF \neq OF$ 且 $ZF = 0$, 说明是带符号整数小于比较, 因此, x 和 y 只能是 short 型数据。

第四组 x 和 y 都是 8 位数据, 指令 `setae` 对应表 3.6 中序号为 10 的指令, 设置条件为 $CF = 0$ 或 $ZF = 1$, 说明是无符号整数大于等于比较, 因此, x 和 y 只能是 unsigned char 型数据。 ■

例 3.8 以下各组指令序列用于测试变量 x 的某种特性, 并将测试结果记录到 CL 寄存器。根据以下各组指令序列, 分别判断数据 x 在 C 语言程序中的数据类型, 并说明指令序列的功能。

第一组: `testl %eax, %eax // R[ $eax$ ] =  $x$`
`sete %cl`

第二组: `testl %eax, %eax // R[ $eax$ ] =  $x$`
`setge %cl`

第三组: `testw %ax, %ax // R[ $ax$ ] =  $x$`
`setns %cl`

第四组: `testb %al, %al // R[ $al$ ] =  $x$`
`setz %cl`

解 TEST 指令执行后, $OF = CF = 0$, 而 ZF 和 SF 则根据两个操作数相“与”的结果来设置: 若结果为全 0, 则 $ZF = 1$; 若最高位为 1, 则 $SF = 1$ 。前三组的 TEST 指令对 x 和 x 相“与”得到的是 x 本身。

第一组 x 为 32 位数据, 指令 `sete` 对应表 3.6 中序号为 3 的指令, 设置条件为 $ZF = 1$, 因而是对位串 x 判断是否等于 0, 显然, x 可能是 unsigned、int 或指针型数据。

第二组 x 为 32 位数据, 指令 `setge` 对应表 3.6 中序号为 14 的指令, 设置条件为 $SF = OF$ 或 $ZF = 1$, 因为 $OF = 0$, 所以设置条件转换为 $SF = 0$ 或 $ZF = 1$, 即判断 x 的符号是否为正或 x 是否为 0, 说明是带符号整数大于等于 0 比较, 因此, x 可能是 int 或 long 型数据。

第三组 x 为 16 位数据, 指令 `setns` 对应表 3.6 中序号为 6 的指令, 设置条件为 $SF = 0$, 说明是带符号整数是否为非负数比较, 即判断 x 是否大于等于 0, 因此, x 只能是 short 型数据。

第四组的 TEST 指令对 x 和 0x0F 相“与”, 析取 x 的低 4 位, x 为 8 位数据, 指令 `setz` 对应表 3.6 中序号为 3 的指令, 设置条件为 $ZF = 1$, 因而是对 TEST 指令析取出的位串判断是否为 0, 即判断 x 的低 4 位是否为 0, 因此, x 可能是 char 或 unsigned char 型数据。 ■

4. 条件传送指令

该类指令的功能是, 如果符合条件就进行传送操作, 否则什么都不做。设置的条件和表 3.6 中的条件转移指令的转移条件完全一样, 指令助记符也类似, 只要将 J 换成 CMOV 即可。其格式为:

`CMOVcc DST, SRC`

源操作数 SRC 可以是 16 位或 32 位寄存器或存储器操作数, 传送目的地 DST 必须是 16 位

或 32 位寄存器。例如，对应表 3.6 中序号 1 的指令“cmovc %eax, %edx”，其含义为：若 $CF = 1$ ，则 $R[edx] \leftarrow R[edx]$ ；否则什么都不做。对应序号 14 的指令“cmovge (%eax), %edx”，其含义为：若 $SF = OF$ 或 $ZF = 1$ ，则 $R[edx] \leftarrow M[R[edx]]$ ；否则什么都不做。

5. 调用和返回指令

为便于模块化程序设计，往往把程序中某些具有独立功能的部分编写成独立的程序模块，称之为子程序。这些子程序可以被主程序调用，并且执行完毕后又返回主程序继续执行原来的程序。子程序的使用有助于提高程序的可读性，并有利于代码重用，它是程序员进行模块化编程的重要手段。子程序的使用主要通过过程调用或函数调用实现，为叙述方便起见，本书将过程（调用）和函数（调用）统称为过程（调用）。为实现这一功能，IA-32 提供了以下两条指令。

(1) 调用指令

调用指令 CALL 是一种无条件转移指令，跳转方式与 JMP 指令类似。它具有两个功能：

① 将返回地址入栈（相当于 PUSH 操作）；② 跳转到指定地址处执行。执行时，首先将当前 EIP 或 CS:EIP 的内容（即返回地址，相当于 CALL 指令下面一条指令的地址）入栈，然后将调用目标地址（即子程序的首地址）装入 EIP 或 CS:EIP，以将控制转移到被调用的子程序执行。显然，CALL 指令会修改栈指针 ESP。

CALL 指令有以下 5 种基本类型。

CALL NEAR PTR DST：段内直接调用（NEAR PTR 可省略），DST 为子程序入口地址。

CALL DST：段内间接调用，DST 为寄存器。

CALL WORD PTR DST：段内间接调用，DST 为存储单元。

CALL FAR PTR DST：段间直接调用，DST 为子程序入口地址。

CALL DWORD PTR DST：段间间接调用，DST 为存储单元。

(2) 返回指令

返回指令 RET 也是一种无条件转移指令，通常放在子程序的末尾，使子程序执行后返回主程序继续执行。该指令执行过程中，返回地址被从栈顶取出（相当于 POP 指令），并送到 EIP 寄存器（段内或段间调用时）和 CS 寄存器（仅段间调用）。显然，RET 指令会修改栈指针。若 RET 指令带有一个立即数 n ，则当它完成上述操作后，还会执行 $R[esp] \leftarrow R[esp] + n$ 操作，从而实现预定的修改栈指针 ESP 的目的。

6. 中断指令

中断的概念和过程调用有些类似，两者都是将返回地址先压栈，然后转到某个程序去执行。它们的主要区别是：① 过程调用跳转到一个用户事先设定好的子程序，而中断跳转则是转向系统事先设定好的中断服务程序；② 过程调用可以是 NEAR 或 FAR 类型，能直接或间接跳转，而中断跳转通常是段间间接转移，因为中断处理会从用户态转到内核态执行；③ 过程调用只保存返回地址，而中断指令还要使标志寄存器入栈保存。IA-32 提供了以下关于中断的指令。

INT n ： n 为中断类型号，取值范围为 0 ~ 255。

iret/iretd：中断返回指令，偏移地址和段地址送 CS:EIP，并恢复标志寄存器。

into: 溢出中断指令, 若 $OF = 1$, 产生型号为 4 的异常, 进入相应的异常处理。

sysenter: 快速进入系统调用指令。

sysexit: 快速退出系统调用指令。

与中断相关的内容详见第 7 章和第 8 章。

3.3.5 x87 浮点处理指令

IA-32 的浮点处理架构有两种: 较早的一种是与 x86 配套的浮点协处理器 x87 架构, 采用栈结构; 另一种是由 MMX 发展而来的 SSE 指令集架构, 采用的是单指令多数据 (Single Instruction Multi Data, SIMD) 技术, 包括 SSE、SSE2、SSE3、SSSE3、SSE4 等。对于 IA-32 架构, GCC 默认生成 x87 指令集代码, 如果想要生成 SSE 指令集代码, 则需要设置适当的编译选项。

x87 FPU 有一个浮点寄存器栈, 栈的深度为 8, 每个浮点寄存器有 80 位。根据指令的操作功能, x87 浮点数指令可分为浮点数装入、浮点数存储、整数浮点数转换、浮点算术运算和浮点数测试比较等几种类型。

浮点数装入指令用来将存储单元或原栈中的浮点数装入到浮点寄存器栈的栈顶 $ST(0)$ 。由于浮点寄存器宽度为 80 位, 所以, 这些指令中指定的从存储单元中取出的浮点数不管是 32 位还是 64 位, 都要先转换为 80 位扩展精度格式后再装入栈顶。

浮点数存储指令用来将浮点寄存器栈的栈顶 $ST(0)$ 中的元素存储到存储单元中。由于浮点寄存器宽度为 80 位, 所以, 需要先将 80 位扩展精度格式转换为 32 位或 64 位格式后再存储到指定存储单元中。

由于 x87 中浮点寄存器为 80 位, 而在内存中的浮点数可能占 32 位、64 位或 96 位, 因而在内存单元和浮点数寄存器之间进行数据传送的过程中, 可能会丢失精度而造成错误计算结果, 需要引起注意。

图 3.7 所示是两个功能完全相同的程序, 但是, 使用 gcc 的一些旧版本对它们进行编译时, 会发生以下情况: 使用 gcc-O2 编译程序一时, 运行后输出结果是 0, 也就是说 a 不等于 b ; 使用 gcc-O2 编译程序二时, 运行后输出结果却是 1, 也就是说 a 等于 b 。两个几乎一模一样的程序, 但运行结果不一致。

程序一: <pre>#include <stdio.h> double f(int x) { return 1.0 / x ; } void main() { double a, b; int i ; a = f(10) ; b = f(10) ; i = a == b; printf("%d\n",i) ; }</pre>	程序二: <pre>#include <stdio.h> double f(int x){ return 1.0 / x ; } void main(){ double a, b, c; int i ; a = f(10) ; b = f(10) ; c = f(10) ; i = a == b ; printf("%d\n",i) ; }</pre>
--	--

图 3.7 浮点运算示例

出现上述情况的主要原因是存储单元和浮点数寄存器之间进行数据传送的过程中丢失了精度。

gcc 对于程序一的处理过程如下：先计算 $a = f(10)$ ，然后将其写到存储单元，由于 $0.1 = 0.00011[0011]B$ ，即转换为二进制数时是无限循环小数，因此无法用有限位数的二进制精确表示。在将其从 80 位的浮点寄存器写入到 64 位（double 类型占 64 位）的存储区域时，产生了精度损失。然后，计算 $b = f(10)$ ，这个结果并没被写入存储器中，这样，在计算关系表达式“ $a == b$ ”时，直接将损失了精度的 a 与栈顶中的 b 进行比较，由于 b 没有精度损失，因此 a 与 b 不相等。

gcc 对于程序二的处理过程如下： a 与 b 在计算完成之后，由于程序中多了一个 $c = f(10)$ 的计算，使得 gcc 必须把先前计算的 a 和 b 都写入存储器中，于是都产生了精度损失，因而它们的值完全一样，再把它们读到浮点寄存器栈中进行比较时，得出的结果就是 a 等于 b 。

使用较新版本的 gcc（如 gcc 4.4.7）编译时，用 -O2 优化选项的情况下，两个程序输出的结果都为 1，并没有发生题目中所说的情况，对它们反汇编后发现，两个程序都没有计算 $f(10)$ 就直接把 i 设置成 1 了，显然编译器进行了相应的优化。

上述 gcc 旧版本出现的问题主要是编译器没有处理好。从这个例子可以看出，编译器的设计和硬件结构是紧密相关的。对于编译器设计者来说，只有真正了解底层硬件结构和真正理解指令集体系结构，才能够翻译出没有错误的目标代码，并为程序员完全屏蔽掉硬件实现的细节，方便应用程序员开发出可靠的程序。对于应用程序开发者来说，也只有真正了解底层硬件的结构，才有能力编制出高效的程序，能够快速定位出错的地方，并对程序的行为作出正确的判断。

3.3.6 MMX/SSE 指令集

在多媒体应用中，图形、图像、视频和音频处理存在大量具有共同特征的操作，因而 Intel 公司于 1997 年推出了 MMX (Multi Media eXtension, 多媒体扩展) 指令集，它是一种多媒体指令增强技术，包括 57 条多媒体处理指令，通过这些指令可以一次处理多个数据。但是，因为 MMX 指令与 x87 FPU 共用同一套寄存器。因而 MMX 指令与 x87 浮点运算指令不能同时执行，降低了整个系统的运行性能。

随着网络、通信、语音、图形、图像、动画和音/视频等多媒体处理软件对处理器性能越来越高的要求，Intel 在多能奔腾以后的处理器中加入了更多流式 SIMD 扩展 (Stream SIMD Extension, 简称 SSE) 指令集，包括 SSE、SSE2、SSE3、SSSE3、SSE4 等，这些都是典型的数据级并行处理技术。

SSE 指令集最早是 1999 年 Intel 在 Pentium III 处理器中推出的，包括了 70 条指令，其中包含提高 3D 图形运算效率的 50 条 SIMD 浮点运算指令、12 条 MMX 整数运算增强指令和 8 条优化内存中连续数据块传输指令。理论上这些指令对图像处理、浮点运算、3D 运算、视频处理、音频处理等诸多多媒体应用起到全面强化的作用。SSE 兼容 MMX 指令，它可以通过 SIMD 技术在单时钟周期内并行处理 4 个单精度浮点数据来有效地提高浮点运算速度。

2001 年 Intel 在 Pentium 4 中发布了一套包括 144 条新指令的 SSE2 指令集，提供了浮点

SIMD 指令、整数 SIMD 指令、浮点数和整数之间转换等指令。SSE2 增加了能处理 128 位整数和同时并行处理两个 64 位双精度浮点数的指令。为了更好地利用高速缓存，还新增了几条缓存指令，允许程序员控制已经缓存过的数据。

2004 年年初 Intel 公司在新款 Pentium 4 (P4E, Prescott 核心) 处理器中发布了 SSE3, 2005 年 4 月 AMD 公司也发表具备部分 SSE3 功效的处理器 Athlon 64, 此后的 x86 处理器几乎都具备 SSE3 的新指令集功能。SSE3 新增了 13 条指令, 其中一条用于视频解码, 两条用于线程同步, 其余用于复杂的数学运算、浮点数与整数之间的转换以及 SIMD 浮点运算, 使处理器对 DSP 及 3D 处理的性能大为提升。此外, SSE3 针对多线程应用进行优化, 使处理器原有的超线程功能得到了更好发挥。

2005 年, 作为 SSE3 指令集的补充版本, SSSE3 出现在酷睿微架构处理器中, 新增 16 条指令, 进一步增强了 CPU 在多媒体、图形图像和 Internet 等方面的处理能力。

2008 年 SSE4 指令集发布, 它被视为最重要的多媒体扩展指令集架构改进方式, 将延续了多年的 32 位架构升级至 64 位。SSE4 增加了 54 条指令, 其中 SSE4.1 指令子集包含 47 条指令, SSE4.2 包含 7 条指令。SSE4.1 主要针对向量绘图运算、3D 游戏加速、视频编码加速及协同处理加速等方面, 此外还加入了 6 条浮点运算增强指令, 这使得图形渲染处理性能和 3D 游戏效果等得到极大提升。除此之外, SSE4.1 指令集还加入了串流式负载指令, 可提高图形帧缓冲区的数据读取频宽, 理论上可获取完整的缓存行, 即单次读取 64 位而非原来的 8 位。SSE4.2 主要针对字符串和文本处理。例如, 对 XML 应用进行高速查找及对比, 在如 Web 服务器应用等方面有显著的性能改善。

下面用一个简单的例子来比较普通指令与数据级并行指令的执行速度。为了使比较结果尽量不受访存操作的影响, 以下例子中的运算操作数主要是寄存器操作数。此外, 为了使比较结果尽量准确, 例子中设置了较大的循环次数值, 为 $0x4000000 = 2^{26}$ 。例子只是为了说明指令执行速度的快慢, 并没有考虑结果是否溢出。

图 3.8 给出了采用普通指令的累加函数 dummy_add 对应的汇编代码, 其中粗体字部分为循环体, 循环控制指令 loop 执行时, 先检测寄存器 ECX 的内容, 若为 0 则退出循环, 否则 ECX 的内容减 1, 并再次进入循环体的第一条指令开始执行, 循环体的第一条指令地址由 loop 指令指出。

```

080484f0 <dummy_add>:
080484f0:  55                push    %ebp
080484f1:  89 e5             mov     %esp, %ebp
080484f3:  b9 00 00 00 04    mov     $0x4000000, %ecx
080484f8:  b0 01            mov     $0x1, %al
080484fa:  b3 00            mov     $0x0, %bl
080484fc:  00 c3            add     %al, %bl
080484fe:  e2 fc            loop    80484fc <dummy_add+0xc>
08048500:  5d                pop     %ebp
08048501:  c3                ret

```

图 3.8 采用普通指令的累加函数

图 3.9 给出了采用数据级并行指令的累加函数 dummy_add_sse 对应的汇编代码, 其中粗体字部分为循环体。

```

08048510 <dummy_add_sse>:
8048510:    55                push    %ebp
8048511:    b8 00 9d 04 10    mov     $0x10049d00, %eax
8048516:    89 e5             mov     %esp, %ebp
8048518:    53                push    %ebx
8048519:    bb 20 9d 04 14    mov     $0x14049d20, %ebx
804851e:    b9 00 00 40 00    mov     $0x400000, %ecx
8048523:    66 0f 6f 00       movdqa  (%eax), %xmm0
8048527:    66 0f 6f 0b       movdqa  (%ebx), %xmm1
804852b:    66 0ffc c8        paddb   %xmm0, %xmm1
804852f:    e2fa             loop    804852b <dummy_add_sse+0x1b>
8048531:    5b                pop     %ebx
8048532:    5d                pop     %ebp
8048533:    c3                ret

```

图 3.9 采用 SSE 指令的累加函数

从图 3.8 可以看出, dummy_add 函数中, 每次循环只完成一个字节的累加, 而在图 3.9 所示的 dummy_add_sse 函数中, 每次循环执行的指令为 “paddb %xmm0, %xmm1”, 也即每次循环并行完成两个 XMM 寄存器中的 16 个一字节数据的累加, 对于与 dummy_add 同样的工作量, 循环次数应为其 1/16, 即 $(0x4000000 \gg 4) = 0x400000 = 2^{22}$, 因而, 可以预期它所用的时间大约只有 dummy_add 的 1/16。

在相同环境下测试两个函数的执行时间, dummy_add 所用时间约为 22.643 816s, 而 dummy_add_sse 所用时间约为 1.411 588s, 两者之比大约为 16.041 378。这与预期的结果一致。

dummy_add_sse 函数中用到的 SSE 指令有两种, 除了 paddb 以外, 还有一种是 movdqa 指令, 它的功能是将双四字 (128 位) 从源操作数处移到目标操作数处。该指令可用于在 XMM 寄存器与 128 位存储单元之间移入/移出双四字, 或在两个 XMM 寄存器之间移动。该指令的源操作数或目标操作数是存储器操作数时, 操作数必须是 16 字节边界对齐, 否则将发生一般保护性异常 (#GP)。若需要在未对齐的存储单元中移入/移出双四字, 可以使用 movdqu 指令。更多有关 SSE 指令集的内容请参看 Intel 的相关资料。

3.4 C 语言程序的机器级表示

用任何汇编语言或高级语言编写的源程序最终都必须翻译 (汇编、解释或编译) 成以指令形式表示的机器语言, 才能在计算机上运行。本节简单介绍高级语言源程序转换为机器代码过程中涉及的一些基本问题。为方便起见, 本节选择具体语言进行说明, 高级语言和机器语言分别选用 C 语言和 IA-32 指令系统。其他情况下, 其基本原理不变。

3.4.1 过程调用的机器级表示

程序员可使用参数将过程与其他程序及数据进行分离。调用过程只要传送输入参数给被调用过程, 最后再由被调用过程返回结果参数给调用过程。引入过程使得每个程序员只需要关注本模块中函数或过程的编写任务。本书主要介绍 C 语言程序的机器级表示, 而 C 语言用函数来实现过程, 因此, 本书中的过程和函数是等价的。

将整个程序分成若干模块后,编译器对每个模块可以分别编译。为了彼此统一,并能配合操作系统工作,编译的模块代码之间必须遵循一些调用接口约定,这些约定由编译器强制执行,汇编语言程序员也必须强制按照这些约定执行,包括寄存器的使用、栈帧的建立和参数传递等。

1. IA-32 中用于过程调用的指令

在 3.3.4 节中提到的调用指令 CALL 和返回指令 RET 是用于过程调用的主要指令,它们都属于一种无条件转移指令,都会改变程序执行的顺序。为了支持嵌套和递归调用,通常利用栈来保存返回地址、入口参数和过程内部定义的非静态局部变量,因此,CALL 指令在跳转到被调用过程执行之前先要把返回地址压栈,RET 指令在返回调用过程之前要从栈中取出返回地址。

2. 过程调用的执行步骤

假定过程 P 调用过程 Q,则 P 称为调用者,Q 称为被调用者。过程调用的执行步骤如下。

- ① P 将入口参数(实参)放到 Q 能访问到的地方。
- ② P 将返回地址存到特定的地方,然后将控制转移到 Q。
- ③ Q 保存 P 的现场,并为自己的非静态局部变量分配空间。
- ④ 执行 Q 的过程体(函数体)。
- ⑤ Q 恢复 P 的现场,并释放局部变量所占空间。
- ⑥ Q 取出返回地址,将控制转移到 P。

上述步骤中,第①~②步是在过程 P 中完成的,其中第②步是由 CALL 指令实现的,通过 CALL 指令,将控制从过程 P 转移到了过程 Q。第③~⑥步都在被调用过程 Q 中完成,在执行 Q 过程体之前的第③步通常称为准备阶段,用于保存 P 的现场并为 Q 的非静态局部变量分配空间,在执行 Q 过程体之后的第⑤步通常称为结束阶段,用于恢复 P 的现场并释放 Q 的局部变量所占空间,最后在第⑥步通过执行 RET 指令返回到过程 P。每个过程的功能主要是通过过程体的执行来完成的。如果过程 Q 有嵌套调用的话,那么在 Q 的过程体和被 Q 调用的过程函数中又会有上述 6 个步骤的执行过程。

小贴士

因为每个处理器只有一套通用寄存器,所以通用寄存器是每个过程共享的资源,当从调用过程跳转到被调用过程执行时,原来在通用寄存器中存放的调用过程的内容,不能因为被调用过程要使用这些寄存器而被破坏掉,因此,在被调用过程使用这些寄存器前,在准备阶段先将寄存器中的值保存到栈中,用完以后,在结束阶段再从栈中将这些值重新写回到寄存器中,这样,回到调用过程后,寄存器中存放的还是调用过程中的值。通常将通用寄存器中的值称为现场。

并不是所有通用寄存器中的值都由被调用过程保存,通常调用过程保存一部分,被调用过程保存一部分。每个 ISA 都会有一个寄存器使用约定,其中会规定哪些寄存器由调用者保存,哪些由被调用者保存。

3. 过程调用所使用的栈

从上述执行步骤来看,在过程调用中,需要为入口参数、返回地址、调用过程执行时用到的寄存器、被调用过程中的非静态局部变量、过程返回时的结果等数据找到存放空间。如果有

足够的寄存器，最好把这些数据都保存在寄存器中，这样，CPU 执行指令时，可以快速地从寄存器取得这些数据进行处理。但是，用户可见寄存器数量有限，并且它们是所有过程共享的，某时刻只能被一个过程使用；此外，对于过程中使用的一些复杂类型的非静态局部变量（如数组和结构等类型数据）也不可能保存在寄存器中。因此，除了寄存器外，还需要有一个专门的存储区域来保存这些数据，这个存储区域就是栈（stack）。那么，上述数据中哪些存放在寄存器，哪些存放在栈中呢？寄存器和栈的使用又有哪些规定呢？

4. IA-32 的寄存器使用约定

尽管硬件对寄存器的用法几乎没有任何规定，但是，因为寄存器是被所有过程共享的资源，若一个寄存器在调用过程中存放了特定的值 x ，在被调用过程执行时，它又被写入了新的值 y ，那么当从被调用过程返回到调用过程执行时，该寄存器中的值就不是当初的值 x ，这样，调用过程的执行结果就会发生错误。因而，在实际使用寄存器时需要遵循一定的惯例，使机器级程序员、编译器和库函数等都按照统一的约定处理。

IA-32 规定，寄存器 EAX、ECX 和 EDX 是调用者保存寄存器。当过程 P 调用过程 Q 时，Q 可以直接使用这三个寄存器，不用将它们的值保存到栈中，这也意味着，如果 P 在从 Q 返回后还要用这三个寄存器的话，P 应在转到 Q 之前先保存它们的值，并在从 Q 返回后先恢复它们的值再使用。寄存器 EBX、ESI、EDI 是被调用者保存寄存器，Q 必须先将它们的值保存到栈中再使用它们，并在返回 P 之前先恢复它们的值。还有另外两个寄存器 EBP 和 ESP 则分别是帧指针寄存器和栈指针寄存器，分别用来指向当前栈帧的底部和顶部。

5. IA-32 的栈、栈帧及其结构

IA-32 使用栈来支持过程的嵌套调用，过程的入口参数、返回地址、被保存寄存器的值、被调用过程中的非静态局部变量等都会被压入栈中。IA-32 中可通过执行 MOV、PUSH 和 POP 指令存取栈中元素，用 ESP 寄存器指示栈顶，栈从高地址向低地址增长。

每个过程都有自己的栈区，称为栈帧（stack frame），因此，一个栈由若干栈帧组成，每个栈帧用专门的帧指针寄存器 EBP 指定起始位置。因而，当前栈帧的范围在帧指针 EBP 和栈指针 ESP 指向区域之间。过程执行时，由于不断有数据入栈，所以栈指针会动态移动，而帧指针可以固定不变。对程序来说，用固定的帧指针来访问变量要比用变化的栈指针方便得多，也不易出错，因此，在一个过程内对栈中信息的访问大多通过帧指针 EBP 进行。

假定 P 是调用过程，Q 是被调用过程。图 3.10 给出了 IA-32 在过程 Q 被调用前、过程 Q 执行中和从 Q 返回到过程 P 这三个时点栈中的状态变化。

在调用过程 P 中遇到一个函数调用（假定被调用函数为 Q）时，在调用过程 P 的栈帧中保存的内容如图 3.10a 所示。首先，P 确定是否需要将某些调用者保存寄存器保存到自己的栈帧中；然后，将入口参数按序保存到 P 的栈帧中，参数压栈的顺序是先右后左；最后执行 CALL 指令，先将返回地址保存到 P 的栈帧中，然后转去执行被调用过程 Q。

在执行被调用函数 Q 的准备阶段，在 Q 的栈帧中保存的内容如图 3.10b 所示。首先，Q 将 EBP 的值保存到自己的栈帧（被调用过程 Q 的栈帧）中，并设置 EBP 指向它，即 EBP 指向当前栈帧的底部；然后，根据需要确定是否将被调用者寄存器保存到 Q 的栈帧中；最后在栈中为 Q 中的非静态局部变量分配空间。通常，如果非静态局部变量为简单变量且有空闲的通用寄存

器, 则编译器会将通用寄存器分配给局部变量, 但是, 对于非静态局部变量是数组或结构等复杂数据类型的情况, 则只能在栈中为其分配空间。

在 Q 过程体执行后的结束阶段, Q 会恢复被调用者保存寄存器和 EBP 寄存器的值, 并使 ESP 指向返回地址, 这样, 栈中的状态又回到了开始执行 Q 时的状态, 如图 3.10c 所示。这时, 执行 RET 指令便能取出返回地址, 以回到过程 P 继续执行。

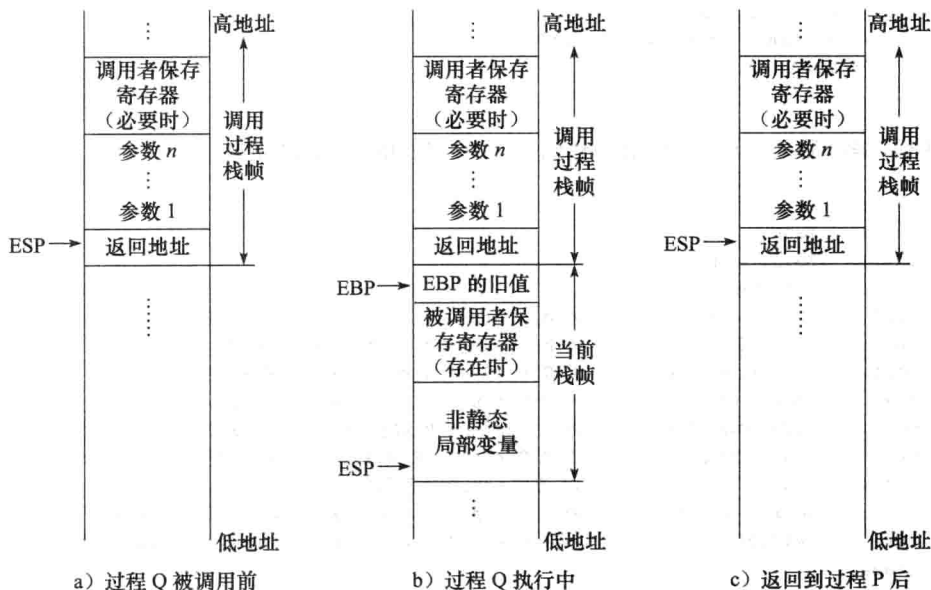


图 3.10 过程调用过程中栈和栈帧的变化

从图 3.10 可看出, 在 Q 的过程体执行时, 入口参数 1 的地址总是 $R[\text{ebp}] + 8$, 后面若干入口参数的地址只要在此地址基础上加上相应数据类型对应的字节数即可。

6. 变量的作用域和生存期

从图 3.10 所示的过程调用前、后栈的变化过程可以看出, 在当前过程 Q 的栈帧中保存的 Q 内部的非静态局部变量只在 Q 执行过程中有效, 当从 Q 返回到 P 后, 这些变量所占的空间全部被释放, 因此, 在 Q 过程以外, 这些变量是无效的。了解了上述过程, 就能很好地理解 C 语言中关于变量的作用域和生存期的问题。C 语言中的 auto 型变量就是过程 (函数) 内的非静态局部变量, 因为它通过编译器动态、自动地在栈中分配并在过程结束时释放, 因而其作用域仅限于过程内部且具有的仅是“局部生存期”。此外, auto 型变量可以和其他过程中的变量重名, 因为其他过程中的同名变量实际占用的是自己栈帧中的空间或静态数据区, 也就是说, 变量名虽相同但实际占用的存储单元不同, 它们分别在不同的栈帧中, 或一个在栈中另一个在静态数据区中。C 语言中的外部参照型变量和静态变量被分配在静态数据区, 而不是分配在栈中, 因而这些变量在整个程序运行期间一直占据着固定的存储单元, 它们具有“全局生存期”。

7. 一个简单的过程调用例子

下面以一个最简单的例子来说明过程调用的机器级实现。假定有一个过程 add 实现两个数相加, 另一个过程 caller 调用 add 以计算 $125 + 80$ 的值, 对应的 C 语言程序如下。

```
1 int add(int x, int y)
2 {
3     return x+y;
4 }
5
6 int caller()
7 {
8     int temp1 = 125;
9     int temp2 = 80;
10    int sum = add(temp1, temp2);
11    return sum
12 }
```

经 GCC 编译后 caller 过程对应的代码如下（//后面的文字是注释）。

```
1 caller:
2 pushl   %ebp
3 movl    %esp, %ebp
4 subl    $24, %esp
5 movl    $125, -12(%ebp) // M[R[ebp]-12]←125,即 temp1=125
6 movl    $80, -8(%ebp)  // M[R[ebp]-8]←80,即 temp2=80
7 movl    -8(%ebp), %eax // R[eax]←M[R[ebp]-8],即 R[eax]=temp2
8 mov     %eax, 4(%esp)   // M[R[esp]+4]←R[eax],即 temp2 入栈
9 movl    -12(%ebp), %eax // R[eax]←M[R[ebp]-12],即 R[eax]=temp1
10 movl    %eax, (%esp)    // M[R[esp]]←R[eax],即 temp1 入栈
11 call    add             // 调用 add,将返回值保存在 EAX 中
12 movl    %eax, -4(%ebp)  // M[R[ebp]-4]←R[eax],即 add 返回值送 sum
13 movl    -4(%ebp), %eax  // R[eax]←M[R[ebp]-4],即 sum 作为 caller 返回值
14 leave
15 ret
```

图 3.11 给出了 caller 栈帧的状态，其中，假定 caller 被过程 P 调用。图中 ESP 的位置是执行了第 4 条指令后 ESP 的值所指的位置，可以看出 GCC 为 caller 的参数分配了 24 字节的空间。从汇编代码中可以看出，caller 中只使用了调用者保存寄存器 EAX，没有使用任何被调用者保存寄存器，因而在 caller 栈帧中无需保存除 EBP 以外的任何寄存器的值；caller 有三个局部变量 temp1、temp2 和 sum，皆被分配在栈帧中；在用 call 指令调用 add 函数之前，caller 先将入口参数从右向左依次将 temp2 和 temp1 的值（即 80 和 125）保存到栈中。在执行 call 指令时再把返回地址压入栈中。此外，在最初进入 caller 时，还将 EBP 的值压入了栈中，因此 caller 的栈帧中用到的空间占 4 + 12 + 8 + 4 = 28 字节。但是，caller 的栈帧总共有 4 + 24 + 4 = 32 字节，其中浪费了 4 字节空间（未使用）。这是因为 GCC 为保证 x86 架构中数据的严格对齐而规定每个函数的栈帧大小必须是 16 字节的倍数。有关对齐规则，在后续的章节中介绍。

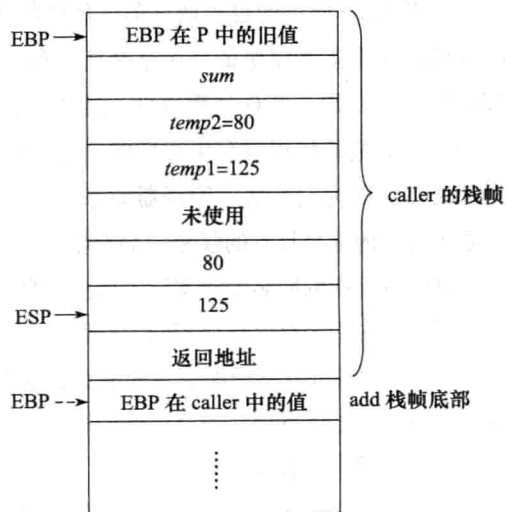


图 3.11 caller 和 add 的栈帧

call 指令执行后，add 函数的返回参数存放在

EAX 中, 因而 call 指令后面的两条指令中, 序号为 12 的 movl 指令用来将 add 的结果存入 sum 变量的存储空间, 其变量的地址为 $R[\text{ebp}] - 4$; 序号为 13 的 movl 指令用来将 sum 变量的值送返回值寄存器 EAX 中。

在执行 ret 指令之前, 应将当前栈帧释放掉并恢复旧 EBP 的值, 上述序号为 14 的 leave 指令实现了这个功能, leave 指令功能相当于以下两条指令的功能。其中, 第一条指令使 ESP 指向当前 EBP 的位置, 第二条指令执行后, EBP 恢复为 P 中的旧值, 并使 ESP 指向返回地址。

```
movl    %ebp, %esp
popl    %ebp
```

执行完 leave 指令后, ret 指令就从 ESP 所指处取返回地址, 以返回 P 执行。当然, 编译器也可通过 pop 指令和对 ESP 的内容做加法来进行退栈操作, 而不一定要使用 leave 指令。

add 过程比较简单, 经 GCC 编译并进行链接后对应的代码如下。

```
1 8048469:55          push    %ebp
2 804846a:89 e5          mov     %esp, %ebp
3 804846c:8b 45 0c       mov     0xc(%ebp), %eax
4 804846f:8b 55 08       mov     0x8(%ebp), %edx
5 8048472:8d 04 02       lea     (%edx,%eax,1), %eax
6 8048475:5d          pop     %ebp
7 8048476:c3          ret
```

通常, 一个过程对应的机器级代码都有三个部分: 准备阶段、过程体和结束阶段。

上述序号 1 和 2 的指令构成准备阶段的代码段, 这是最简单的准备阶段代码段, 它通过将当前栈指针 ESP 传送到 EBP 来完成将 EBP 指向当前栈帧底部的任务, 如图 3.11 所示, EBP 指向 add 栈帧底部, 从而可以方便地通过 EBP 获取入口参数。这里 add 的入口参数 x 和 y 对应的值 (125 和 80) 分别在地址为 $R[\text{ebp}] + 8$ 、 $R[\text{ebp}] + 12$ 的存储单元中。

上述序号 3、4 和 5 的指令序列是过程体代码段, 过程体结束时将返回值放在 EAX 中。这里好像没有加法指令, 实际上序号 5 的 lea 指令执行的是加法运算 $R[\text{edx}] + R[\text{eax}] * 1 = x + y$ 。

上述序号 6 和 7 的指令序列是结束阶段代码, 通过将 EBP 弹出栈帧来恢复 EBP 在 caller 过程中的值, 并在栈中退出 add 过程的栈帧, 使得执行到 ret 指令时栈顶中已经是返回地址。这里的返回地址应该是 caller 代码中序号为 12 的那条指令的地址。

add 过程中没有用到任何被调用者保存寄存器, 没有局部变量, 此外, add 是一个被调用过程, 并且不再调用其他过程, 即它是个叶子过程, 因而也没有入口参数和返回地址要保存, 因此, 在 add 的栈帧中除了需要保存 EBP 以外, 无需保留其他任何信息。

8. 按值传递参数和按地址传递参数

使用参数传递数据是 C 语言函数间传递数据的主要方式。C 语言中的数据类型分为基本数据类型和复杂数据类型, 而复杂数据类型中又分为构造类型和指针类型。C 语言的数据类型如图 3.12 所示。



图 3.12 C 语言中的数据类型

C 函数中的形式参数可以是基本类型变量名、构造类型变量名和指针类型变量名。对于不同类型的形式参数，其传递参数的方式不同，总体来说分为两种：按值传递和按地址传递。当形参是基本类型变量名时，采用按值传递方式；当形参是指针类型变量名或构造类型变量名时，采用按地址传递方式。显然，上面的 add 过程采用的是按值传递方式。

下面通过例子说明两种方式的差别。图 3.13 给出了两个相似的程序。

程序一： <pre>#include <stdio.h> main() { int a=15, b=22; printf("a=%d\tb=%d\n", a, b); swap(&a, &b); printf("a=%d\tb=%d\n", a, b); } swap(int *x, int *y) { int t=*x; *x=*y; *y=t; }</pre>	程序二： <pre>#include <stdio.h> main() { int a=15, b=22; printf("a=%d\tb=%d\n", a, b); swap(a, b); printf("a=%d\tb=%d\n", a, b); } swap(int x, int y) { int t=x; x=y; y=t; }</pre>
--	--

图 3.13 按值传递参数和按地址传送参数的程序示例

上述图 3.13 中两个程序的输出结果如图 3.14 所示。

程序一的输出： <pre>a=15 b=22 a=22 b=15</pre>	程序二的输出： <pre>a=15 b=22 a=15 b=22</pre>
---	---

图 3.14 图 3.13 中程序的输出结果

从图 3.14 可看出，程序一实现了 a 和 b 的值的交换，而程序二并没有实现对 a 和 b 的值进行交换的功能。下面从这两个程序的机器级代码来分析为何它们之间有这种差别。

首先，看一下这两个程序栈帧内容有何差别，图 3.15 中的粗体字处给出了它们之间的差别。

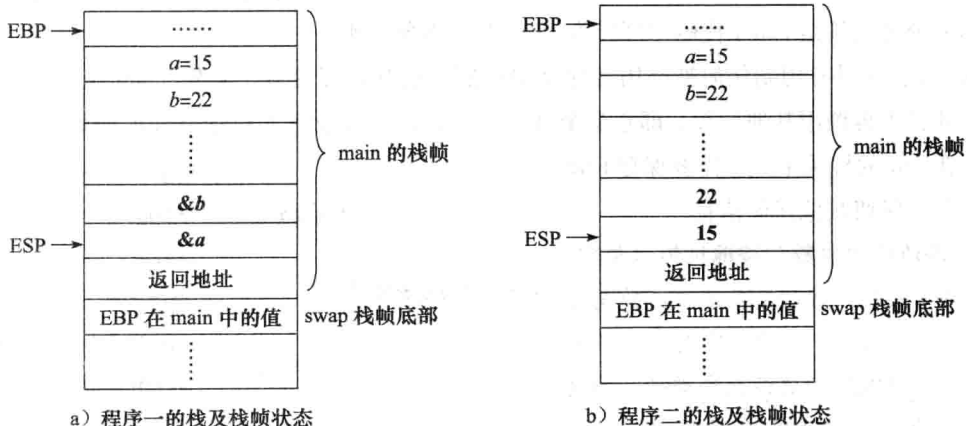


图 3.15 执行 swap 之前 main 的栈帧状态

如图 3.15 所示, 它们在栈中的状态仅在于压入栈中的参数不同。在图 3.15a 所示的程序一中, `main` 函数把变量 `a` 和 `b` 的地址作为实参压入了栈中, 而在图 3.15b 所示的程序二中, 则把变量 `a` 和 `b` 的值作为实参压入了栈中。

程序一和程序二对应的 `swap` 函数的机器级代码也不同。图 3.16 中给出两个程序对应的参数传递代码和 `swap` 函数的汇编代码 (AT&T 格式), 不同之处用粗体字表示。给出的代码假定 `swap` 函数的局部变量 `t` 分配在 `EDX` 中。

程序一汇编代码片段: <pre> main: leal -8(%ebp), %eax movl %eax, 4(%esp) leal -4(%ebp), %eax movl %eax, (%esp) call swap ret swap: // 以下是准备阶段 pushl %ebp movl %esp, %ebp pushl %ebx // 以下是过程体 movl 8(%ebp), %edx movl (%edx), %ecx movl 12(%ebp), %eax movl (%eax), %ebx movl %ebx, (%edx) movl %ecx, (%eax) // 以下是结束阶段 popl %ebx popl %ebp ret </pre>	程序二汇编代码片段: <pre> main: movl -8(%ebp), %eax movl %eax, 4(%esp) movl -4(%ebp), %eax movl %eax, (%esp) call swap ret swap: // 以下是准备阶段 pushl %ebp movl %esp, %ebp // 以下是过程体 movl 8(%ebp), %edx movl 12(%ebp), %eax movl %eax, 8(%ebp) movl %edx, 12(%ebp) // 以下是结束阶段 popl %ebp ret </pre>
--	--

图 3.16 两个程序对应的汇编代码片段

从图 3.16 可看出, 在给 `swap` 过程传递参数时, 程序一用了 `leal` 指令, 而程序二用的是 `movl` 指令, 因而程序一传递的是 `a` 和 `b` 的地址, 而程序二传递的是 `a` 和 `b` 的内容。

从图 3.16 可看出, 程序一的 `swap` 过程体比程序二的 `swap` 过程体多了三条指令。而且, 由于程序一的 `swap` 过程体更复杂, 使用了较多的寄存器, 除了三个调用者保存寄存器外, 还使用了被调用者保存寄存器 `EBX`, 它的值必须在准备阶段被保存到栈中, 而在结束阶段从栈中恢复。因而它比程序二又多了一条 `push` 指令和一条 `pop` 指令。

图 3.17 反映了执行 `swap` 过程后 `main` 的栈帧中的状态, 与图 3.15 中反映的执行 `swap` 前的情况进行对照发现, 粗体字处发生了变化。

因为程序一的 `swap` 函数的形式参数 `x` 和 `y` 用的是指针型变量名, 相当于间接寻址, 需要先取出地址, 然后根据地址再存取 `x` 和 `y` 的值, 因而改变了调用过程 `main` 的栈帧中局部变量 `a` 和

b 所在位置的内容, 如图 3.17a 中粗体字所示; 而程序二中的 `swap` 函数的形参 x 和 y 用的是基本数据类型变量名, 直接存取 x 和 y 的内容, 因而改变的是 `swap` 函数的入口参数 x 和 y 所在位置的值, 如图 3.17b 中粗体字所示。

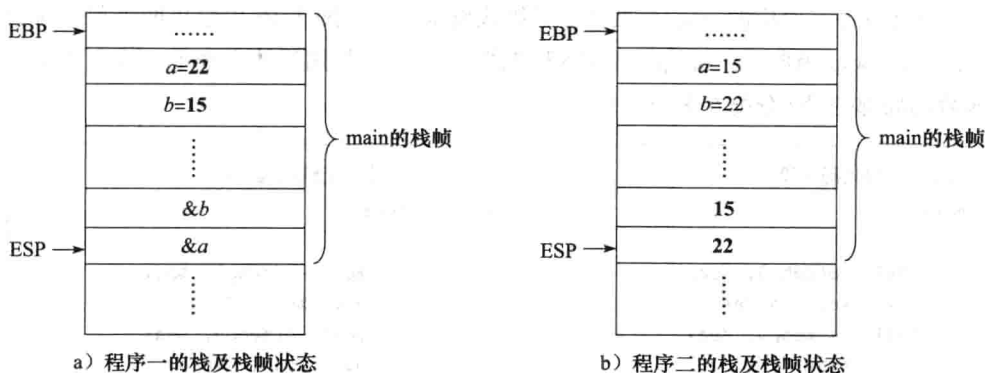


图 3.17 执行 `swap` 之后 `main` 的栈帧状态

至此, 我们分析了程序一和程序二之间明显的差别。由这些差别造成的最终结果的不同是重要的。这个不同就是, 程序一中调用 `swap` 后回到 `main` 执行时, a 和 b 的值已经交换过了, 而在程序二的执行中, `swap` 过程实际上交换的是其两个入口参数所在位置上的内容, 并没有真正交换 a 和 b 的值。由此, 也就不难理解为什么会出现如图 3.14 所示的两个程序的执行结果了。

从上面对例子的分析中可以看出, 编译器并不为形式参数分配存储空间, 而是给形式参数对应的实参分配空间, 形式参数实际上只是被调用函数使用实参时的一个名称而已。不管是按值传递参数还是按地址传递参数, 在调用过程用 `CALL` 指令调用被调用过程时, 对应的实参应该都已有具体的值, 并已将实参的值存放到调用过程的栈帧中作为入口参数, 以等待被调用过程中的指令所用。例如, 在图 3.13 所示的程序一中, `main` 函数调用 `swap` 函数的实参是 `&a` 和 `&b`, 在执行 `CALL` 指令调用 `swap` 之前, `&a` 和 `&b` 的值分别是 `R[ebp] - 4` 和 `R[ebp] - 8`。在如图 3.13 所示的程序二中, `main` 函数调用 `swap` 函数的实参是 a 和 b , 在执行 `CALL` 指令调用 `swap` 之前, a 和 b 的值分别是 15 和 22。

需要说明的是, 在 IA-32 中, 若栈中存放的参数的类型是 `char`、`unsigned char` 或 `short`、`unsigned short`, 也都分配 4 个字节。因而, 在被调用函数的执行过程中, 可以使用 `R[ebp] + 8`、`R[ebp] + 12`、`R[ebp] + 16`、……作为有效地址来访问函数的入口参数。

例 3.9 以下是两个 C 语言函数:

```

1 void test(int x, int *ptr)
2 {
3     if (x > 0 && *ptr > 0)
4         *ptr += x;
5 }
6
7 void caller(int a, int y)
8 {
9     int x = a > 0 ? a : a + 100;

```

```

10     test(x, &y);
11 }

```

假定调用 caller 的过程为 P, P 中给出的对应 caller 中形参 a 和 y 的实参分别是常数 100 和 200, 对于上述两个 C 语言函数, 画出相应的栈帧中的状态, 并回答下列问题。

① test 的形参是按值传递还是按地址传递? test 的形参 ptr 对应的实参是一个什么类型的值?

② test 中被改变的 $*ptr$ 的结果如何返回给它的调用过程 caller?

③ caller 中被改变的 y 的结果能否返回给过程 P? 为什么?

解 P、caller 和 test 对应的栈帧状态如图 3.18 所示。根据图 3.18 中所反映的栈帧状态, 可以比较容易地给出以下答案。

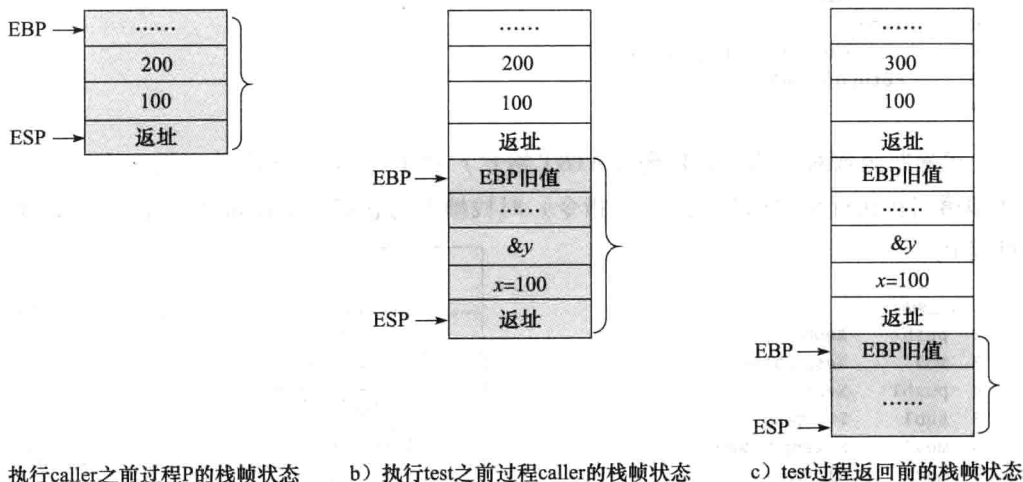


图 3.18 执行 caller 之前和执行 test 之前的栈帧状态

① test 的两个形参中, 前者是基本数据类型变量名, 后者是指针类型变量名, 因此前者按值传递, 后者按地址传递。形参 ptr 是指向 int 型的一个指针, 因而对应的实参一定是一个地址。形参 ptr 对应的实参的值反映了实参所指向的目标数据所在的存储地址。若这个地址是栈区的某个地址, 则说明这个目标数据是个非静态局部变量; 若是静态数据区的某个地址, 则说明这个目标数据是个全局变量或静态变量。此例中, 形参 ptr 对应的实参所指目标数据就是栈中的 y (即 200), 即实参为 y 所在的存储单元地址, 因而在 caller 中用一条取地址指令 lea 可以得到这个地址, 这个地址就是 $\&y$ 。

② test 执行的结果反映在对形参 ptr 对应实参所指向的目标单元进行的修改, 这里是将 200 修改为 300。因为所修改的存储单元不在 test 的栈帧内, 不会因 test 栈帧的释放而丢失, 因而 y 的值可在 test 执行结束后继续在 caller 中使用。也即第 10 行语句执行后, y 的值为 300。

③ caller 执行过程中对 y 所在单元内容的改变不能返回给它的调用过程 P。caller 执行的结果就是调用 test 后由 test 留下的对地址 $\&y$ 处所做的修改, 也即 200 被修改为 300, 虽然这个修改结果不会因为 caller 栈帧的释放而丢失, 似乎在过程 P 中可以访问到这个结果, 但是, 当从

caller 回到过程 P 后, caller 的形参 y 并不能被 P 所用。P 中无法通过 y 对存储单元 $\&y$ 进行引用。因而 y 的值 300 不能在 caller 执行结束后继续传递到 P 中。 ■

9. 递归过程调用

过程调用中使用的栈机制和寄存器使用约定, 使得可以进行过程的嵌套调用和递归调用。下面用一个简单的例子来说明递归调用过程的执行。

以下是一个计算自然数之和的递归函数 (自然数求和可以直接用公式计算, 这里的程序仅是为了说明问题而给出的)。

```

1 int nn_sum(int n)
2 {
3     int result;
4     if (n <= 0)
5         result = 0;
6     else
7         result = n + nn_sum(n-1);
8     return result;
9 }
```

上述递归函数对应的汇编代码 (AT&T 格式) 如下。图 3.19 给出了第 3 次进入递归调用 (即第三次执行完 “call nn_sum” 指令) 时栈帧中的状态, 假定最初调用 nn_sum 函数的是过程 P。

```

1 nn_sum:
2 pushl   %ebp
3 movl    %esp, %ebp
4 pushl   %ebx
5 subl    $4, %esp
6 movl    8(%ebp), %ebx
7 movl    $0, %eax
8 cmpl    $0, %ebx
9 jle     .L2
10 leal    -1(%ebx), %eax
11 movl    %eax, (%esp)
12 call    nn_sum
13 addl    %ebx, %eax
14 .L2:
15 addl    $4, %esp
16 popl    %ebx
17 popl    %ebp
18 ret
```

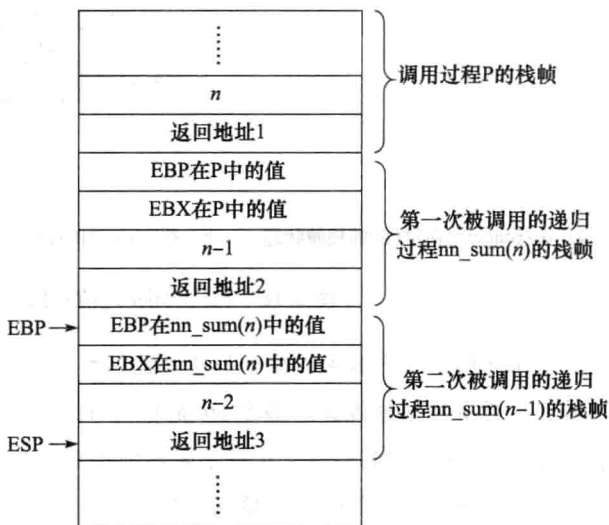


图 3.19 递归过程 nn_sum 的栈

递归过程 nn_sum 对应的汇编代码中, 用到了一个被调用者保存寄存器 EBX, 所以其栈帧中除了保存常规的 EBP 外, 还要保存 EBX。过程的入口参数只有一个, 因此, 序号 5 对应的指令 “subl \$4, %esp” 实际上是为参数 $n-1$ (或 $n-2$ 或...或 1 或 0) 在栈帧中申请了 4 字节的空间, 递归过程直到参数为 0 时才第一次退出 nn_sum 过程, 并回到序号为 12 的指令 call nn_sum 的后面一条指令 (序号为 13 的指令) 执行。在递归调用过程中, 应该每次都回到同样的地方执行, 因此, 图 3.19 中的返回地址 2 和返回地址 3 是相同的, 但不同于返回地址 1, 因为返回地址 1 是过程 P 中指令 call nn_sum 的后面一条指令的地址。

图 3.20 给出了上述递归过程的执行流程。

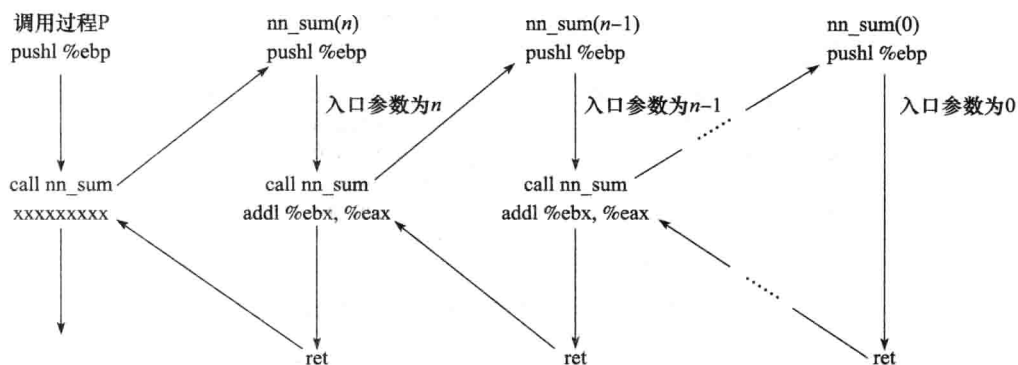


图 3.20 递归过程 nn_sum 的执行流程

从图 3.20 可看出，递归调用过程的执行一直要等到满足跳出过程的条件时才结束，这里跳出过程的条件是入口参数为 0，只要入口参数不为 0，就一直递归调用 nn_sum 函数自身。因此，在递归调用 nn_sum 的过程中，栈中最多会形成 $n+1$ 个 nn_sum 栈帧。每个 nn_sum 栈帧占用了 16 字节的空间，因而 nn_sum 过程在执行中最多占用 $(16n+12)$ 字节的栈空间（以入口参数为 0 调用 nn_sum 时，没有返回地址入栈，故只分配 12 字节）。虽然占用的栈空间都是临时的，过程执行结束后其所占的所有栈空间都会被释放，但是，如果递归深度非常大的时候，栈空间的开销还是比较大的。此外，过程调用的时间开销也不得不考虑。虽然过程的功能由过程体中的指令来实现，但是，为了支持过程调用，每个过程中还包含了准备阶段和结束阶段。每增加一次过程调用，就要增加许多条包含在准备阶段和结束阶段的额外指令，这些额外指令的执行时间开销对程序的性能影响很大，因而，应该尽量避免不必要的过程调用，特别是递归调用。

3.4.2 选择语句的机器级表示

C 语言主要通过选择结构（条件分支）和循环结构语句来控制程序中语句的执行顺序，有 9 种流程控制语句，分成三类：选择语句、循环语句和辅助控制语句，如图 3.21 所示。

1. if ~ else 语句的机器级表示

if ~ (then)、if ~ (then) ~ else 选择结构根据判定条件来控制一些语句是否被执行。其通用形式如下。

```

if (cond_expr)
    then_statement
else
    else_statement
  
```

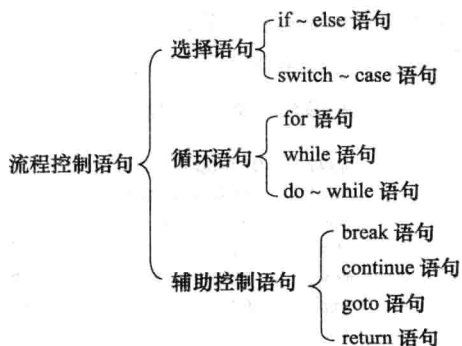


图 3.21 C 语言中的流程控制语句

其中，cond_expr 是条件表达式，根据其值为非 0（真）或 0（假），分别选择 then_statement 或 else_statement 执行。通常，编译后得到的对应汇编代码可以有如下两种不同的结构，如图 3.22 所示。

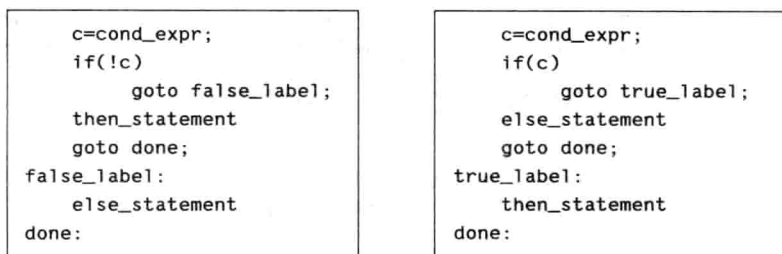


图 3.22 if ~ else 语句对应的汇编代码结构

图 3.22 中的“if() goto...”语句对应条件转移指令，“goto...”语句对应无条件转移指令。编译器可以使用在底层 ISA 中提供的各种条件标志设置功能、条件转移指令、条件设置指令、条件传送指令、无条件转移指令等相应的机器级程序支持机制（参见 3.3.4 节有关内容）来实现这类选择语句。

例 3.10 以下是一个 C 语言函数：

```

1 int get_lowaddr_content(int *p1, int *p2)
2 {
3     if (p1 > p2)
4         return *p2;
5     else
6         return *p1;
7 }

```

已知形式参数 $p1$ 和 $p2$ 对应的实参已被压入调用函数的栈帧， $p1$ 和 $p2$ 对应实参的存储地址分别为 $R[ebp] + 8$ 、 $R[ebp] + 12$ ，这里，EBP 指向当前栈帧底部。返回结果存放在 EAX 中。请写出上述函数体对应的汇编代码，要求用 GCC 默认的 AT&T 格式书写。

解 因为 $p1$ 和 $p2$ 是指针类型参数，所以指令助记符中的长度后缀是 l，比较指令 `cmpl` 的两个操作数应该都来自寄存器，故应先将 $p1$ 和 $p2$ 对应的实参从栈中取到通用寄存器中，比较指令执行后得到各个条件标志位，程序需要根据条件标志的组合条件值选择执行不同的指令，因此需要用到条件转移指令，转移目标地址用标号 `.L1` 和 `.L2` 等标识。

以下汇编代码能够正确完成上述函数的功能（不包括过程调用的准备阶段和结束阶段）。

```

1  movl    8(%ebp), %eax    // R[ eax ] ← M[ R[ ebp ] + 8 ], 即 R[ eax ] = p1
2  movl    12(%ebp), %edx   // R[ edx ] ← M[ R[ ebp ] + 12 ], 即 R[ edx ] = p2
3  cmpl    %edx, %eax      // 比较 p1 和 p2, 即根据 p1 - p2 的结果置标志
4  jbe     .L1             // 若 p1 ≤ p2, 则转 L1 处执行
5  movl    (%edx), %eax     // R[ eax ] ← M[ R[ edx ] ], 即 R[ eax ] = M[ p2 ]
6  jmp     .L2             // 无条件跳转到 L2 执行
7  .L1:
8  movl    (%eax), %eax     // R[ eax ] ← M[ R[ eax ] ], 即 R[ eax ] = M[ p1 ]
9  .L2

```

上述汇编代码中，// 后面的文字给出的是对指令的功能说明，其中的 $p1$ 和 $p2$ 实际上是函数的形式参数 $p1$ 和 $p2$ 对应的实参。本例中函数的形式参数 $p1$ 和 $p2$ 都是指针型变量，因此是按地址调用的情况。序号为 3 的 `cmpl` 指令实际上是两个地址大小的比较，随后序号 4 对应的指令应该使用无符号整数比较转移指令。参照表 3.6 中的条件转移指令可知，其对应的条件转

移指令是 jbe。

2. 条件运算表达式的机器级表示

C 语言中唯一的三目运算符是由符号“?”和“:”组成的,它可以构成一个条件运算表达式,这个条件运算表达式的值可以赋值给一个变量。其通用形式如下:

```
x = cond_expr ? then_expr : else_expr;
```

对应的机器级代码可以编译成如图 3.22 所示的两种结构,只要将图 3.22 中的 then_statement 改成“x = then_expr;”,将 else_statement 改成“x = else_expr;”即可。如果机器级代码用这两种结构来实现的话,与 if ~ else 语句一样,要用到比较指令、条件转移指令以及无条件转移指令。

对于条件转移指令,它在条件满足时会跳转到其他地方执行,因而会破坏程序既定的执行流程,这在用流水线方式执行程序的情况下,就会破坏流水线的执行,导致流水线停顿,从而影响程序执行的性能。那么,用条件传送指令来代替条件转移指令是否更好呢?

在 3.3.4 节中介绍过条件传送指令 CMOVcc,该指令的功能是,在满足指定条件时,将源数据送到目的地,否则什么也不做。也就是说,该指令执行完后,CPU 还是继续执行它后续的指令,不会改变程序既定的执行流程,因而不会破坏指令流水线的执行,这样看来好像使用条件传送指令比使用条件转移指令更好。

实际上,条件传送指令并不比条件转移指令更好,不建议使用条件传送指令来代替条件转移指令。这是因为 CMOVcc 指令会使指令之间的依赖性加大,因此在乱序执行指令时使用 CMOVcc 指令反而会降低程序执行效率。此外,在基于条件传送指令实现的机器级代码中,由于两个分支表达式的值都需要计算,特别是当运算表达式比较复杂时,计算量会增加较多,因而也不宜用条件传送指令。

例 3.11 以下是两个 C 语言函数:

```
1 void test(int x, int *ptr)
2 {
3     if (x > 0 && *ptr > 0)
4         *ptr += x;
5 }
6
7 void caller(int a, int y)
8 {
9     int x = a > 0 ? a : a + 100;
10    test(x, &y);
11 }
```

对于上述两个 C 语言函数,完成下列任务(汇编代码用 AT&T 格式)。

- ① 写出函数 test 的过程体对应的汇编代码。
- ② 基于条件传送指令写出行号为 9 的语句对应的汇编代码(假定 x 被分配在寄存器 EAX 中)。

解 ① 以下给出能正确完成 test 函数体功能的汇编代码(不包括过程调用的准备阶段和结束阶段)。

```
1  movl 8(%ebp), %eax    // R[ eax] ← M[R[ ebp] + 8], 即 R[ eax] = x
2  movl 12(%ebp), %edx   // R[ edx] ← M[R[ ebp] + 12], 即 R[ edx] = ptr
3  testl %eax, %eax      // 根据 x 与 x 相“与”的结果置标志
```

```

4   jle    .L1          // 若 x <= 0, 则转 L1 处执行
5   movl   (%edx), %ecx  // R[ecx] ← *ptr
6   testl  %ecx, %ecx    // 根据 *ptr 与 *ptr 相“与”的结果置标志
7   jle    .L1          // 若 *ptr <= 0, 则转 L1 处执行
8   addl   %eax, (%edx)  // 实现 *ptr += x 的功能
9   .L1:

```

这里两条条件转移指令，分别用来判断条件表达式“(x > 0 && *ptr > 0)”分解出来的两个结果为“假”的条件“x <= 0”和“*ptr <= 0”，在这两个条件下，都不会执行“*ptr += x”的功能。

② 行号为 9 的语句“x = a > 0 ? a : a + 100;”对应的汇编代码如下，其中第 5 条为条件传送指令。

```

1   movl   8(%ebp), %edx // R[edx] ← M[R[ebp] + 8], 即 R[edx] = a
2   movl   %edx, %eax    // R[edx] ← R[edx], 即 R[edx] = a
3   addl   $100, %eax    // R[edx] ← a + 100
4   testl  %edx, %edx    // 根据 a 与 a 相“与”的结果置标志
5   cmovg  %edx, %eax    // 若 a > 0, 则 R[edx] ← R[edx], 即 R[edx] = a

```

3. switch 语句的机器级表示

解决多分支选择问题可以用连续的 if ~ else ~ if 语句，不过，这种情况下，只能按顺序一一测试条件，直到满足条件时才执行对应分支的语句。因而，通常用 switch 语句来实现多分支选择问题，它可以直接跳到某个条件处的语句执行，而不用一一测试条件。那么，switch 语句对应的机器级代码是如何实现直接跳转的呢？下面用一个简单的例子来说明 switch 语句的机器级表示。图 3.23a 是一个含有 switch 语句的过程，图 3.23b 是对应的过程体的汇编表示和跳转表。

<pre> 1 int switch_test(int a, int b, int c) 2 { 3 int result; 4 switch(a) { 5 case 15: 6 c=b&0xf; 7 case 10: 8 result=c+50; 9 break; 10 case 12: 11 case 17: 12 result=b+50; 13 break; 14 case 14: 15 result=b 16 break; 17 default: 18 result=a; 19 } 20 return result; 21 } </pre>	<pre> 1 movl 8(%ebp), %eax 1 .section .rodata 2 subl \$10, %eax 2 .align 4 3 cmpl \$7, %eax 3 .L8 4 ja .L5 4 .long .L2 5 jmp *.L8(, %eax, 4) 5 .long .L5 6 .L1: 6 .long .L3 7 movl 12(%ebp), %eax 7 .long .L5 8 andl \$15, %eax 8 .long .L4 9 movl %eax, 16(%ebp) 9 .long .L1 10 .L2: 10 .long .L5 11 movl 16(%ebp), %eax 11 .long .L3 12 addl \$50, %eax 13 jmp .L7 14 .L3: 15 movl 12(%ebp), %eax 16 addl \$50, %eax 17 jmp .L7 18 .L4: 19 movl 12(%ebp), %eax 20 jmp .L7 21 .L5: 22 addl \$10, %eax 23 .L7: 24 </pre>
--	--

a) switch 语句所在的函数

b) switch 语句对应的汇编代码表示

图 3.23 switch 语句与对应的汇编表示

由图 3.23a 可知, 过程 `switch_test` 的 `switch` 语句中共有 6 个 `case` 分支, 在机器级代码中分别用标号 `.L1`、`.L2`、`.L3`、`.L3`、`.L4`、`.L5` 来标识这 6 个分支, 它们分别对应条件 $a = 15$ 、 $a = 10$ 、 $a = 12$ 、 $a = 17$ 、 $a = 14$ 和其他情况, 其中, $a = 15$ 时所执行的语句 (与 `.L1` 分支对应) 包含了 $a = 10$ 时的语句 (与 `.L2` 分支对应); $a = 12$ 和 $a = 17$ 时所执行的语句一样, 都是对应 `.L3` 分支; 默认 (default) 时包含了 $a = 11$ 或 $a = 13$ 或 $a = 16$ 或 $a > 17$ 的几种情况, 与 `.L5` 分支对应。因而, 可以用一个跳转表来实现 a 的取值与跳转标号之间的对应关系。将 a 的值减去 10 以后, 其值从 0 开始, 可以将 $a - 10$ 得到的值作为跳转表的索引, 每个跳转表中存放的是一个段内直接转移的 4 字节偏移地址, 因而跳转表中每个表项的偏移量分别为 0、4、8、12、16、20、24 和 28, 即偏移量等于“索引值 $\times 4$ ”。这个偏移量与跳转表的首地址 (由标号 `.L8` 指定) 相加就是转移目标地址。因此, 可以用图 3.23b 中第 5 行指令 “`jmp *.L8(, %eax, 4)`” 来实现直接跳转, 这里寄存器 `EAX` 中存放的就是索引值。从上述示例可以看出, 对 `switch` 语句进行编译转换的关键是构造跳转表, 并正确设置索引值。图 3.23b 中右边的跳转表属于只读数据, 即数据段属性为 “`.rodata`”, 并且在跳转表中的每个跳转地址都必须在 4 字节边界上, 即 “`align 4`” 方式。

当然, 当 `case` 的条件值相差较大时, 例如, `case 10`、`case 100`、`case 1000` 等, 编译器还是会生成分段跳转代码, 而不会采用构造跳转表的方式来进行跳转。

3.4.3 循环结构的机器级表示

图 3.21 中总结了 C 语言中的所有程序控制语句, 其中循环结构有三种: `for` 语句、`while` 语句和 `do ~ while` 语句。大多数编译程序将这三种循环结构都转换为 `do ~ while` 形式来产生机器级代码, 下面按照与 `do ~ while` 结构相似程度由近到远的顺序来介绍三种循环语句的机器级表示。

1. `do ~ while` 循环的机器级表示

C 语言中的 `do ~ while` 语句形式如下。

```
do
{
    loop_body_statement
} while (cond_expr);
```

该循环结构的执行过程可以用以下更接近于机器级语言的低级行为描述结构来描述。

```
loop:
    loop_body_statement
    c = cond_expr;
    if (c) goto loop;
```

上述结构对应的机器级代码中, `loop_body_statement` 用一个指令序列来完成, 然后用一个指令序列实现对 `cond_expr` 的计算, 并将计算或比较的结果记录在标志寄存器中, 然后用一条条件转移指令来实现 “`if (c) goto loop;`” 的功能。

2. `while` 循环的机器级表示

C 语言中的 `while` 语句形式如下。

```
while (cond_expr)
    loop_body_statement
```

该循环结构的执行过程可以用以下更接近于机器级语言的低级行为描述结构来描述。

```
c = cond_expr;
if (!c) goto done;
loop:
    loop_body_statement
    c = cond_expr;
    if (c) goto loop;
done:
```

从上述结构可看出，与 do ~ while 循环结构相比，while 循环仅在开头多了一段计算条件表达式的值并根据条件选择是否跳出循环体执行的指令序列，其余地方与 do ~ while 语句一样。

3. for 循环的机器级表示

C 语言中的 for 语句形式如下。

```
for (begin_expr; cond_expr; update_expr)
    loop_body_statement
```

for 循环结构的执行过程大多可以用以下更接近于机器级语言的低级行为描述结构来描述。

```
begin_expr;
c = cond_expr;
if (!c) goto done;
loop:
    loop_body_statement
    update_expr;
    c = cond_expr;
    if (c) goto loop;
done:
```

从上述结构可看出，与 while 循环结构相比，for 循环仅在两个地方多了一段指令序列。一个是开头多了一段循环变量赋初值的指令序列，另一个是循环体中多了更新循环变量值的指令序列，其余地方与 while 语句一样。

在 3.4.1 节中，我们以计算自然数之和的递归函数为例说明了递归过程调用的原理，这个递归函数仅是为了说明原理而给出的，实际上可以直接用公式计算。同样，这里为了说明循环结构的机器级表示，我们用 for 语句来实现这个功能。

```
1 int nn_sum(int n)
2 {
3     int i;
4     int result=0;
5     for (i=1; i <=n; i++)
6         result += i;
7     return result;
8 }
```

根据上述对应 for 循环的低级行为描述结构，不难写出上述过程对应的汇编表示，以下是其过程体的 AT&T 格式汇编代码。


```

1  movl 8(%ebp), %ecx
2  movl $0, %eax
3  movl $1, %edx
4  cmp  %ecx, %edx
5  jg   .L2
6  .L1:
7  addl %edx, %eax
8  addl $1, %edx
9  cmpl %ecx, %edx
10 jle  .L1
11 .L2

```

从上述汇编代码可以看出，过程 `nn_sum` 中的非静态局部变量 `i` 和 `result` 被分别分配在寄存器 `EDX` 和 `EAX` 中，`ECX` 中始终存放入口参数 `n`，返回参数在 `EAX` 中。这个过程体中没有用到被调用过程保存寄存器。因而，可以推测在该过程的栈帧中仅保留了 `EBP` 的原值，即其栈帧仅占用了 4 字节的空间，而 3.4.1 节给出的递归方式则占用了 $(16n + 12)$ 字节的栈空间，多用了 $(16n + 8)$ 字节的栈空间。特别是每次过程调用都要执行 16 条指令，递归情况下一共多了 n 次过程调用，因而，递归方式比非递归方式至少多执行了 $16n$ 条指令。由此可以看出，为了提高程序的性能，可能的话最好用非递归方式。

例 3.12 一个 C 语言函数被 GCC 编译后得到的过程体对应的汇编代码如下。

```

1  movl 8(%ebp), %ebx
2  movl $0, %eax
3  movl $0, %ecx
4  .L12:
5  leal (%eax,%eax), %edx
6  movl %ebx, %eax
7  andl $1, %eax
8  orl  %edx, %eax
9  shrl %ebx
10 addl $1, %ecx
11 cmpl $32, %ecx
12 jne  .L12

```

该 C 语言函数的整体框架结构如下。

```

int function_test(unsigned x)
{
    int result=0;
    int i;
    for ( _____ ① _____ ; _____ ② _____ ; _____ ③ _____ ) {
        _____ ④ _____
    }
    return result;
}

```

根据对应的汇编代码填写函数中缺失的①、②、③和④部分。

解 从对应汇编代码来看，因为 `ECX` 初始为 0，在比较指令 `cmpl` 之前 `ECX` 做了一次加 1 操作后，再与 32 比较，最后根据比较结果选择是否转到 `.L12` 继续执行，所以，可以很明显地看出循环变量 `i` 被分配在 `ECX` 中，①处为“`i = 0`”，②处为“`i != 32`”，③处为“`i++`”。

第 5~9 行汇编指令对应④处的语句，入口参数 `x` 在 `EBX` 中，返回参数 `result` 在 `EAX` 中。

第5条指令 `leal` 实现 “ $2 * result$ ”，相当于将 `result` 左移一位；第6和第7条指令则实现 “`x&0x01`”；第8条指令实现 “`result = (result << 1) | (x & 0x01)`”；第9条指令实现 “`x >>= 1`”。综上所述，④处的两条语句是 “`result = (result << 1) | (x & 0x01); x >>= 1;`”。■

因为本例中循环终止条件是 “`i!=32`”，而循环变量 `i` 的初值为0，可以确定第一次终止条件肯定不满足，所以可以省掉循环体前面一次条件判断。从本例中给出的汇编代码来看，它确实只有一个无符号整数条件转移指令，而不像最初给出的 `for` 循环对应的低级行为描述结构那样有两处条件转移指令。显然，本例中给出的结构更简洁。

3.5 复杂数据类型的分配和访问

本节以 C 语言为例说明复杂类型数据在机器级的处理，包括在寄存器和存储器中的存储与访问。在机器级代码中，基本类型对应的数据通常通过单条指令就可以访问和处理，这些数据在指令中或者是以立即数的方式出现，或者是以寄存器数据的形式出现，或者是以存储器数据的形式出现；而对于构造类型的数据，由于其包含多个基本类型数据，因而不能直接用单条指令来访问和运算，通常需要特定的代码结构和寻址方式对其进行处理。本节主要介绍构造类型和指针类型的数据在机器级程序中的访问和处理。

3.5.1 数组的分配和访问

数组可以将同类基本类型数据组合起来形成一个大的数据集合。数组是一个数据集合，因而不可能放在一个寄存器中或作为立即数存放在指令中，它一定被分配在存储器中，数组中的每个元素在存储器中连续存放，可以用一个索引值来访问数据元素。对于数组的访问和处理，编译器最重要的是要找到一种简便的数组元素地址的计算方法。

1. 数组元素在存储空间的存放和访问

在程序中使用数组，必须遵循定义在前，使用在后的原则。一维数组定义的一般形式如下。

存储类型 数据类型 数组名[元素个数];

其中，存储类型可以缺省。例如，定义一个具有4个元素的静态存储型 `short` 数据类型数组 `A`，可以写成 “`static short A[4];`”。这4个数组元素为 `A[0]`、`A[1]`、`A[2]` 和 `A[3]`，它们连续存放在静态数据存储区中，每个数组元素都为 `short` 型数据，故占用2个字节，数组 `A` 共占用8个字节，数组首地址就是第一个元素 `A[0]` 的地址，因而通常用 `&A[0]` 表示，也可简单以 `A` 表示数组 `A` 的首地址，第 i ($0 \leq i \leq 3$) 个元素的地址计算公式为 `&A[0] + 2 * i`。

假定数组 `A` 的首地址存放在 `EDX` 中，`i` 存放在 `ECX` 中，现需要将 `A[i]` 取到 `AX` 中，则可用以下汇编指令来实现。

```
movw (%edx, %ecx, 2), %ax
```

表3.7给出了若干数组的定义以及它们在内存中的存放情况的说明。

表 3.7 数组定义及其内存存放情况示例

数组定义	数组元素类型	元素大小 (B)	数组大小 (B)	起始地址	元素 i 的地址
<code>char S[10]</code>	<code>char</code>	1	10	<code>&S[0]</code>	<code>&S[0] + i</code>
<code>char * SA[10]</code>	<code>char *</code>	4	40	<code>&SA[0]</code>	<code>&SA[0] + 4 * i</code>
<code>double D[10]</code>	<code>double</code>	8	80	<code>&D[0]</code>	<code>&D[0] + 8 * i</code>
<code>double * DA[10]</code>	<code>double *</code>	4	40	<code>&DA[0]</code>	<code>&DA[0] + 4 * i</code>

表 3.7 给出的 4 个数组定义中, 数组 `SA` 和 `DA` 中每个元素都是一个指针, `SA` 中每个元素指向一个 `char` 型数据, `DA` 中每个元素指向一个 `double` 型数据。

2. 数组的存储分配和初始化

数组可以定义为静态存储型 (`static`)、外部存储型 (`extern`)、自动存储型 (`auto`), 或者定义为全局静态区数组, 其中, 只有 `auto` 型数组被分配在栈中, 其他存储型数组都分配在静态数据区。

数组的初始化就是在定义数组时给数组元素赋初值。例如, 以下声明可以对数组 `A` 的 4 个元素进行初始化。

```
static short A[4] = {3, 80, 90, 65};
```

因为在编译、链接时就可以确定在静态区中的数组的地址, 所以在编译、链接阶段就可将数组首地址和数组变量建立关联。对于分配在静态区的已初始化的数组, 机器级指令中可通过数组首地址和数组元素的下标来访问相应的数组元素。例如, 对于下面给出的例子:

```
int buf[2] = {10, 20};
int main()
{
    int i, sum = 0;
    for (i = 0; i < 2; i++)
        sum += buf[i];
    return sum;
}
```

该例中, `buf` 是一个在静态区分配的可被其他程序模块使用的全局静态区数组, 编译、链接后 `buf` 在可执行目标文件的数据段中分配了相应的空间。假定分配给 `buf` 的地址为 `0x8048908`, 则在该地址开始的 8 个字节空间中存放数据的情况如下:

```
1 08048908 <buf>;
2 08048908: 0A 00 00 00 14 00 00 00
```

编译器在处理语句 “`sum += buf[i];`” 时, 假定 `i` 分配在 `ECX` 中, `sum` 分配在 `EAX` 中, 则该语句可转换为指令 “`addl buf(, %ecx, 4), %eax`”, 其中 `buf` 的值为 `0x8048908`。

对于 `auto` 型数组, 因为被分配在栈中, 因此数组首地址通过 `ESP` 或 `EBP` 来定位, 机器级代码中数组元素地址由首地址与数组元素的下标值进行计算得到。例如, 对于下面给出的例子:

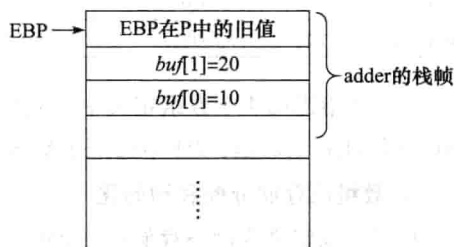
```
int adder()
{
    int buf[2] = {10, 20};
```

```

int i, sum=0;
for (i=0; i<2; i++)
    sum += buf[i];
return sum;
}

```

该例中, *buf* 是一个在栈区分配的局部数组, 在栈中分配了相应的 8 字节空间。假定调用 *adder* 的函数为 *P*, 并且在 *adder* 中没有使用被调用者保存寄存器 *EBX*、*ESI*、*EDI*, 局部变量 *i* 和 *sum* 分别分配在寄存器 *ECX* 和 *EAX* 中, 则函数 *adder* 对应的栈帧中的情况如图 3.24 所示。

图 3.24 *adder* 的栈帧

在处理 *auto* 型数组赋初值的语句 “*int buf[2] = {10,20};*” 时, 编译器可以生成以下指令序列:

```

1 movl $10, -8(%ebp)    // buf[0]的地址为 R[ebp]-8, 将 10 赋给 buf[0]
2 movl $20, -4(%ebp)    // buf[1]的地址为 R[ebp]-4, 将 20 赋给 buf[1]
3 leal -8(%ebp), %edx    // buf[0]的地址为 R[ebp]-8, 将 buf 的首地址送 EDX

```

执行完上述指令序列后, 数组 *buf* 的首地址在 *EDX* 中, 在处理语句 “*sum += buf[i];*” 时, 编译器可以将该语句转换为机器级指令 “*addl (%edx, %ecx, 4), %eax*”。

3. 数组与指针

C 语言中指针与数组之间的关系十分密切, 它们均用于处理存储器中连续存放的一组数据, 因而在访问存储器时两者的地址计算方法是统一的, 数组元素的引用可以用指针来实现。

在指针变量的目标数据类型与数组的数据类型相同的前提下, 指针变量可以指向数组或者数组中的任意元素。例如, 对于存储器中连续的 10 个 *int* 型数据, 可以用数组 *a* 来说明, 也可以用指针变量 *ptr* 来说明。以下两个程序段的功能完全相同, 都是使指针 *ptr* 指向数组 *a* 的第 0 个元素 *a[0]*。同时数组变量 *a* 的值就是其首地址, 即 *a = &a[0]*, 因而 *a = ptr*, 从而有 *&a[i] = ptr + i = a + i* 以及 *a[i] = ptr[i] = *(ptr + i) = *(a + i)*。

```

// 代码段 1
int a[10];
int *ptr=&a[0];
// 代码段 2
int a[10], *ptr;
ptr=&a[0];

```

0x08048A24

0x08048A04

0x08048A00

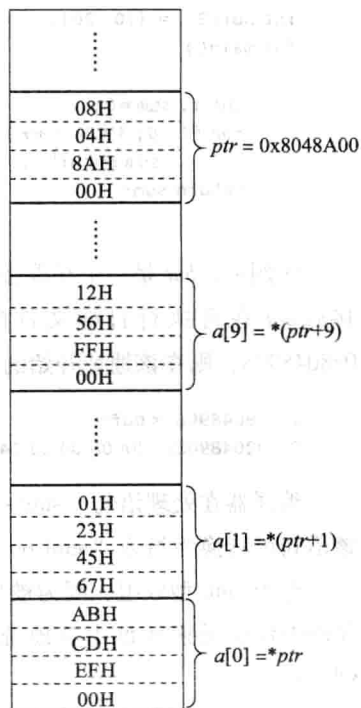


图 3.25 用指针和数组表示连续存放的一组数据

假定 0x08048A00 处开始的存储区有 10 个 *int* 型数据, 部分内容如图 3.25 所示, 以小端方式存放。图 3.25 给出了用数组和指针表示的内存中连

续存放的数据, 以及指针和数组元素之间的关系。图中 $a[0] = 0xABCDEF00$ 、 $a[1] = 0x1234567$ 、 $a[9] = 0x1256FF00$ 。数组首地址 $0x8048A00$ 存放在指针变量 ptr 中, 从图中可以看出, $ptr + i$ 的值并不是用 $0x8048A00$ 加 i 得到, 而是等于 $0x8048A00 + 4 * i$ 。

表 3.8 给出了一些数组元素或指针变量的表达式及其计算方式。假定 $\text{sizeof}(\text{int}) = 4$, 表中数组 A 为 int 型, 其首地址 SA 在 ECX 中, 数组的下标变量 i 在 EDX 中, 表达式的结果在 EAX 中。

表 3.8 关于数组元素和指针变量的表达式计算示例

序号	表达式	类型	值的计算方式	汇编代码
1	A	$\text{int} *$	SA	<code>leal (%ecx), %eax</code>
2	$A[0]$	int	$M[SA]$	<code>movl (%ecx), %eax</code>
3	$A[i]$	int	$M[SA + 4 * i]$	<code>movl (%ecx, %edx, 4), %eax</code>
4	$\&A[3]$	$\text{int} *$	$SA + 12$	<code>leal 12(%ecx), %eax</code>
5	$\&A[i] - A$	int	$(SA + 4 * i - SA) / 4 = i$	<code>movl %edx, %eax</code>
6	$*(A + i)$	int	$M[SA + 4 * i]$	<code>movl (%ecx, %edx, 4), %eax</code>
7	$*(\&A[0] + i - 1)$	int	$M[SA + 4 * i - 4]$	<code>movl -4(%ecx, %edx, 4), %eax</code>
8	$A + i$	$\text{int} *$	$SA + 4 * i$	<code>leal (%ecx, %edx, 4), %eax</code>

表 3.8 中序号为 2、3、6 和 7 的表达式都是引用数组元素, 其中 3 和 6 是等价的。对应的汇编指令都需要有访存操作, 指令中源操作数的寻址方式分别是“基址”、“基址加比例变址”、“基址加比例变址”和“基址加比例变址加位移”的方式, 因为数组元素的类型为 int 型, 故比例因子为 4。

序号为 1、4 和 8 的表达式都是有关数组元素地址的计算, 都可以用取有效地址指令 `leal` 来实现, 对于序号为 1 的表达式, 也可以用指令“`movl %ecx, %eax`”实现。

序号为 5 的表达式则是计算两个数组元素之间相差的元素个数, 也即两个指针之间的运算, 因此, 表达式的值的类型应该是 int , 运算时应该是两个数组元素地址之差再除以 4。

4. 指针数组和多维数组

由若干个指向同类目标的指针变量组成的数组称为指针数组。在 C 程序中使用指针数组必须事先定义, 其定义的一般形式如下。

存储类型 数据类型 *指针数组名[元素个数];

指针数组中每个元素都是指针, 每个元素指向的目标数据类型都相同, 就是上述定义中的数据类型, 存储类型通常是缺省的。例如, “`int *a[10];`”定义了一个指针数组 a , 它有 10 个元素, 每个元素都是一个指向 int 型数据的指针。

一个指针数组可以实现一个二维数组。以下用一个简单的例子来说明指针数组和二维数组之间的关联, 并说明如何在机器级程序中访问指针数组元素所指的目标数据和二维数组元素。

以下是一个 C 语言程序, 用来计算一个两行四列整数矩阵中每一行数据的和。

```

1 #include <stdio.h>
2 main()
3 {
4     static short num[ ][4] = { {2, 9, -1, 5}, {3, 8, 2, -6}};
```

```

5     static short *pn[] = {num[0], num[1]};
6     static short s[2] = {0, 0};
7     int i, j;
8     for (i=0; i<2; i++) {
9         for (j=0; j<4; j++)
10            s[i] += *pn[i] ++;
11        printf("sum of line %d:%d\n", i+1, s[i]);
12    }
13 }

```

该例中, *num* 是一个在静态区分配的静态数组, 因而在可执行目标文件的数据段中分配了相应的空间。假定分配给 *num* 的地址为 0x8049300, 则在该地址开始的一段存储区中存放数据的情况如下。

```

1 08049300 <num> :
2 08049300: 02 00 09 00 ff ff 05 00 03 00 08 00 02 00 fa ff
3 08049310 <pn> :
4 08049310: 00 93 04 08 08 93 04 08

```

因此, $num = num[0] = \&num[0][0] = 0x8049300$, $pn = \&pn[0] = 0x8049310$, $pn[0] = num[0] = 0x8049300$, $pn[1] = num[1] = 0x8049308$ 。编译器在处理第 10 行语句 “ $s[i] += *pn[i] ++;$ ” 时, 若 *i* 在 ECX, *s[i]* 在 AX, 则可通过指令 “ $movl\ pn(, \%ecx, 4), \%edx$ ” 先将 $pn[i]$ 送到 EDX 中, 再通过以下两条指令实现其功能。

```

addw(edx), %ax
addl $2, pn(, \%ecx, 4)

```

执行上述第一条加法指令 *addw* 时, $pn[i]$ 已在 EDX 中, 因为是 short 型数据, 所以数据宽度为 16 位, 即指令助记符长度后缀为 w; 因为 *pn* 为指针数组, 所以在引用 *pn* 的元素时其比例因子为 4。例如, 当 $i=1$ 时, $pn[i] = *(pn+i) = M[pn+4*i] = 0x8049308$ 。

第二条加法指令 *addl* 用来实现 “ $pn[i] + 1 \rightarrow pn[i]$ ” 的功能, 因为 $pn[i]$ 是指针, 故 “ $pn[i] + 1 \rightarrow pn[i]$ ” 是指针运算, 因此, 操作数长度为 4B, 助记符长度后缀为 l, 实际应加指针变量的目标数据长度, 即 short 型数据的宽度, 也就是指令中的立即数 2。

3.5.2 结构体数据的分配和访问

C 语言的结构体 (也称结构) 可以将不同类型的数据结合在一个数据结构中。组成结构体的每个数据称为结构体的成员或字段。

1. 结构体成员在存储空间的存放和访问

结构体中的数据成员存放在存储器中一段连续的存储区中, 指向结构的指针就是其第一个字节的地址。编译器在处理结构型数据时, 根据每个成员的数据类型获得相应的字节偏移量, 然后通过每个成员的字节偏移量来访问结构成员。

例如, 以下是一个关于个人联系信息的结构体:

```

struct cont_info {
    char id[8];
    char name[12];
    unsigned post;
}

```

```

char address[100];
char phone[20];
}

```

该结构体定义了关于个人联系信息的一个数据类型 `struct cont_info`，可以把一个变量 x 定义成这个类型，并赋初值，例如，在定义了上述数据类型 `struct cont_info` 后，可以对变量 x 进行如下声明。

```

struct cont_info x = {"0000000", "ZhangS", 210022,
    "273 long street, High Building #3015", "12345678"};

```

与数组一样，分配在栈中的 `auto` 型结构类型变量的首地址由 `EBP` 或 `ESP` 来定位，分配在静态存储区的静态和外部型结构体变量首地址是一个确定的静态存储区地址。

结构体变量 x 的每个成员的首地址等于 x 加上一个偏移量。假定上述变量 x 分配在地址 `0x8049200` 开始的区域，那么， $x = \&(x.id) = 0x8049200$ ，其他成员的地址计算如下。

```

&(x.name) = 0x8049200 + 8 = 0x8049208
&(x.post) = 0x8049200 + 8 + 12 = 0x8049214
&(x.address) = 0x8049200 + 8 + 12 + 4 = 0x8049218
&(x.phone) = 0x8049200 + 8 + 12 + 4 + 100 = 0x804927C

```

可以看出 x 初始化后，对于 `name` 字段，在地址 `0x8049208 ~ 0x804920D` 处存放的是字符串“ZhangS”，`0x804920E` 处存放的是字符‘\0’，在地址 `0x804920F ~ 0x8049213` 处存放的都是空格字符。

访问结构体变量的成员时，对应的机器级代码可以通过“基址加偏移量”的寻址方式来实现。例如，假定编译器在执行语句“`unsigned xpost = x.post;`”时， x 被分配在 `EDX` 中， $xpost$ 被分配在 `EAX` 中，则对应的汇编指令为“`movl 20(%edx), %eax`”。这里的基址就是 `0x8049200`，它被存放在 `EDX` 中，偏移量为 $8 + 12 = 20$ 。

2. 结构体数据作为入口参数

当结构体变量需要作为一个函数的形式参数时，形式参数和调用函数中的实参应该具有相同的结构。和普通变量传递参数的方式一样，它也有按值传递和按地址传递两种方式。如果采用按值传递方式，则结构的每个成员都要被复制到栈中参数区，这既增加时间开销，又增加空间开销，因而对于结构体变量通常采用按地址传递的方式。也就是说，对于结构类型参数，通常不会直接作为参数，而是把指向结构的指针作为参数。这样，在执行 `call` 指令之前，就无需把结构成员复制到栈中的参数区，而只要把相应的结构体首地址送到参数区，也就是说，仅传递指向结构体的指针而不复制每个成员。

例如，以下是处理学生电话信息的两个函数：

```

1 void stu_phone1(struct cont_info *stu_info_ptr)
2 {
3     printf("%s phone number: %s", (*stu_info_ptr).name, (*stu_info_ptr).phone);
4 }
5
6 void stu_phone2(struct cont_info stu_info)
7 {

```



```

8   printf("%s phone number: %s", stu_info.name, stu_info.phone);
9 }

```

函数 `stu_phone1` 按地址传递参数，而 `stu_phone2` 按值传递参数。对于上述结构体变量 x ，若需调用函数 `stu_phone1`，则调用函数使用的语句应该为 “`stu_phone1(&x);`”；若需调用函数 `stu_phone2`，则调用函数使用的语句应该为 “`stu_phone2(x);`”。这两种情况下对应的栈中状态如图 3.26 所示。

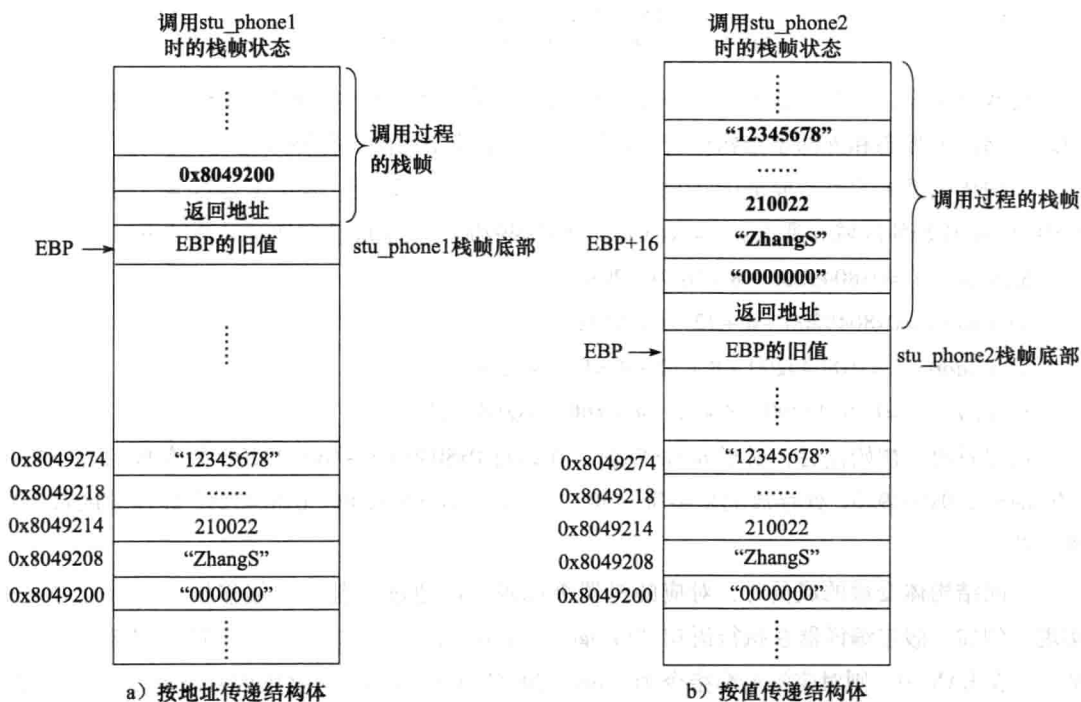


图 3.26 按地址传递和按值传递结构体数据

如图 3.26a 所示，对于按地址传递结构体数据方式，调用函数将会把 x 的地址 `0x8049200` 作为实参存到参数区，此时， $M[R[ebp] + 8] = 0x8049200$ 。在函数 `stu_phone1` 中，使用表达式 `(*stu_info).name` 来引用结构体成员 `name`，也可以将 `(*stu_info).name` 写成 `stu_info->name`。实现将表达式 `(*stu_info).name` 的结果送 `EAX` 的指令序列如下：

```

movl    8(%ebp), edx
leal    8(%edx), eax

```

执行完上述两条指令，`EAX` 中存放的是字符串 “ZhangS” 在静态存储区内的首地址 `0x8049208`。

如图 3.26b 所示，如果是按值传递结构体数据，调用函数将会把 x 的所有成员值作为实参存到参数区，此时，形参 `stu_info` 的地址为 $R[ebp] + 8$ 。在函数 `stu_phone2` 中，使用表达式 `stu_info.name` 来引用结构体成员 `name`。实现将表达式 `stu_info.name` 的结果送 `EAX` 的指令序列如下：

```

leal    8(%ebp), edx
leal    8(%edx), eax

```

上述两条指令的功能实际上是将 $R[\text{ebp}] + 16$ 的值送到 EAX 中, EAX 中存放的是字符串“ZhangS”在栈中参数区内的首地址。

从上述图 3.26 可以看出, 虽然调用 `stu_phone1` 和 `stu_phone2` 可以实现完全相同的功能, 但是两种方式下的时间和空间开销都不一样。显然, 后者的开销大, 因为它需要对结构体成员整体从静态存储区复制到栈中。

3.5.3 联合体数据的分配和访问

与结构体类似的还有一种联合体 (也称联合) 数据类型, 它也是不同数据类型的集合, 不过它与结构体数据相比, 在存储空间的使用方式上不同。结构体的每个成员占用各自的存储空间, 而联合体的各个成员共享存储空间, 也就是说, 在某一时刻, 联合体的存储空间中仅存在一个成员数据。因此, 联合体也称为共用体。

因为联合体的每个成员所占的存储空间大小可能不同, 因而分配给它的存储空间总是按最大数据长度成员所需空间大小为目标。例如, 对于以下联合体数据结构:

```
union uarea {
    char c_data;
    short s_data;
    int i_data;
    long l_data;
};
```

在 IA-32 上编译时, 因为 `long` 的长度和 `int` 的长度一样, 都是 32 位, 所以数据类型 `uarea` 所占存储空间大小为 4 字节。而对于与 `uarea` 有相同成员的结构体数据类型来说, 其占用的存储空间大小至少有 $1 + 2 + 4 + 4 = 11$ 字节, 如果考虑数据对齐的话, 则占用的空间更多。

联合体数据结构通常用在一些特殊的场合, 例如, 当事先知道某种数据结构中的不同字段 (成员) 的使用时间是互斥的, 就可以将这些字段声明为联合, 以减少分配的存储空间。但有时这种做法可能会得不偿失, 它可能只会减少少量的存储空间却大大增加处理复杂性。

利用联合体数据结构, 还可以实现对相同位序列进行不同数据类型的解释。例如, 以下函数可以对一个 `float` 型数据重新解释为一个无符号整数。

```
1 unsigned float2unsign(float f)
2 {
3     union {
4         float f;
5         unsigned u;
6     } tmp_union;
7     tmp_union.f = f;
8     return tmp_union.u;
9 }
```

上述函数的形式参数是 `float` 型, 按值传递参数, 因而从调用过程传递过来的实参是一个 `float` 型数据, 该数据被赋值给了一个非静态局部变量 `tmp_union` 中的成员 `f`, 由于成员 `u` 和成员 `f` 共享同一个存储空间, 所以在执行序号为 8 的 `return` 语句后, 32 位的浮点数被转换成了 32 位无符号整数。函数 `float2unsign` 的过程体中主要的指令就是 “`movl 8(%ebp), %eax`”, 它实现了将存放在地址 $R[\text{ebp}] + 8$ 处的入口参数 `f` 送到返回值所在寄存器 EAX 的功能。

从上述例子可以看出，机器级代码在很多时候并不区分所处理对象的数据类型，不管高级语言中将其说明成 float 型还是 int 型或 unsigned 型，都把它当成一个 0/1 序列来处理。明白这一点非常重要！

联合体数据结构可以嵌套，以下是一个关于联合体数据结构 node 的定义：

```
union node {
    struct {
        int *ptr;
        int data1
    } node1
    struct {
        int data2;
        union node *next;
    } node2;
}
```

可以看出数据结构 `node` 是一个如图 3.27 所示的链表, 在这个链表中除了最后一个节点采用 `node1` 结构类型外, 前面节点的数据类型都是 `node2` 结构, 其中有一个字段 `next` 又指向了一个 `node` 结构。



图 3.27 node 数据结构示意图

有一个处理 node 数据结构的过程 node_proc 如下:

```

1 void node_proc(union node *np)
2 {
3     np->next->data1 = *(np->next->ptr) + np->data2;
4 }

```

过程 `node_proc` 中的形式参数是一个指向 `node` 联合数据结构的指针，显然，它采用按地址传递参数方式，因此，在调用程序栈帧的参数区存放的实参是一个地址，这个地址是一个 `node` 型数据（即链表）的首地址。假定处理的链表被分配在某个存储区（通常像链表这种动态生成的数据结构都被分配在动态的堆区），其首地址为 `0xf0493000`。根据过程 `node_proc` 中第 3 行语句可知，所处理的链表共有两个节点，其中第一个节点是 `node2` 型结构，第二个节点是 `node1` 型结构，图 3.28 给出了其存放情况示意。

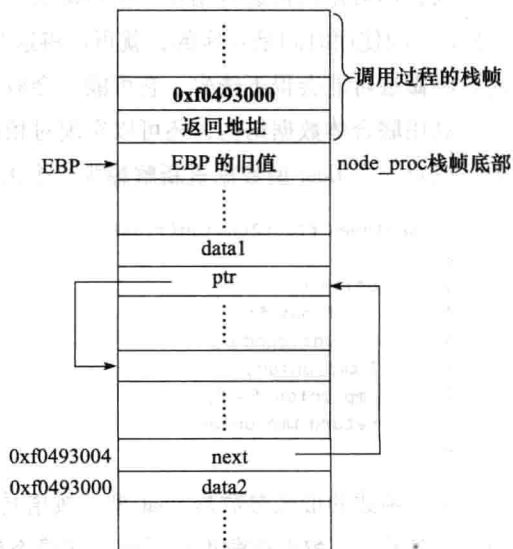


图 3.28 过程 node_proc 处理的 node 链表存放情况示意

过程 `node_proc` 的过程体对应的汇编代码如下。

```
1  movl    8(%ebp), %ecx    // 将实参(链表首址 0xf0493000)送 ECX
```

```

2  movl    4(%ecx), %edx    // 将地址 0xf0493004 中的 next 送 EDX
3  movl    (%edx), %eax     // 将 next 所指单元的内容 ptr 送 EAX
4  movl    (%eax), %eax     // 将 ptr 所指单元的内容送 EAX
5  addl    (%ecx), %eax     // 将 EAX 内容与 data2 相加
6  movl    %eax, 4(%edx)    // 将相加结果送 data1 所在单元

```

显然, 执行完上述机器级指令后, ECX 中存放的内容是链表首地址 0xf0493000, EDX 中存放的是指针 next。

3.5.4 数据的对齐

可以把存储器看作由连续的位 (cell) 构成, 每 8 位为一个字节, 每个字节有一个地址编号, 称为按字节编址。假定计算机系统中访存机制限制每次访存最多只能读写 64 位, 即 8 个字节, 那么, 第 0~7 字节可以同时读写, 第 8~15 字节可以同时读写, 以此类推, 这种称为 8 字节宽的存储机制。因此, 如果一条指令要访问的数据不在地址为 $8i \sim 8i+7$ ($i=0,1,2,\dots$) 之间的存储单元内, 那么就需要多次访存, 因而延长了指令的执行时间。例如, 若访问的数据在第 6、7、8、9 这四个字节中, 则需要访问存储器两次。因此, 数据在存储器中存放时需要进行对齐, 以避免多次访存而带来指令执行效率的降低。

当然, 对于底层机器级代码来说, 它应该能够支持按任意地址访问存储器数据的功能, 因此, 无论数据是否对齐, IA-32 都能正确工作, 只是在对齐方式下程序的执行效率更高。为此, 操作系统通常按照对齐方式分配管理内存, 编译器也按照对齐方式转换代码。

最简单的对齐策略是, 要求不同的基本类型按照其数据长度进行对齐, 例如, int 型数据长度是 4 个字节, 因此规定 int 型数据的地址是 4 的倍数。同理, short 型数据的地址是 2 的倍数, double 和 long long 型数据的地址是 8 的倍数, float 型数据的地址是 4 的倍数, char 型数据则无需对齐。微软 Windows 采用的就是这种对齐策略。这种情况下, 对于 8 字节宽的存储机制来说, 所有基本类型数据都仅需访存一次。Linux 的对齐策略更为宽松一点, 它规定: short 数据的地址是 2 的倍数, 其他的如 int、float、double 和指针等类型数据的地址都是 4 的倍数。这种情况下, 对于 8 字节宽的存储机制来说, double 型数据就可能需要访问两次。对于扩展精度浮点数, IA-32 中规定长度是 80 位, 即 10 个字节, 由于操作系统规定至少是 4 字节对齐, 所以, GCC 对于 long double 型数据分配了 12 个字节, 以使随后的相同类型数据能够落在 4 字节地址边界上。

对于由基本数据类型构造而成的 struct 结构体数据, 编译器为了保证其中每个字段都满足对齐要求, 它首先使结构体数据的首地址按 4 字节对齐, 同时, 根据基本数据类型对齐要求, 在各字段中插入相应的空间, 以使每个字段的成员变量满足对齐要求。

例如, 考虑下面的结构定义:

```

struct SD {
    int    i;
    short  si;
    char   c;
    double d;
}

```

如果不按照对齐方式分配空间，那么，SD 所占的存储空间大小为 $4 + 2 + 1 + 8 = 15$ 字节，每个字段的首地址偏移如图 3.29a 所示，字段 i、si、c 和 d 的偏移地址分别是 0、4、6 和 7，因此，即使 SD 的首地址按 4 字节边界对齐，字段 d 也不满足 4 字节或 8 字节对齐要求。为此，需要在字段 c 后面插入一个空字节，以使字段 d 的偏移从 8 开始，此时，每个字段的首地址偏移如图 3.29b 所示。这种情况下，只要 SD 首地址按 4 字节边界对齐，那么所有字段都能按要求对齐。而且，因为 SD 所占空间大小为 16 字节，因此，当定义一个数据元素为 SD 类型的结构数组时，每个数组元素也都能在 4 字节边界上对齐。

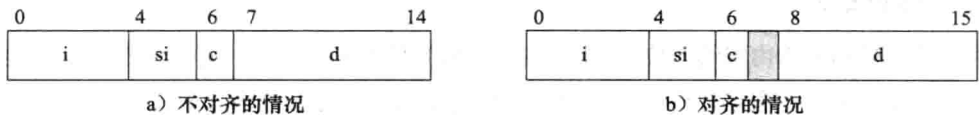


图 3.29 结构 SD 的存储分配情况

为了使结构数组中的每个元素都能满足对齐要求，编译器有时需要在结构的最后字段后面插入空间，例如，对于下面的结构数组定义：

```
struct SDT {
    int i;
    short si;
    double d;
    char c;
} sa[10];
```

如果按照图 3.30a 的方式在字段中插空，那么对于第一个元素 sa[0] 来说，则能够保证每个字段的对齐要求，但是，因为 SDT 所占总长度为 17 字节，所以，对于 sa[1] 来说，其首地址就不是按 4 字节方式对齐，因而导致 sa[1] 中各字段不能满足对齐要求。此时，编译器将在 SDT 结构的最后字段后面插入 3 个字节的空间，如图 3.30b 所示。此时，SDT 总长度为 20 字节，保证了结构数组中所有元素的首地址都是 4 的倍数。

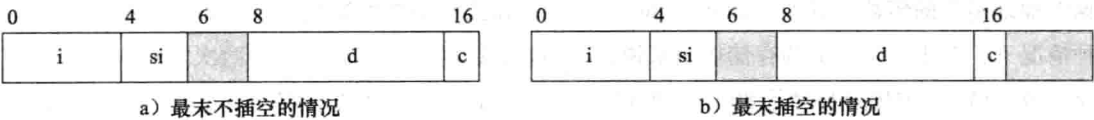


图 3.30 结构 SDT 的存储分配情况

3.6 越界访问和缓冲区溢出

在 3.4.1 节中我们介绍了有关 C 语言过程调用的机器级代码表示，在 C 语言程序执行过程中，当前正在执行的过程（即函数）在栈中会形成本过程的栈帧，一个过程的栈帧中除了保存 EBP 和被调用者保存寄存器的值外，还会保存本过程的非静态局部变量和过程调用的返回地址。

在 3.5.1 节中我们介绍了 C 语言中数组的分配和访问，C 语言中的数组元素可以使用指针来访问，因而对数组的引用没有边界约束，也即程序中对数组的访问可能会有意或无意地超越

数组存储区范围而无法发现。可以把这种数组存储区看成是一个缓冲区，这种超越数组存储区范围的访问称为缓冲区溢出。例如，对于一个有 10 个元素的 char 型数组，其定义的缓冲区有 10 个字节。如果写一个字符串到这个缓冲区，那么只要写入的字符串多于 9 个字符（结束符 ‘\0’ 占一个字节），则这个缓冲区就会发生“写溢出”。

3.6.1 缓冲区溢出攻击

缓冲区溢出是一种非常普遍、非常危险的漏洞，在各种操作系统、应用软件中广泛存在。缓冲区溢出攻击是利用缓冲区溢出漏洞所进行的攻击行动。缓冲区溢出攻击可以导致程序运行失败、系统关机、重新启动等后果。如果有人恶意利用在栈中分配的缓冲区的写溢出，悄悄地将一个恶意代码段的首地址作为“返回地址”覆盖地写到原先正确的返回地址处，那么，程序就会在执行 ret 指令时悄悄地转到这个恶意代码段执行，从而可以轻易取得系统特权，进而进行各种非法操作。

造成缓冲区溢出的原因是程序没有对栈中作为缓冲区的数组进行越界检查。下面用一个简单的例子说明攻击者如何利用缓冲区溢出跳转到自己设定的程序 hacker 去执行。

以下是在文件 test.c 中的三个函数，假定编译、链接后的可执行代码为 test。

```

1  #include <stdio.h>
2  #include "string.h"
3
4  void outputs(char *str)
5  {
6      char buffer[16];
7      strcpy(buffer, str);
8      printf("%s\n", buffer);
9  }
10
11 void hacker(void)
12 {
13     printf("being hacked\n");
14 }
15
16 int main(int argc, char *argv[])
17 {
18     outputs(argv[1]);
19     return 0;
20 }
21

```

上述函数 outputs 是一个有漏洞的程序，当命令行中给定的字符串超过 15 个字符时，使用 strcpy 函数就会使缓冲区造成写溢出。首先来看一下使用反汇编工具得到的 outputs 汇编代码。

```

1  0x080483e4 <outputs+0>:  push    %ebp
2  0x080483e5 <outputs+1>:  mov     %esp,%ebp
3  0x080483e7 <outputs+3>:  sub     $0x18,%esp
4  0x080483ea <outputs+6>:  mov     0x8(%ebp),%eax
5  0x080483ed <outputs+9>:  mov     %eax,0x4(%esp)
6  0x080483f1 <outputs+13>:  lea     0xffffffff0(%ebp),%eax
7  0x080483f4 <outputs+16>:  mov     %eax,(%esp)
8  0x080483f7 <outputs+19>:  call    0x8048330 <__gmon_start_@plt+16>

```

```
9 0x080483fc <outputs+24>: lea      0xffffffff0(%ebp),%eax
10 0x080483ff <outputs+27>: mov      %eax,0x4(%esp)
11 0x08048403 <outputs+31>: movl     $0x8048500, (%esp)
12 0x0804840a <outputs+38>: call     0x8048310
13 0x0804840f <outputs+43>: leave
14 0x08048410 <outputs+44>: ret
```

第3行指令说明编译器在栈帧中分配了 $0x18 = 24$ 个字节空间；在第8行 call 指令调用 strcpy 函数之前，栈中存放了两个参数，一个是 outputs 函数的入口参数 str(存放在栈中地址为 $R[ebp] + 8$ 之处)，另一个是 buffer 数组在栈中的首地址 $R[ebp] - 16$ ，可以看出第6行指令中的偏移量为 0xffffffff(真值为 -16)；第12行用 call 指令调用 printf 函数。根据上述分析，可以画出如图 3.31 所示的 outputs 的栈帧状态。

图 3.31 中传递给 strcpy 的实参 $M[R[ebp] + 8]$ 实际上就是在 main 函数中指定的命令行参数首地址，它是一个字符串的首址，此程序中函数 strcpy 实现的功能就是，将命令行中指定的字符串复制到缓冲区 buffer 中，如果攻击者在命令行中构造一个长度为 $16 + 4 + 4 + 1 = 25$ 个字符的字符串，并将 hacker 函数的首地址置于结束符 ‘\0’ 前面的 4 个字节，则在执行完 strcpy 函数后 hacker 代码段首地址被置于过程 main 栈帧最后的返回地址处，当执行到 outputs 代码的第14行 ret 指令时，便会转到 hacker 函数实施攻击。这里，25 个字符中的前 16 个字符填满 buffer 区，4 个字符覆盖掉 EBP 的旧值，4 个字节的 hacker 函数首地址覆盖返回地址，还有一个是字符串结束符。

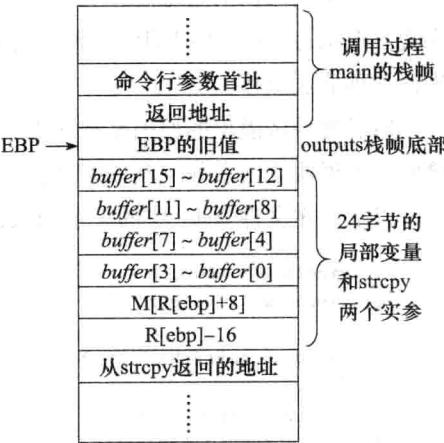


图 3.31 outputs 栈帧中的内容

假定 hacker 代码首地址为 0x08048411，则可编写如下的攻击代码实施攻击。

```
1 #include "stdio.h"
2
3 char code[] =
4 "0123456789ABCDEFXXXX"
5 "\x11\x84\x04\x08"
6 "\x00";
7 int main(void)
8 {
9     char *arg[3];
10    arg[0] = "./test";
11    arg[1] = code;
12    arg[2] = NULL;
13    execve(arg[0], arg, NULL);
14    return 0;
15 }
```

执行上述程序时，通过系统调用 execve() 可装入 test 可执行文件，并将 code 中的字符串作为命令行参数来启动执行 test。因此，字符串中前 16 个字符 ‘0’、‘1’、‘2’、‘3’、‘4’、‘5’、‘6’、‘7’、‘8’、‘9’、‘A’、‘B’、‘C’、‘D’、‘E’、‘F’ 被复制到 buffer 中，4 个字

符‘X’覆盖掉 EBP 的旧值，地址 0x08048411 覆盖掉返回地址。

执行上述攻击程序后的输出结果为：

```
"0123456789ABCDEFXXXX"
being hacked
Segmentation fault
```

输出结果中第一行为执行 outputs 函数后的结果，其中后面 4 个为不可显示字符（对应 ASCII 码 11H、84H、x04H 和 08H）；执行完 outputs 后程序被恶意地跳转到 hacker 函数执行，因此会显示第二行字符串；最后一行显示“Segmentation fault（段错误）”，其原因是在调用 hacker 函数时并没有保存其调用函数的返回地址，所以在执行到 hacker 过程的 ret 指令时取到的“返回地址”是一个不确定的值，因而可能跳转到数据区或系统区或其他非法访问的存储区去执行，造成段错误。

上面的错误主要是 strcpy() 函数没有进行缓冲区边界检查而直接把 str 中的内容复制到 buffer 造成的。存在像 strcpy 这样问题的标准函数还有 strcat()、sprintf()、vsprintf()、gets()、scanf() 等。

缓冲区溢出攻击有多种英文名称：buffer overflow, buffer overrun, smash the stack, trash the stack, scribble the stack, mangle the stack, memory leak, overrun screw 等。第一个缓冲区溢出攻击是 Morris 蠕虫，发生在 20 年前，它曾造成全世界 6000 多台网络服务器瘫痪。

随便往缓冲区中填东西造成它溢出一般只会出现段错误，而不能达到攻击的目的。最常见的手段是通过制造缓冲区溢出使程序运行一个用户 shell，再通过 shell 执行其他命令。如果该程序属于 root 且有 suid 权限的话，攻击者就获得了一个有 root 权限的 shell，就可以对系统进行任意操作了。

缓冲区溢出攻击之所以成为一种常见安全攻击手段，其原因在于缓冲区溢出漏洞太普遍了，并且易于实现。而且，缓冲区溢出成为远程攻击的主要手段，其原因在于缓冲区溢出漏洞使攻击者能够植入并且执行攻击代码。被植入的攻击代码以一定的权限运行有缓冲区溢出漏洞的程序，从而得到被攻击主机的控制权。

3.6.2 缓冲区溢出攻击的防范

缓冲区溢出攻击的存在给计算机的安全带来了很大的威胁，对于缓冲区溢出攻击，主要可以从两个方面来采取相应的防范措施，一个是从程序员角度，另一个是从编译器和操作系统方面。

对于程序员来说，应该尽量编写出没有漏洞的正确代码。当然，对于编写像 C 这种语法灵活、风格自由的高级语言程序，如果要编写出正确代码，通常需要花费较多的时间精力。为了帮助经验不足的程序员编写安全、正确的程序，人们开发了一些辅助工具和技术。最简单的方法就是用 grep 来搜索源代码中容易产生漏洞的库函数的调用，比如对 strcpy 和 sprintf 的调用，这两个函数都不会检查输入参数的长度；此外，人们还开发了一些高级的查错工具，如 fault injection 等，这些工具的目的在于通过人为随机地产生一些缓冲区溢出来寻找代码的安全漏洞；还有一些静态分析工具用于侦测缓冲区溢出的存在。虽然这些工具能帮助程序员开发更安全的

程序，但是，由于 C 语言的特点，这些工具不一定能找出所有缓冲区溢出漏洞，只能用来减少缓冲区溢出的可能。

对于编译器和操作系统来说，应该尽量生成没有漏洞的安全代码。现代编译器和操作系统已经采用了多种机制来保护缓冲区免受缓冲区溢出的攻击和影响，例如，有地址空间随机化、栈破坏检测和可执行代码区域限制等方式。

1. 地址空间随机化

地址空间随机化（Address Space Layout Randomization，简称 ASLR）是一种比较有效的防御缓冲区溢出攻击的技术，目前在 Linux、FreeBSD 和 Windows Vista 等主流操作系统中都使用了该技术。

基于缓冲区溢出漏洞的攻击者必须了解缓冲区的起始地址，以便将一个“溢出”的字符串以及指向攻击代码的指针植入具有漏洞的程序的栈中。对于早先的系统，每个程序的栈位置是固定的，在不同机器上生成和运行同一个程序时，只要操作系统相同，则栈的位置就完全一样。因而，程序中函数的栈帧首地址非常容易预测。如果攻击者可以确定一个有漏洞的常用程序所使用的栈地址空间，就可以设计一个针对性的攻击，在使用该程序的很多机器上实施攻击。

地址空间随机化的基本思路是，将加载程序时生成的代码段、静态数据段、堆区、动态库和栈区各部分的首地址进行随机化处理（起始位置在一定范围内是随机的），使得每次启动执行时，程序各段被加载到不同的地址起始处。由此可见，在不同机器上运行相同的程序时，程序加载的地址空间是不同的，显然，这种不同包括了栈地址空间的不同，因此，对于一个随机生成的栈起始地址，基于缓冲区溢出漏洞的攻击者不太容易确定栈的起始位置。通常将这种使程序加载的栈空间的起始位置随机变化的技术称为栈随机化。

如果攻击者使用蛮力多次反复使用不同的栈地址进行试探性攻击，那随机化防范措施还是有可能被攻破。这时就要用到下一步的栈破坏检测措施。

2. 栈破坏检测

如果在程序跳转到攻击代码执行之前，能够检测出程序的栈已被破坏，就可避免受到严重攻击。新的 GCC 版本在产生的代码中加入了一种栈保护者（stack protector）机制，用于检测缓冲区是否越界。主要思想是，在函数的准备阶段，在其栈帧中的缓冲区底部与保存的寄存器状态之间（例如，图 3.31 中 outputs 栈帧中的 `buffer[15]` 与保留的 EBP 之间）加入一个随机生成的特定值；在函数的恢复阶段，在恢复寄存器并返回到调用函数前，先检查该值是否被改变。若值发生改变，则程序异常终止。因为插入在栈帧中的特定值是随机生成的，所以攻击者很难猜测出它是什么。

3. 可执行代码区域限制

通过将程序的数据段地址空间设置为不可执行，从而使得攻击者不可能执行被植入在输入缓冲区的代码，这种技术也被称为非执行的缓冲区技术。早期 UNIX 系统只允许程序代码在代码段中执行，也即只有代码段的访问属性是可执行，其他区域的访问属性是可读或可读可写。但是，近来 UNIX 和 Windows 系统由于要实现更好的性能和功能，往往允许在数据段中动态地加入可执行代码，这是缓冲区溢出的根源。当然，为了保持程序的兼容性，不可能使所有数据

段都设置成不可执行。不过,可以将动态的栈段设置为不可执行,这样可以既保证程序的兼容性,又可以有效防止把代码植入栈(自动变量缓冲区)的溢出攻击。因为除了信息传递等少数情况下会使栈中存在可执行代码外,几乎没有任何合法的程序会在栈中存放可执行代码,因此这种做法几乎不产生任何兼容性问题。

不幸的是,栈的“不可执行”保护对于将攻击代码植入堆或者静态数据段的攻击没有效果,通过引用一个驻留程序的指针,就可以跳过这种保护措施。

3.7 兼容 IA-32 的 64 位系统

20 多年来,IA-32 体系结构一直是市场上最流行的通用处理器架构,1985 年推出的 Intel 80386 处理器是 IA-32 家族中第一款产品,它是典型的 CISC 风格指令集体系结构,而随后的 IA-32 处理器都与它保持向后兼容,因此,从整体上来看,IA-32 仍是基于 CISC 架构的体系结构。随着计算机技术及应用领域的不断发展,32 位处理器逐步开始向 64 位处理器过渡,最早的 64 位微处理器架构是 Intel 提出的采用全新指令集的 IA-64,而最早兼容 IA-32 的 64 位架构是 AMD 提出的 x86-64。

3.7.1 x86-64 的发展简史

Intel 最早推出的 64 位架构是基于超长指令字(Very Long Instruction Word, VLIW)技术的 IA-64 体系结构,Intel 称其为显式并行指令计算机(Explicitly Parallel Instruction Computer, EPIC)。Intel 的安腾(Itanium)和安腾 2(Itanium 2)处理器分别在 2000 年和 2002 年问世,它们是 IA-64 体系结构的最早的具体实现。安腾体系结构试图完全脱离 IA-32 CISC 架构的束缚,最大限度地提高软件和硬件之间的协同性,力求将处理器的处理能力和编译软件的功能结合起来,在指令中将并行执行信息以明显的方式告诉硬件。但是,这种思路被证明是不易实现的,而且,因为安腾采用了全新的指令集,虽然可以在兼容模式中执行 IA-32 代码,但是性能不太好,因而安腾并没有在市场上获得成功。

AMD 公司利用 Intel 公司在 IA-64 架构上的失败,抢先在 2003 年推出了兼容 IA-32 的 64 位版本指令集 x86-64,它在保留 IA-32 指令集的基础上,增加了新的数据格式及其操作指令,寄存器长度扩展为 64 位,并将通用寄存器个数从 8 个扩展到 16 个。通过 x86-64,AMD 获得了以前属于 Intel 的一些高端市场。AMD 后来将 x86-64 更名为 AMD64。

Intel 发现用 IA-64 直接替换 IA-32 行不通,于是,在 2004 年推出了 IA32-EM64T (Extended Memory 64 Technology, 64 位内存扩展技术),它支持 x86-64 指令集。Intel 为了表示 EM64T 的 64 位模式特点,又使其与 IA-64 有所区别,2006 年开始把 EM64T 改名为 Intel 64。因此,Intel 64 是与 IA-64 完全不同的体系结构,它与 IA-32 和 AMD64 兼容。

目前,AMD 的 64 位处理器架构 AMD64 和 Intel 的 64 位处理器架构 Intel 64 都支持 x86-64 指令集,因而,通常人们直接使用 x86-64 代表 64 位 Intel 指令集架构。x86-64 有时也简称为 x64。

3.7.2 x86-64 的基本特点

对于编译器来说,对高级语言程序进行编译可以有两种选择,一种是按 IA-32 指令集将目标编译成 IA-32 代码,一种是按 x86-64 指令集将目标编译成 x86-64 代码。通常,在 IA-32 架构上运行的是 32 位操作系统, GCC 默认生成 IA-32 代码;在 x86-64 架构上运行的是 64 位操作系统, GCC 默认生成 x86-64 代码。Linux 和 GCC 将前者称为“i386”平台,将后者称为“x86-64”平台。

与 IA-32 代码相比, x86-64 代码主要有以下几个方面的特点。

① 比 IA-32 具有更多的通用寄存器个数。新增的 8 个 64 位通用寄存器名称分别为 R8、R9、R10、R11、R12、R13、R14 和 R15。它们可以作为 8 位寄存器 (R8B ~ R15B)、16 位寄存器 (R8W ~ R15W) 或 32 位寄存器 (R8D ~ R15D) 使用,以访问其中的低 8 位、低 16 位或低 32 位。

② 比 IA-32 具有更长的通用寄存器位数,从 32 位扩展到 64 位。在 x86-64 中,所有通用寄存器 (GPR) 都从 32 位扩充到了 64 位,名称也发生了变化。8 个 32 位通用寄存器 EAX、EBX、ECX、EDX、EBP、ESP、ESI 和 EDI 对应的扩展为 64 位的寄存器分别被命名为 RAX、RBX、RCX、RDX、RBP、RSP、RSI 和 RDI。

在 IA-32 中,四个寄存器 EBP、ESP、ESI 和 EDI 的低 8 位不能使用,而在 x86-64 架构中,可以使用这些寄存器的低 8 位,其对应的寄存器名称为 BPL、SPL、SIL 和 DIL。

整数操作不仅支持 8、16、32 位数据类型,还支持 64 位数据类型。所有算术逻辑运算、寄存器与内存之间的数据传输,都能以最多 64 位为单位进行操作。栈的压入和弹出操作都以 8 字节为单位进行。

③ 字长从 32 位变为 64 位,因而逻辑地址从 32 位变为 64 位。指针 (如 `char*` 型) 和长整型 (long 型) 数据从 32 位扩展到 64 位,与 IA-32 平台相比,理论上其数据访问的空间大小从 $2^{32} \text{B} = 4\text{GB}$ 扩展到了 $2^{64} \text{B} = 16\text{EB}$ 。不过,目前仅支持 48 位逻辑地址空间,即逻辑地址从 4GB 增加到了 256TB。

④ 对于 long double 型数据,虽然还是采用与 IA-32 相同的 80 位扩展精度格式,但是,所分配的存储空间从 IA-32 的 12 字节扩展为 16 字节。也即,此类数据的边界从 12B 对齐改为 16B 对齐,不管是分配 12 字节还是分配 16 字节,都只会用到低 10 个字节。

⑤ 过程调用时,对于入口参数只有 6 个以内的整型变量和指针型变量的情况,通常就用通用寄存器而不是栈来传递,因而,很多过程可以不用访问栈,这样,使得大多数情况下执行时间比 IA-32 代码更短。

⑥ 128 位的 MMX 寄存器从原来的 8 个增加到了 16 个,浮点操作采用基于 SSE 的面向 XMM 寄存器的指令集,而不采用基于浮点寄存器栈的指令集。

3.7.3 x86-64 的基本指令和对齐

x86-64 指令集在兼容 IA-32 的基础上,还能支持 64 位数据操作指令,大部分操作数指示符与 IA-32 一样,所不同的是,当指令中的操作数为存储器操作数时,其基址寄存器或变址寄存器都必须是 64 位寄存器;此外,在运算类指令中,除了支持原来 IA-32 中的寻址方式以外, x86-64 还支持 PC 相对寻址方式,相对地址是一个 32 位补码表示的数。

1. 数据传送指令

在 x86-64 中, 提供了一些在 IA-32 中没有的新的数据传送指令, 例如, `movabsq` 指令用于将一个 64 位立即数送到一个 64 位通用寄存器中; `movq` 指令用于传送一个 64 位的四字; `movsbq`、`movswq`、`movslq` 用于将源操作数进行符号扩展并传送到一个 64 位寄存器或存储单元中; `movzbq`、`movzwq` 用于将源操作数进行零扩展后传送到一个 64 位寄存器或存储单元中; `pushq` 和 `popq` 分别是四字压栈和四字出栈指令。汇编指令中指令助记符结尾处的“q”表示操作数长度为四字 (64 位)。

在 x86-64 中, `movl` 指令的功能与在 IA-32 中有点不同, 它在传送 32 位寄存器内容的同时, 还会将目的寄存器的高 32 位自动清 0, 因此, 在 x86-64 中, `movl` 指令的功能相当于 `movzlw` 指令, 因而在 x86-64 中不需要 `movzlw` 指令。

例 3.13 以下是一个 C 语言函数, 其功能是将类型为 `source_type` 的参数转换为 `dest_type` 类型的数据并返回。

```
dest_type convert(source_type x) {
    dest_type y = (dest_type) x;
    return y;
}
```

根据过程调用时的参数传递约定可知, `x` 存放在寄存器 RDI 对应的适合宽度的寄存器 (如 RDI、EDI、DI 和 DIL) 中, `y` 存放在 RAX 对应宽度的寄存器 (RAX、EAX、AX 或 AL) 中。表 3.9 中给出了 `source_type` 和 `dest_type` 的类型组合, 请给出相应的汇编指令, 以实现 `convert` 函数中的赋值语句。

表 3.9 例 3.13 中 `source_type` 和 `dest_type` 的类型

source_type	dest_type
char	long
int	long
long	long
long	int
unsigned int	unsigned long
unsigned long	unsigned int
unsigned char	unsigned long

解 根据 x86-64 数据传输指令的功能, 得到对应表 3.9 所示类型的汇编指令见表 3.10, 表中汇编指令为 AT&T 格式。在表 3.10 中, 将 long 型数据转换为 int 型数据时, 可以用两种不同的指令 `movslq` 和 `movl`。虽然执行这两种指令得到的 RAX 中高 32 位内容可能不同, 但是, EAX 中的结果是一样的。因为函数返回的是 int 型数据, 所以 RAX 中高 32 位的值没有意义, 只要 EAX 中的 32 位值正确即可。

表 3.10 例 3.13 的答案

source_type	dest_type	汇 编 指 令
char	long	<code>movsbq %dil, %rax</code>
int	long	<code>movslq %edi, %rax</code>
long	long	<code>movq %rdi, %rax</code>
long	int	<code>movslq %edi, %rax</code> //符号扩展到 64 位,使 RAX 中高 32 位为符号 <code>movl %edi, %eax</code> //零扩展到 64 位,使 RAX 中高 32 位为 0
unsigned int	unsigned long	<code>movl %edi, %eax</code> //零扩展到 64 位,使 RAX 中高 32 位为 0
unsigned long	unsigned int	<code>movl %edi, %eax</code> //零扩展到 64 位,使 RAX 中高 32 位为 0
unsigned char	unsigned long	<code>movzbq %dil, %rax</code> //零扩展到 64 位,使 RAX 中高 56 位为 0

2. 算术逻辑运算指令

在 x86-64 中, 增加了操作数长度为四字的运算类指令 (长度后缀为 q), 例如, addq (四字相加)、subq (四字相减)、imulq (带符号整数四字相乘)、orq (64 位相或)、leaq (有效地址加载到 64 位寄存器) 等。

例 3.14 以下是 C 语言赋值语句 “ $x = a * b + c * d;$ ” 对应的 x86-64 汇编代码, 已知 x 、 a 、 b 、 c 和 d 分别在寄存器 RAX、RDI、RSI、RDX 和 RCX 对应宽度的寄存器中。根据以下汇编代码, 推测变量 x 、 a 、 b 、 c 和 d 的数据类型。

```
1 movslq %ecx, %rcx
2 imulq %rdx, %rcx
3 movsbl %sil, %esi
4 imull %edi, %esi
5 movslq %esi, %rsi
6 leaq (%rcx, %rsi), %rax
```

解 根据第 1 行可知, 在 ECX 中的变量 d 从 32 位符号扩展为 64 位, 因此, 变量 d 的数据类型为 int 型; 根据第 2 行可知, 在 RDX 中的变量 c 为 64 位整型, 即 c 的数据类型为 long 型; 根据第 3 行可知, 在 SIL 中的变量 b 为 char 型数据; 根据第 4 行可知, 在 EDI 中的 a 是 int 型数据; 根据第 5 行和第 6 行可知, 存放在 RAX 中的 x 是 long 型数据。 ■

3. 过程调用

在 x86-64 中, 过程调用通过寄存器传送参数, 因而寄存器的功能有一点变化。首先, 在 x86-64 中, 无需帧指针寄存器, 因而 RBP 可作为普通寄存器使用; 其次, 传送入口参数的寄存器依次为 RDI、RSI、RDX、RCX、R8 和 R9, 返回参数存放在 RAX 中; 第三, 调用者保存寄存器为 R10 和 R11, 被调用者保存寄存器为 RBX、RBP、R12、R13、R14 和 R15; 第四, RSP 用于指向栈顶元素; 第五, RIP 用于指向正在执行或即将执行的指令。

如果入口参数是整数类型或指针类型且少于等于 6 个, 则无需用栈来传递参数, 如果同时该过程无需在栈中存放局部变量和被调用者保存寄存器的值, 那么, 该过程就不需要栈帧。传递参数时, 如果参数是 32 位、16 位或 8 位, 则参数被置于对应宽度的寄存器部分。例如, 若第一个入口参数是 char 型, 则放在 RDI 中对应字节宽度的寄存器 DIL 中; 若返回参数是 short 型, 则放在 RAX 中对应 16 位宽度的寄存器 AX 中。表 3.11 给出了每个入口参数和返回参数所在的对应寄存器。

表 3.11 过程调用时参数对应的寄存器

操作数宽度 (字节)	入口参数						返回 参数
	1	2	3	4	5	6	
8	RDI	RSI	RDX	RCX	R8	R9	RAX
4	EDI	ESI	EDX	ECX	R8D	R9D	EAX
2	DI	SI	DX	CX	R8W	R9W	AX
1	DIL	SIL	DL	CL	R8B	R9B	AL

在 x86-64 中, 最多可以有 6 个整型或指针型入口参数通过寄存器传递, 超过 6 个入口参数时, 后面的通过栈来传递, 在栈中传递的参数若是基本类型数据, 则不管是什么基本类型都被

分配8个字节。当入口参数少于6个或者当入口参数已经被用过而不再需要时,存放对应参数的寄存器可以被函数作为临时寄存器使用。对于存放返回结果的 RAX 寄存器,在产生最终结果之前,也可以作为临时寄存器被函数重复使用。

在 x86-64 中,调用指令 call (或 callq) 将一个 64 位返回地址保存在栈中并执行 $R[\text{rsp}] \leftarrow R[\text{rsp}] - 8$ 。返回指令 ret 也是从栈中取出 64 位返回地址并执行 $R[\text{rsp}] \leftarrow R[\text{rsp}] + 8$ 。

例 3.15 以下是函数 caller 和 test 的 C 语言源程序。

```
1 long caller()
2 {
3     char a=1; short b=2; int c=3; long d=4;
4     test(a, &a, b, &b, c, &c, d, &d);
5     return a*b+c*d;
6 }
7 void test(char a, char *ap, short b, short *bp, int c, int *cp, long d, long *dp)
8 {
9     *ap += a;    *bp += b;    *cp += c;    *dp += d;
10 }
```

假定上述源程序对应的 x86-64 汇编代码如下。

函数 caller 的汇编代码如下:

```
1 caller:
2 subq    $32, %rsp           // R[rsp] ← R[rsp] - 32
3 movb    $1, 16(%rsp)        // M[R[rsp] + 16] ← 1
4 movw    $2, 18(%rsp)        // M[R[rsp] + 18] ← 2
5 movl    $3, 20(%rsp)        // M[R[rsp] + 20] ← 3
6 movq    $4, 24(%rsp)        // M[R[rsp] + 24] ← 4
7 leaq    24(%rsp), %rax       // R[rax] ← R[rsp] + 24
8 movq    %rax, 8(%rsp)        // M[R[rsp] + 8] ← R[rax]
9 movq    $4, (%rsp)          // M[R[rsp]] ← 4
10 leaq    20(%rsp), %r9       // R[r9] ← R[rsp] + 20
11 movl    $3, %r8d            // R[r8d] ← 3
12 leaq    18(%rsp), %rcx      // R[rcx] ← R[rsp] + 18
13 movw    $2, %dx             // R[dx] ← 2
14 leaq    16(%rsp), %rsi      // R[rsi] ← R[rsp] + 16
15 movb    $1, %dil            // R[dil] ← 1
16 call    test
17 movslq   20(%rsp), %rcx      // R[rcx] ← M[R[rsp] + 20], 符号扩展
18 movq    24(%rsp), %rdx      // R[rdx] ← M[R[rsp] + 24]
19 imulq    %rdx, %rcx         // R[rcx] ← R[rcx] × R[rdx]
20 movsbw   16(%rsp), %ax      // R[ax] ← M[R[rsp] + 16], 符号扩展
21 movw    18(%rsp), %dx       // R[dx] ← M[R[rsp] + 18]
22 imulw    %dx, %ax           // R[ax] ← R[ax] × R[dx]
23 movswq   %ax, %rax          // R[rax] ← R[ax], 符号扩展
24 leaq    (%rax, %rcx), %rax   // R[rax] ← R[rax] + R[rcx]
25 addq    $32, %rsp           // R[rsp] ← R[rsp] + 32
26 ret
```

函数 test 的汇编代码如下:

```
1 test:
2 movq    16(%rsp), %r10       // R[r10] ← M[R[rsp] + 16]
3 addb    %dil, (%rsi)         // M[R[rsi]] ← M[R[rsi]] + R[dil]
4 addw    %dx, (%rcx)          // M[R[rcx]] ← M[R[rcx]] + R[dx]
5 addl    %r8d, (%r9)          // M[R[r9]] ← M[R[r9]] + R[r8d]
```



```

6 movq    8(%rsp), %rax    // R[rax]←M[R[rsp]+8]
7 addq    %rax, (%r10)     // M[R[r10]]←M[R[r10]]+R[rax]
8 ret

```

要求根据上述汇编代码，分别画出在执行到 caller 函数的 call 指令时、执行到 test 函数的 ret 指令时栈中信息的存放情况，并说明 caller 是如何把实参传递给 test 中的形参的，而 test 执行时其每个入口参数又是如何获得的。

解 从 caller 汇编代码可以看出，栈指针寄存器 RSP 仅在第 2 行做了一次减法，申请了 32 字节的空间，在最后第 25 行恢复 RSP 之前一直没有变化，说明 caller 栈帧就是 32 字节。第 3~6 行用来在栈帧中分配局部变量 *a*、*b*、*c* 和 *d*，并将初值存入相应单元。可以看出，这 4 个变量一共占用了 16 个字节。第 7~15 行用来将实参存入 test 入口参数对应的寄存器中，因为有 8 个入口参数，所以还有两个参数需要通过栈进行传递，其中第 7~8 行用于在栈中存入第 8 个参数，第 9 行用于在栈中存入第 7 个参数，第 10~15 行分别用于在相应的寄存器中存入第 6、第 5、第 4、第 3、第 2 和第 1 个参数。因此，在执行到第 16 行 call 指令之时，前 6 个参数分别在寄存器 RDI、RSI、DX、RCX、R8D 和 R9 中，第 7 和第 8 个参数在栈中的位置分别由 R[*rsp*] 和 R[*rsp*]+8 指出。图 3.32a 给出了此时 caller 栈帧中信息的存放情况。

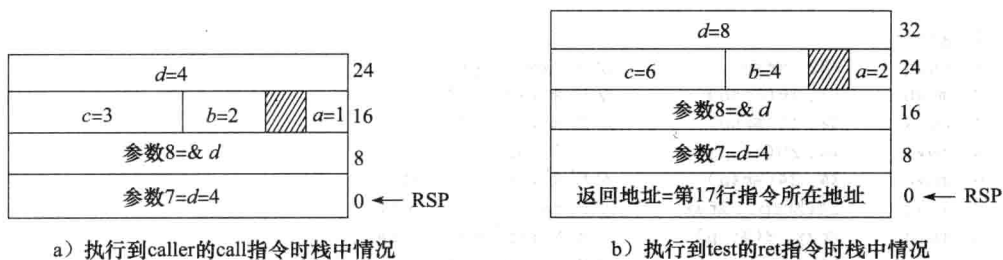


图 3.32 caller 和 test 执行时栈中信息存放情况

在 caller 中的 call 指令执行后，栈指针寄存器 RSP 的内容减 8，并将 call 指令下面一条指令的地址（第 17 行指令所在地址）作为返回地址存入当前 RSP 所指单元，然后跳转到 test 执行。在 test 执行过程中，第 2 行指令用来取出第 8 个参数，第 3~5 行分别用于实现赋值语句 “*ap += a;”、“*bp += b;” 和 “*cp += c;”。其中，指针类型变量 *ap*、*bp* 和 *cp* 的值各是 caller 中局部变量 *a*、*b* 和 *c* 在栈中的地址，即 *ap = *a*、*bp = *b*、*cp = *c*。执行完第 3~5 行指令后，栈中 *a*、*b* 和 *c* 处的内容为原来内容的两倍。第 6 行指令用来取出第 7 个参数（其值为 4），第 7 行指令用来将 4 加到第 8 个参数所指单元 *d* 处，使得 *d* 处的内容变为 8。综上所述，在执行到 test 的 ret 指令时，栈中信息存放情况如图 3.32b 所示。

这里要特别说明的是，即使在栈中传递的最后两个参数不是 long 型或指针类型，也都被分配 8 个字节。例如，假定上述 test 函数的原型为 “void test(char a, char *ap, short b, short *bp, long d, long *dp, int c, int *cp)”，即在栈中传递的最后两个参数类型是 4 字节的 int 型和 8 字节的指针型，它们在栈中所占的空间都是 8 个字节。

4. 浮点操作与 SIMD 指令

在 x86-64 中，浮点运算采用基于 SSE 的面向 XMM 寄存器的 SIMD 指令，浮点数存放在

128 位的 XMM 寄存器中，而不是存放 x87 FPU 的 80 位浮点寄存器栈中。

例 3.16 以下是一段 C 语言代码：

```
1 #include <stdio.h>
2
3 main()
4 {
5     double a = 10;
6     printf("a = %d\n", a);
7 }
```

上述代码在 IA-32 平台上运行时，打印出来的结果总是 $a=0$ ，但是在 x86-64 平台上运行时，打印出来的 a 却是一个不确定的值，请问为什么？

解 本题给出的代码的主要功能是，将一个 64 位双精度浮点数 ‘10’ 转换为一个 32 位二进制数，然后以十进制数形式打印出来。

IEEE 754 双精度浮点数由 64 位组成，最高位为符号位 s ，随后的 11 位为阶码 e ，其偏置常数为 1023，余下 52 位为尾数 f 。因为 $10 = 1010\text{B} = 1.01\text{B} \times 2^3$ ，因此 $s = 0$ ， $e = 1023 + 3 = 100\ 0000\ 0010$ ， $f = 0100\cdots 0$ 。也即 64 位机器数为 0 100 0000 0010 0100 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000，因此， a 的机器数用十六进制形式表示的字节序列为 40H、24H、00H、00H、00H、00H、00H、00H。将其高 32 位转换为十进制数，其值为 1 076 101 120，低 32 位的值为 0。

在 IA-32 中，过程采用栈传递参数。图 3.33 给出了在 IA-32 中 printf 的参数在栈中的存放情况。因为 IA-32 是小端方式，所以 a 的高位部分在栈中的高地址上，低位部分在栈中的低地址上。

当 printf 将变量 a 的值使用 “%d” 输出时，对应的数据类型是 int 型，因此取 a 中低 4 字节作为第 2 个参数。显然，printf 会从栈中 $R[\text{ebp}] + 12$ 的位置开始从低地址到高地址读取 4 个字节作为 int 型数据来解释并输出，因此，代码打印输出的结果为 “ $a=0$ ”。

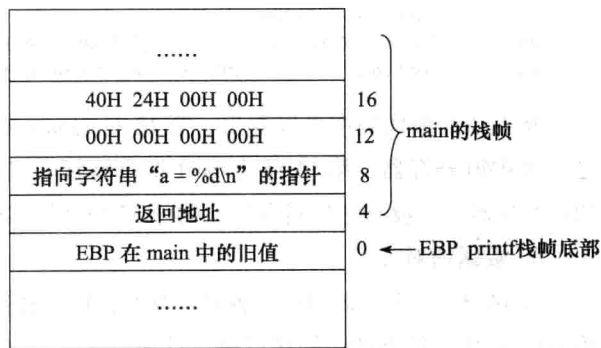


图 3.33 IA-32 平台使用栈进行参数传递

在 x86-64 中，过程采用通用寄存器传递参数，因为该题 printf 共有两个参数且使用 “%d” 输出，因此，这两个参数应该各自通过 RDI 和 ESI 进行传递，其中 RDI 中存放字符串 “a = %d\n” 的首地址，而 ESI 中存放 a 的低 32 位，printf 函数会到默认的 RDI 和 ESI 中去取相应的参数进行处理。但是，因为本题中 a 是一个 double 型的浮点数据，所以，在 x86-64 中会把 a 的值送到浮点寄存器 XMM 中，而不会传到 ESI 中，因此，在 printf 执行时，当从 ESI 中读取要打印的 int 型变量时，实际上不会得到 a 的低 32 位，而是当时 ESI 寄存器中的内容。每次执行上述代码时，ESI 中的内容都可能发生变化，因而每次打印出来的值都可能不同。 ■

以下是在某个 x86-64 平台上对上述源代码进行编译的结果。

(以下代码来自网页 <http://www.2cto.com/kf/201211/172355.html>)

```

1      .file "double_as_int.c"
2      .section .rodata.str1.1,"aMS",@progbits,1
3      .LC1:
4      .string "a = %d\n"
5      .text
6      .globl main
7      .type main,@function
8      main:
9      .LFB11:
10     .cfi_startproc
11     subq    $8,%rsp
12     .cfi_def_cfa_offset 16
13     movsd   .LC0(%rip),%xmm0
14     movl    $.LC1,%edi
15     movl    $1,%eax
16     call    printf
17     addq    $8,%rsp
18     .cfi_def_cfa_offset 8
19     ret
20     .cfi_endproc
21     .LFE11:
22     .size    main,.-main
23     .section .rodata.cst8,"aM",@progbits,8
24     .align 8
25     .LC0:
26     .long    0
27     .long    1076101120
28     .ident   "GCC: (GNU) 4.4.6 20120305 (Red Hat 4.4.6-4)"
29     .section .note.GNU-stack,"",@progbits

```

从上述汇编代码中可以看出，第 13 行的 `movsd` 指令用来将标号 `.LC0` 处的双精度浮点数 a 送入 `XMM0` 寄存器，第 14 行的 `movl` 指令将标号 `.LC1` 的值（指向字符串 “`a = %d\n`”）送入 `EDI` 寄存器，并没有任何指令将 a 的低 32 位送到 `ESI` 寄存器中。

5. 数据的对齐

与 IA-32 一样，x86-64 中各种类型数据应该遵循一定的对齐规则，而且要求更加严格。因为 x86-64 中存储器的访问接口被设计成以 8 字节或 16 字节为单位进行存取，其对齐规则是，任何 K 字节宽的基本类型数据和指针类型数据的起始地址一定是 K 的倍数。因此，`long` 型、`double` 型数据和指针型变量都必须按 8 字节边界对齐；`long double` 型数据必须按 16 字节边界对齐。

3.8 小结

本章对 C 语言中的各类语句、各种数据类型以及运算在 IA-32 和 x86-64 上的机器级实现做了比较详细的介绍。虽然高级语言选用了 C 语言，机器级表示选用了 IA-32 和 x86-64 架构，但是，实际上从其他高级语言到其他体系结构的对应关系也是类似的。

为了系统地说明 C 语言程序在 IA-32 体系结构上的机器级表示，首先介绍了 IA-32 支持的数据类型、寄存器组织、寻址方式、常用指令类型、指令格式和指令的功能。然后，对过程调

用、选择语句、循环结构、复杂数据类型操作等所对应的机器级表示进行了介绍。最后还简单介绍了兼容 IA-32 的 x86-64 指令集体系结构,并简单说明了 C 语言程序在 x86-64 架构上的机器级表示。

从本章内容可以看出,编译器在将高级语言源程序转换为机器级代码时,必须对目标代码对应的指令集体系结构有充分的了解。编译器需要决定高级语言程序中的变量应该使用哪种数据类型表示格式,需要为高级语言程序中的常数和变量合理地分配寄存器或存储空间,需要确定哪些变量应该分配在静态数据区,哪些变量应该分配在动态的堆区或栈区,需要选择合适的指令序列来实现选择结构和循环结构。对于过程调用,编译器需要按约定实现参数传递、保存和恢复寄存器的状态等。由于 C 语言对数组边界没有约束检查,容易导致缓冲区溢出漏洞,因此,需要程序员、操作系统和编译器采用相应的防范措施。

如果一个应用程序员能够熟练掌握应用程序所运行的平台与环境,包括指令集体系结构、操作系统和编译工具,并且能够深刻理解高级语言程序与机器级程序之间的对应关系,那么,就能更容易理解程序的行为和执行结果,更容易编写出高效、安全、正确的程序,并在程序出现问题时能够较快地定位错误发生的根源。

习题

1. 给出以下概念的解释说明。

机器语言程序	机器指令	汇编语言	汇编指令
汇编语言程序	汇编助记符	汇编程序	反汇编程序
机器级程序	通用寄存器	定点通用寄存器	栈指针寄存器
指令指针寄存器	标志寄存器	条件标志 (条件码)	控制标志
寻址方式	立即寻址	寄存器寻址	存储器操作数
实地址模式	保护模式	比例变址	非比例变址
比例系数 (比例因子)	MMX 指令	SSE 指令集	SIMD
多媒体扩展通用寄存器	栈	调用者保存寄存器	被调用者保存寄存器
帧指针寄存器	当前栈帧	按值传递参数	按地址传递参数
嵌套调用	递归调用	缓冲区溢出	缓冲区溢出攻击
栈随机化			

2. 简单回答下列问题。

- (1) 一条机器指令通常由哪些字段组成? 各字段的含义分别是什么?
- (2) 将一个高级语言源程序转换成计算机能直接执行的机器代码通常需要哪几个步骤?
- (3) IA-32 中的逻辑运算指令如何生成条件标志? 移位指令可能会改变哪些条件标志?
- (4) 执行条件转移指令时所用到的条件标志信息从何而来? 请举例说明。
- (5) 无条件转移指令和调用指令的相同点和不同点是什么?
- (6) 按值传递参数和按地址传递参数两种方式有哪些不同点?
- (7) 为什么在递归深度较深时递归调用的时间开销和空间开销都会较大?

(8) 为什么数据在存储器中最好按地址对齐方式存放?

(9) 造成缓冲区溢出的根本原因是什么?

3. 对于以下 AT&T 格式汇编指令, 根据操作数的长度确定对应指令助记符中的长度后缀, 并说明每个操作数的寻址方式。

(1) `mov 8(%ebp, %ebx, 4), %ax`

(2) `mov %al, 12(%ebp)`

(3) `add (, %ebx, 4), %ebx`

(4) `or (%ebx), %dh`

(5) `push $0xF8`

(6) `mov $0xFFFF0, %eax`

(7) `test %cx, %cx`

(8) `lea 8(%ebx, %esi), %eax`

4. 使用汇编器处理以下各行 AT&T 格式代码时都会产生错误, 请说明每一行存在什么错误。

(1) `movl 0xFF, (%eax)`

(2) `movb %ax, 12(%ebp)`

(3) `addl %ecx, $0xF0`

(4) `orw $0xFFFF0, (%ebx)`

(5) `addb $0xF8, (%d1)`

(6) `movl %bx, %eax`

(7) `andl %esi, %esx`

(8) `movw 8(%ebp, , 4), %ax`

5. 假设变量 x 和 ptr 的类型声明如下:

```
src_type x;  
dst_type *ptr;
```

这里, `src_type` 和 `dst_type` 是用 `typedef` 声明的数据类型。有以下 C 语言赋值语句:

```
*ptr = (dst_type) x;
```

若 x 存储在寄存器 EAX 或 AX 或 AL 中, ptr 存储在寄存器 EDX 中, 则对于下表中给出的 `src_type` 和 `dst_type` 的类型组合, 写出实现上述赋值语句的机器级代码。要求用 AT&T 格式汇编指令表示机器级代码。

src_type	dst_type	机器级表示
char	int	
int	char	
int	unsigned	
short	int	
unsigned char	unsigned	
char	unsigned	
int	int	

6. 假设某个 C 语言函数 `func` 的原型声明如下:

```
void func(int *xptr, int *yptr, int *zptr);
```

函数 func 的过程体对应的机器级代码用 AT&T 汇编形式表示如下：

```
1 movl 8(%ebp), %eax
2 movl 12(%ebp), %ebx
3 movl 16(%ebp), %ecx
4 movl (%ebx), %edx
5 movl (%ecx), %esi
6 movl (%eax), %edi
7 movl %edi, (%ebx)
8 movl %edx, (%ecx)
9 movl %esi, (%eax)
```

请回答下列问题或完成下列任务。

- (1) 在过程体开始时三个入口参数对应实参所存放的存储单元地址是什么？（提示：当前栈帧底部由帧指针寄存器 EBP 指示。）
 - (2) 根据上述机器级代码写出函数 func 的 C 语言代码。
7. 假设变量 x 和 y 分别存放在寄存器 EAX 和 ECX 中，请给出以下每条指令执行后寄存器 EDX 中的结果。
- (1) `leal (%eax), %edx`
 - (2) `leal 4(%eax, %ecx), %edx`
 - (3) `leal (%eax, %ecx, 8), %edx`
 - (4) `leal 0xC(%ecx, %eax, 2), %edx`
 - (5) `leal (, %eax, 4), %edx`
 - (6) `leal (%eax, %ecx), %edx`
8. 假设以下地址以及寄存器中存放的机器数如下表所示。

地址	机器数	寄存器	机器数
0x8049300	0xffffffff0	EAX	0x8049300
0x8049400	0x80000008	EBX	0x100
0x8049384	0x80f7ff00	ECX	0x10
0x8049380	0x908f12a8	EDX	0x80

分别说明执行以下指令后，哪些地址或寄存器中的内容会发生改变？改变后的内容是什么？条件标志 OF、SF、ZF 和 CF 会发生什么改变？

- (1) `addl (%eax), %edx`
 - (2) `subl (%eax, %ebx), %ecx`
 - (3) `orw 4(%eax, %ecx, 8), %bx`
 - (4) `testb $0x80, %dl`
 - (5) `imull $32, (%eax, %edx)`
 - (6) `decw %cx`
9. 假设函数 operate 的部分 C 代码如下：
- ```
1 int operate(int x, int y, int z, int k)
2 {
```

```

3 int v = _____ ;
4 return v;
5 }

```

以下汇编代码用来实现第3行语句的功能，请写出每条汇编指令的注释，并根据以下汇编代码，填写 operate 函数缺失的部分。

```

1 movl 12(%ebp), %ecx
2 sall $8, %ecx
3 movl 8(%ebp), %eax
4 movl 20(%ebp), %edx
5 imull %edx, %eax
6 movl 16(%ebp), %edx
7 andl $65520, %edx
8 addl %ecx, %edx
9 subl %edx, %eax

```

10. 假设函数 product 的 C 语言代码如下，其中 num\_type 是用 typedef 声明的数据类型。

```

1 void product(num_type *d, unsigned x, num_type y) {
2 *d = x*y;
3 }

```

函数 product 的过程体对应的主要汇编代码如下：

```

1 movl 12(%ebp), %eax
2 movl 20(%ebp), %ecx
3 imull %eax, %ecx
4 mull 16(%ebp)
5 leal (%ecx, %edx), %edx
6 movl 8(%ebp), %ecx
7 movl %eax, (%ecx)
8 movl %edx, 4(%ecx)

```

请给出上述每条汇编指令的注释，并说明 num\_type 是什么类型。

11. 已知 IA-32 是小端方式处理器，根据给出的 IA-32 机器代码的反汇编结果（部分信息用 x 表示）回答问题。

- (1) 已知 je 指令的操作码为 01110100，je 指令的转移目标地址是什么？call 指令中的转移目标地址 0x80483b1 是如何反汇编出来的？

```

804838c: 74 08 je xxxxxxxx
804838e: e8 1e 00 00 00 call 80483b1 <test >

```

- (2) 已知 jb 指令的操作码为 01110010，jb 指令的转移目标地址是什么？movl 指令中的目的地址是如何反汇编出来的？

```

8048390: 72 f6 jb xxxxxxxx
8048392: c6 05 00 a8 04 08 01 movl $0x1, 0x804a800
8048399: 00 00 00

```

- (3) 已知 jle 指令的操作码为 01111110，mov 指令的地址是什么？

```

xxxxxxx: 7e 16 jle 80492e0
xxxxxxx: 89 d0 mov %edx, %eax

```

- (4) 已知 jmp 指令的转移目标地址采用相对寻址方式，jmp 指令操作码为 11101001，其转移目标地址是什么？

```

8048296: e9 00 ff ff ff jmp xxxxxxxx
804829b: 29 c2 sub %eax, %edx

```

12. 已知函数 comp 的 C 语言代码及其过程体对应的汇编代码如下：



```

1 void comp(char x, int *p)
2 {
3 if(p && x<0)
4 *p += x;
5 }

```

```

1 movb 8(%ebp), %d1
2 movl 12(%ebp), %eax
3 testl %eax, %eax
4 je .L1
5 testb $0x80, %d1
6 je .L1
7 addb %d1, (%eax)
8 .L1:

```

要求回答下列问题或完成下列任务。

- (1) 给出每条汇编指令的注释, 并说明为什么 C 代码只有一个 if 语句而汇编代码有两条条件转移指令。
- (2) 按照图 3.22 给出的“if()goto...”语句形式写出汇编代码对应的 C 语言代码。

13. 已知函数 func 的 C 语言代码框架及其过程体对应的汇编代码如下, 根据对应的汇编代码填写 C 代码中缺失的表达式。

```

1 int func(int x, int y)
2 {
3 int z = ____;
4 if(____) {
5 if(____)
6 z = ____;
7 else
8 z = ____;
9 } else if(____)
10 z = ____;
11 return z;
12 }

```

```

1 movl 8(%ebp), %eax
2 movl 12(%ebp), %edx
3 cmpl $-100, %eax
4 jg .L1
5 cmpl %eax, %edx
6 jle .L2
7 addl %edx, %eax
8 jmp .L3
9 .L2:
10 subl %edx, %eax
11 jmp .L3
12 .L1:
13 cmpl $16, %eax
14 jl .L4
15 andl %edx, %eax
16 jmp .L3
17 .L4:
18 imull %edx, %eax
19 .L3:

```

14. 已知函数 do\_loop 的 C 语言代码如下:

```

1 short do_loop(short x, short y, short k) {
2 do {
3 x*=(y%k);
4 k--;
5 } while ((k>0) && (y>k));
6 return x;
7 }

```

函数 do\_loop 的过程体对应的汇编代码如下:

```

1 movw 8(%ebp), %bx
2 movw 12(%ebp), %si
3 movw 16(%ebp), %cx
4 .L1:
5 movw %si, %dx
6 movw %dx, %ax
7 sarw $15, %dx

```

```

8 idiv %cx
9 imulw %dx, %bx
10 decw %cx
11 testw %cx, %cx
12 jle .L2
13 cmpw %cx, %si
14 jg .L1
15 .L2:
16 movswl %bx, %eax

```

请回答下列问题或完成下列任务：

- (1) 给每条汇编指令添加注释，并说明每条指令执行后，目的寄存器中存放的是什么信息？
  - (2) 上述函数过程体中用到了哪些被调用者保存寄存器和哪些调用者保存寄存器？在该函数过程体前面的准备阶段哪些寄存器必须保存到栈中？
  - (3) 为什么第7行中的 DX 寄存器需要算术右移 15 位？
15. 已知函数 f1 的 C 语言代码框架及其过程体对应的汇编代码如下，根据对应的汇编代码填写 C 代码中缺失的部分，并说明函数 f1 的功能。

```

1 int f1(unsigned x)
2 {
3 int y = 0 ;
4 while (_____){
5 _____;
6 }
7 return _____;
8 }

```

```

1 movl 8(%ebp), %edx
2 movl $0, %eax
3 testl %edx, %edx
4 je .L1
5 .L2:
6 xorl %edx, %edx
7 shr $1, %edx
8 jne .L2
9 .L1:
10 andl $1, %eax

```

16. 已知函数 sw 的 C 语言代码框架如下：

```

int sw(int x) {
 int v=0;
 switch (x) {
 /* switch 语句中的处理部分省略 */
 }
 return v;
}

```

对函数 sw 进行编译，得到函数过程体中开始部分的汇编代码以及跳转表如下：

```

1 movl 8(%ebp), %eax
2 addl $3, %eax
3 cmpl $7, %eax
4 ja .L7
5 jmp *.L8(, %eax, 4)
6 .L7:
7
8

```

```

1 .L8:
2 .long .L7
3 .long .L2
4 .long .L2
5 .long .L3
6 .long .L4
7 .long .L5
8 .long .L7
9 .long .L6

```

回答下列问题：函数 sw 中的 switch 语句处理部分标号的取值情况如何？标号的取值在什么

情况下执行 default 分支? 哪些标号的取值会执行同一个 case 分支?

17. 已知函数 test 的入口参数有  $a$ 、 $b$ 、 $c$  和  $p$ , C 语言过程体代码如下:

```
*p = a;
return b*c;
```

函数 test 过程体对应的汇编代码如下:

```
1 movl 20(%ebp), %edx
2 movsbw 8(%ebp), %ax
3 movw %ax, (%edx)
4 movzwl 12(%ebp), %eax
5 movzwl 16(%ebp), %ecx
6 mull %ecx, %eax
```

写出函数 test 的原型, 给出返回参数的类型以及入口参数  $a$ 、 $b$ 、 $c$  和  $p$  的类型和顺序。

18. 已知函数 funct 的 C 语言代码如下:

```
1 int funct(viod) {
2 int x, y;
3 scanf("%x %x", &x, &y);
4 return x - y;
5 }
```

函数 funct 对应的汇编代码如下:

```
1 funct:
2 pushl %ebp
3 movl %esp, %ebp
4 subl $40, %esp
5 leal -8(%ebp), %eax
6 movl %eax, 8(%esp)
7 leal -4(%ebp), %eax
8 movl %eax, 4(%esp)
9 movl $.LC0, (%esp) // 将指向字符串"%x %x"的指针入栈
10 call scanf // 假定 scanf 执行后 x=15, y=20
11 movl -4(%ebp), %eax
12 subl -8(%ebp), %eax
13 leave
14 ret
```

假设函数 funct 开始执行时  $R[\text{esp}] = 0\text{xbc}000020$ ,  $R[\text{ebp}] = 0\text{xbc}000030$ , 执行第 10 行 call 指令后, scanf 从标准输入读入的值为  $0\text{x}16$  和  $0\text{x}100$ , 指向字符串 “%x %x” 的指针为  $0\text{x}804\text{c}000$ 。回答下列问题或完成下列任务。

- (1) 执行第 3、10 和 13 行的指令后, 寄存器 EBP 中的内容分别是什么?
- (2) 执行第 3、10 和 13 行的指令后, 寄存器 ESP 中的内容分别是什么?
- (3) 局部变量  $x$  和  $y$  所在存储单元的地址分别是什么?
- (4) 画出执行第 10 行指令后 funct 的栈帧, 指出栈帧中的内容及其地址。

19. 已知递归函数 refunc 的 C 语言代码框架如下:

```
1 int refunc(unsigned x) {
2 if (_____)
3 return _____ ;
4 unsigned nx = _____ ;
5 int rv = refunc(nx);
6 return _____ ;
7 }
```

上述递归函数过程体对应的汇编代码如下：

```
1 movl 8(%ebp), %ebx
2 movl $0, %eax
3 testl %ebx, %ebx
4 je .L2
5 movl %ebx, %eax
6 shrl $1, %eax
7 movl %eax, (%esp)
8 call refunc
9 movl %ebx, %edx
10 andl $1, %edx
11 leal (%edx, %eax), %eax
12 .L2:

 ret
```

根据对应的汇编代码填写 C 代码中缺失的部分，并说明函数的功能。

20. 填写下表，说明每个数组的元素大小、整个数组的大小以及第  $i$  个元素的地址。

| 数组                 | 元素大小 (B) | 数组大小 (B) | 起始地址  | 元素 $i$ 的地址 |
|--------------------|----------|----------|-------|------------|
| char A[10]         |          |          | &A[0] |            |
| int B[100]         |          |          | &B[0] |            |
| short *C[5]        |          |          | &C[0] |            |
| short **D[6]       |          |          | &D[0] |            |
| long double E[10]  |          |          | &E[0] |            |
| long double *F[10] |          |          | &F[0] |            |

21. 假设 short 型数组  $S$  的首地址  $A_S$  和数组下标（索引）变量  $i$ （int 型）分别存放在寄存器 EDX 和 ECX 中，下列给出的表达式的结果存放在 EAX 或 AX 中，仿照例子填写下表，说明表达式的类型、值和相应的汇编代码。

| 表达式            | 类型      | 值                 | 汇编代码                        |
|----------------|---------|-------------------|-----------------------------|
| $S$            |         |                   |                             |
| $S + i$        |         |                   |                             |
| $S[i]$         | short   | $M[A_S + 2 * i]$  | movw (%edx, %ecx, 2), %ax   |
| $\&S[10]$      |         |                   |                             |
| $\&S[i + 2]$   | short * | $A_S + 2 * i + 4$ | leal 4(%edx, %ecx, 2), %eax |
| $\&S[i] - S$   |         |                   |                             |
| $S[4 * i + 4]$ |         |                   |                             |
| $*(S + i - 2)$ |         |                   |                             |

22. 假设函数 sumij 的 C 代码如下，其中， $M$  和  $N$  是用#define 声明的常数。

```
1 int a[M][N], b[N][M];
2
3 int sumij(int i, int j) {
4 return a[i][j] + b[j][i];
5 }
```

已知函数 sumij 的过程体对应的汇编代码如下：

```
1 movl 8(%ebp), %ecx
```

```

2 movl 12(%ebp), %edx
3 leal (,%ecx, 8), %eax
4 subl %ecx, %eax
5 addl %edx, %eax
6 leal (%edx, %edx, 4), %edx
7 addl %ecx, %edx
8 movl a(, %eax, 4), %eax
9 addl b(, %edx, 4), %eax

```

根据上述汇编代码，确定  $M$  和  $N$  的值。

23. 假设函数 `st_ele` 的 C 代码如下，其中， $L$ 、 $M$  和  $N$  是用 `#define` 声明的常数。

```

1 int a[L][M][N];
2
3 int st_ele(int i, int j, int k, int *dst) {
4 *dst = a[i][j][k];
5 return sizeof(a);
6 }

```

已知函数 `st_ele` 的过程体对应的汇编代码如下：

```

1 movl 8(%ebp), %ecx
2 movl 12(%ebp), %edx
3 leal (%edx, %edx, 8), %edx
4 movl %ecx, %eax
5 sall $6, %eax
6 subl %ecx, %eax
7 addl %eax, %edx
8 addl 16(%ebp), %edx
9 movl a(, %edx, 4), %eax
10 movl 20(%ebp), %edx
11 movl %eax, (%edx)
12 movl $4536, %eax

```

根据上述汇编代码，确定  $L$ 、 $M$  和  $N$  的值。

24. 假设函数 `trans_matrix` 的 C 代码如下，其中， $M$  是用 `#define` 声明的常数。

```

1 void trans_matrix(int a[M][M]) {
2 int i, j, t;
3 for (i = 0; i < M; i++)
4 for (j = 0; j < M; j++) {
5 t = a[i][j];
6 a[i][j] = a[j][i];
7 a[j][i] = t;
8 }
9 }

```

已知采用优化编译（选项 -O2）后函数 `trans_matrix` 的内循环对应的汇编代码如下：

```

1 .L2:
2 movl (%ebx), %eax
3 movl (%esi, %ecx, 4), %edx
4 movl %eax, (%esi, %ecx, 4)
5 addl $1, %ecx
6 movl %edx, (%ebx)
7 addl $76, %ebx
8 cmpl %edi, %ecx
9 jl .L2

```

根据上述汇编代码，回答下列问题或完成下列任务。

- (1)  $M$  的值是多少？常数  $M$  和变量  $j$  分别存放在哪个寄存器中？

(2) 写出上述优化编译所生成的汇编代码对应的函数 `trans_matrix` 的 C 代码。

25. 假设结构类型 `node` 的定义、函数 `np_init` 的部分 C 代码及其对应的部分汇编代码如下：

```
struct node {
 int *p;
 struct {
 int x;
 int y;
 } s;
 struct node *next;
};
```

```
void np_init(struct node *np)
{
 np->s.x = _____ ;
 np->p = _____ ;
 np->next= _____ ;
}
```

```
movl 8(%ebp), %eax
movl 8(%eax), %edx
movl %edx, 4(%eax)
leal 4(%eax), %edx
movl %edx, (%eax)
movl %eax, 12(%eax)
```

回答下列问题或完成下列任务。

- (1) 结构 `node` 所需存储空间有多少字节？成员 `p`、`s.x`、`s.y` 和 `next` 的偏移地址分别为多少？
- (2) 根据汇编代码填写 `np_init` 中缺失的表达式。

26. 假设联合类型 `utype` 的定义如下：

```
typedef union {
 struct {
 int x;
 short y;
 short z;
 } s1
 struct {
 short a[2];
 int b;
 char *p;
 } s2
} utype;
```

若存在具有如下形式的一组函数：

```
void getvalue(utype *uptr, TYPE *dst) {
 *dst = EXPR;
}
```

该组函数用于计算不同表达式 `EXPR` 的值，返回值的数据类型根据表达式的类型确定。假设函数 `getvalue` 的入口参数 `uptr` 和 `dst` 分别被装入寄存器 `EAX` 和 `EDX` 中，仿照例子填写下表，说明在不同的表达式下的 `TYPE` 类型以及表达式对应的汇编指令序列（要求尽量只用 `EAX` 和 `EDX`，不够用时再使用 `ECX`）。

| 表达式 EXPR                                  | TYPE 类型          | 汇编指令序列                                                           |
|-------------------------------------------|------------------|------------------------------------------------------------------|
| <code>uptr-&gt;s1.x</code>                | <code>int</code> | <code>movl (%eax), %eax</code><br><code>movl %eax, (%edx)</code> |
| <code>uptr-&gt;s1.y</code>                |                  |                                                                  |
| <code>&amp;uptr-&gt;s1.z</code>           |                  |                                                                  |
| <code>uptr-&gt;s2.a</code>                |                  |                                                                  |
| <code>uptr-&gt;s2.a[uptr-&gt;s2.b]</code> |                  |                                                                  |
| <code>*uptr-&gt;s2.p</code>               |                  |                                                                  |

27. 给出下列各个结构类型中每个成员的偏移量、结构总大小以及在 `IA-32/Linux` 下结构起始位置的对齐要求。

- (1) struct S1 {short s; char c; int i; char d;;};
- (2) struct S2 {int i; short s; char c; char d;;};
- (3) struct S3 {char c; short s; int i; char d;;};
- (4) struct S4 {short s[3]; char c;};
- (5) struct S5 {char c[3]; short \*s; int i; char d; double e;;};
- (6) struct S6 {struct S1 c[3]; struct S2 \*s; char d;;};

28. 以下是结构 test 的声明:

```
struct {
 char c;
 double d;
 int i;
 short s;
 char *p;
 long l;
 long long g;
 void *v;
} test;
```

假设在 Windows 平台上编译, 则这个结构中每个成员的偏移量是多少? 结构总大小为多少字节? 如何调整成员的先后顺序使得结构所占空间最小?

29. 以下是函数 getline 存在漏洞和问题的 C 语言代码实现, 右边是其对应的反汇编部分结果:

```
char *getline()
{
 char buf[8];
 char *result;
 gets(buf);
 result=malloc(strlen(buf));
 strcpy(result, buf);
 return result;
}
```

```
1 0804840c <getline>:
2 804840c: 55 push %ebp
3 804840d: 89 e5 mov %esp, %ebp
4 804840f: 83 ec 28 sub $0x28, %esp
5 8048412: 89 5d f4 mov %ebx, -0xc(%ebp)
6 8048415: 89 75 f8 mov %esi, -0x8(%ebp)
7 8048418: 89 7d fc mov %edi, -0x4(%ebp)
8 804841b: 8d 75 ec lea 0x14(%ebp), %esi
9 804841e: 89 34 24 mov %esi, (%esp)
10 8048421: e8 a3 ff ff ff call 80483c9 <gets>
```

假定有一个调用过程 P 调用了函数 getline, 其返回地址为 0x80485c8, 在过程 P 中为了调用函数 getline 而执行 call 指令后, 部分寄存器的内容如下: R[ebp] = 0xbffc0800, R[esp] = 0xbffc07f0, R[ebx] = 0x5, R[esi] = 0x10, R[edi] = 0x8。执行程序时从标准输入读入的一行字符串为“0123456789ABCDEF0123456789A”, 此时, 程序会发生段错误 (segmentation fault) 并终止执行, 经调试确认错误是在执行 getline 的 ret 指令时发生的。回答下列问题或完成下列任务。

- (1) 分别画出执行第 7 行和第 10 行汇编指令后栈中的信息存放情况。要求给出存储地址和存储内容, 并指出存储内容的含义 (如返回地址、EBX 旧值、局部变量、入口参数等)。
- (2) 当执行到 getline 的 ret 指令时, 假如程序不发生段错误, 则正确的返回地址是什么? 发生段错误是因为执行 getline 的 ret 指令时得到了什么样的返回地址?
- (3) 执行完第 10 行汇编指令后, 哪些寄存器的内容已被破坏?
- (4) 除了可能发生缓冲区溢出以外, getline 的 C 代码还有哪些错误?

30. 假定函数 `abc` 的入口参数有 `a`、`b` 和 `c`，每个参数都可能是带符号整数类型或无符号整数类型，而且它们的长度也可能不同。该函数具有如下过程体：

```
*b += c;
*a += *b;
```

在 x86-64 机器上编译后的汇编代码如下：

```
1 abc:
2 addl (%rdx), %edi
3 movl %edi, (%rdx)
4 movslq %edi, %rdi
5 addq %rdi, (%rsi)
6 ret
```

分析上述汇编代码，以确定三个入口参数的顺序和可能的数据类型，写出函数 `abc` 可能的 4 种合理的函数原型。

31. 函数 `lproc` 的过程体对应的汇编代码如下：

```
1 movl 8(%ebp), %edx
2 movl 12(%ebp), %ecx
3 movl $255, %esi
4 movl $-2147483648, %edi
5 .L3:
6 movl %edi, %eax
7 andl %edx, %eax
8 xorl %eax, %esi
9 movl %ecx, %ebx
10 shrl %bl, %edi
11 testl %edi, %edi
12 jne .L3
13 movl %esi, %eax
```

上述代码根据以下 `lproc` 函数的 C 代码编译生成：

```
1 int lproc(int x, int k)
2 {
3 int val = _____;
4 int i;
5 for (i = _____; i _____; i = _____) {
6 val ^= _____;
7 }
8 return val;
9 }
```

回答下列问题或完成下列任务。

- (1) 给每条汇编指令添加注释。
  - (2) 参数 `x` 和 `k` 分别存放在哪个寄存器中？局部变量 `val` 和 `i` 分别存放在哪个寄存器中？
  - (3) 局部变量 `val` 和 `i` 的初始值分别是什么？
  - (4) 循环终止条件是什么？循环控制变量 `i` 是如何被修改的？
  - (5) 填写 C 代码中缺失的部分。
32. 假设你需要维护一个大型 C 语言程序，其部分代码如下：

```
1 typedef struct {
2 unsigned l_data;
3 line_struct x[LEN];
4 unsigned r_data;
```



```

5 } str_type;
6
7 void proc(int i, str_type *sptr) {
8 unsigned val = sptr->l_data + sptr->r_data;
9 line_struct *xptr = &sptr->x[i];
10 xptr->a[xptr->idx] = val;
11 }

```

编译时常量 LEN 以及结构类型 line\_struct 的声明都在一个你无权访问的文件中,但是,你有代码的 .o 版本(可重定位目标)文件,通过 OBJDUMP 反汇编该文件后,得到函数 proc 对应的反汇编结果如下,根据反汇编结果推断常量 LEN 的值以及结构类型 line\_struct 的完整声明(假设其中只有成员 a 和 idx)。

|    |          |                      |                              |
|----|----------|----------------------|------------------------------|
| 1  | 00000000 | <proc >:             |                              |
| 2  | 0:       | 55                   | push %ebp                    |
| 3  | 1:       | 89 e5                | mov %esp, %ebp               |
| 4  | 3:       | 53                   | push %ebx                    |
| 5  | 4:       | 8b 45 08             | mov 0x8(%ebp), %eax          |
| 6  | 7:       | 8b 4d 0c             | mov 0xc(%ebp), %ecx          |
| 7  | a:       | 6b d8 1c             | imul \$0x1c, %eax, %ebx      |
| 8  | d:       | 8d 14 c5 00 00 00 00 | lea 0x0(, %eax, 8), %edx     |
| 9  | 14:      | 29 c2                | sub %eax, %edx               |
| 10 | 16:      | 03 54 19 04          | add 0x4(%ecx, %ebx, 1), %edx |
| 11 | 1a:      | 8b 81 c8 00 00 00 00 | mov 0xc8(%ecx), %eax         |
| 12 | 20:      | 03 01                | add (%ecx), %eax             |
| 13 | 22:      | 89 44 91 08          | mov %eax, 0x8(%ecx, %edx, 4) |
| 14 | 26:      | 5b                   | pop %ebx                     |
| 15 | 27:      | 5d                   | pop %ebp                     |
| 16 | 28:      | c3                   | ret                          |

33. 假设嵌套的联合数据类型 node 声明如下:

```

1 union node {
2 struct {
3 int *ptr;
4 int data1;
5 } n1;
6 struct {
7 int data2;
8 union node *next;
9 } n2;
10 };

```

有一个进行链表处理的过程 chain\_proc 的部分 C 代码如下:

```

1 void chain_proc(union node *uptr) {
2 uptr->_____ = *(uptr->_____) - uptr->_____;
3 }

```

过程 chain\_proc 的过程体对应的汇编代码如下:

```

1 movl 8(%ebp), %edx
2 movl 4(%edx), %ecx
3 movl (%ecx), %eax
4 movl (%eax), %eax
5 subl (%edx), %eax
6 movl %eax, 4(%ecx)

```

回答下列问题或完成下列任务。

- (1) node 类型中结构成员 n1.ptr、n1.data1、n2.data2、n2.next 的偏移量分别是多少?
- (2) node 类型总大小占多少字节?
- (3) 根据汇编代码写出 chain\_proc 的 C 代码中缺失的表达式。

34. 以下声明用于构建一棵二叉树:

```
1 typedef struct TREE *tree_ptr;
2 struct TREE {
3 tree_ptr left;
4 tree_ptr right;
5 long val;
6 };
```

有一个进行二叉树处理的函数 trace 的原型为 “long trace(tree\_ptr tptr);”, 其过程体对应的 x86-64 汇编代码 (64 位版本) 如下:

```
1 trace:
2 movl $0, %eax
3 testq %rdi, %rdi
4 je .L2
5 .L3:
6 movq 16(%rdi), %rax
7 movq (%rdi), %rdi
8 testq %rdi, %rdi
9 jne .L3
10 .L2:
11 rep // 在此相当于空操作指令, 避免使 ret 指令作为跳转目标指令
12 ret
```

回答下列问题或完成下列任务。

- (1) 函数 trace 的入口参数 tptr 通过哪个寄存器传递?
- (2) 写出函数 trace 完整的 C 语言代码。
- (3) 说明函数 trace 的功能。